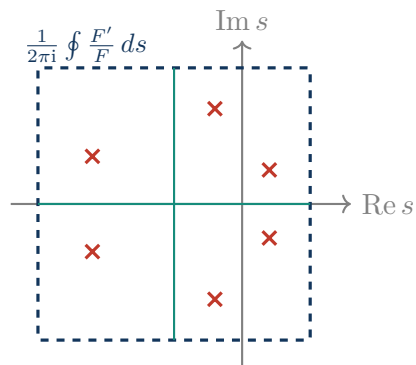


ContourRoots

Poles, zeros and roots of nonrational functions in MATLAB

Technical manual

Version 0.9.0



Flávio Luiz Cardoso-Ribeiro

<https://github.com/flavioluiz/ContourRoots>

MIT License

Abstract

ContourRoots is a MATLAB toolbox that computes the roots of scalar analytic functions, and the poles and zeros of scalar nonrational transfer functions, inside a rectangle of the complex plane. The original function is evaluated as given, without Padé approximation or modal truncation. Roots are counted with the argument principle, located by recursive subdivision and damped Newton iterations, and confirmed by local contour counts; pole-zero cancellations are resolved by counting the zeros of the numerator and denominator separately. Every result carries diagnostics that state whether the counts were reconciled, so that incomplete searches are reported rather than hidden. For scalar retarded time-delay systems, the toolbox also computes every critical delay and crossing direction directly and gives a bound-based count of unstable roots. This manual describes the mathematical background, the algorithm and its numerical safeguards, the interface, and three families of case studies: time-delay systems and the limits of Padé approximations, distributed-parameter systems, a beam coupled to an oscillator, and aeroelastic flutter with exact Theodorsen aerodynamics. Proofs are collected in the appendices.

Contents

1	Introduction	4
1.1	The problem	4
1.2	What the toolbox provides	4
1.3	What the toolbox does not claim	5
1.4	How to read this manual	5
2	Quick start	6
2.1	Installation	6
2.2	Roots of a characteristic function	6
2.3	Poles and zeros	6
2.4	Time-delay stability	7
3	Mathematical background	8
3.1	Analytic and meromorphic functions	8
3.2	The argument principle	8
3.3	Cancellation of poles and zeros	9
3.4	Modes and transfer poles	10
4	Algorithm and numerical safeguards	11
4.1	Contour counts	11
4.2	Newton's method	11
4.3	Local confirmation	12
4.4	Recursive subdivision	12
4.5	Cancellation	12
4.6	Exploratory mode	13
4.7	The completeness contract	13
4.8	Cost	13
4.9	Related methods and software	13
5	Model input and the analyticity contract	14
5.1	Input forms	14
5.2	The symbolic analyticity check	14
5.3	What a declaration of analyticity means	14
5.4	Singular points	15
5.5	Scaling	15
6	Interface reference	16
6.1	Functions	16
6.2	Options	17
6.3	Diagnostics	17

7	Time-delay systems and the limits of Padé approximations	18
7.1	Formulation	18
7.2	Structure of the spectrum	19
7.3	Roots for a given delay	20
7.4	Critical delays	20
7.5	Direction of the crossings	21
7.6	Stability map	21
7.7	The Padé approximation	22
7.8	What Padé preserves	24
7.9	What Padé loses	25
7.9.1	A non-conservative delay margin	25
7.9.2	Shifted and missing branches	25
7.9.3	Stability windows erased, shifted and invented	27
7.10	Guidelines	29
7.11	Limitations	29
8	Distributed-parameter systems	30
8.1	Removing apparent branch points	30
8.2	Heat equation: the sensor position	30
8.3	String and duct	31
8.4	Kelvin–Voigt beam: an accumulation point	32
8.5	Results	33
9	A beam coupled to an oscillator	34
9.1	Model	34
9.2	The bare-beam receptance	35
9.3	Characteristic function and transfer functions	35
9.4	Implementation	35
9.5	Reference case and parameter studies	36
9.6	Independent validation	38
10	Aeroelastic flutter without rational approximations	39
10.1	The typical section as a feedback system	39
10.2	Theodorsen’s function in the Laplace domain	40
10.3	Stability in three steps	41
10.4	Benchmarks	42
10.5	What rational approximations change	43
10.6	Parameter study	44
10.7	Validation and limitations	44
11	Time responses	45
11.1	Motivation	45
11.2	Problem statement and the domain assumption	45
11.3	Inversion on a Bromwich line	46
11.4	Dirac terms, the origin and jumps	47
11.5	Forced response and input reconstruction	48
11.5.1	Several inputs with the same model	48
11.6	Validation	49
11.7	Limitations	51

12 Flutter of a continuous wing with nothing discretized	52
12.1 Model	52
12.2 An exact characteristic function	53
12.3 Stability, root locus and flutter	54
12.4 A finite-element model converges to the continuous answer	54
12.5 Time responses	55
12.6 Scope	56
13 Building a wing from parts; MIMO responses	57
13.1 A strip as a hybrid two-port	57
13.2 Connection is not multiplication	57
13.3 Artificial poles of the hybrid form	58
13.4 Matrix models and simultaneous inputs	58
13.5 Verification	59
13.6 Scope	60
14 Matrix modes without determinants	61
14.1 Characteristic modes are not transfer poles	61
14.2 Counting without a product of pivots	61
14.3 Multiplicity, clusters and numerical limits	62
14.4 Determinant and continuous-wing comparisons	62
14.5 The flutter mode shape	63
15 Validation and limitations	65
15.1 Test suite	65
15.2 Independent references in the studies	65
15.3 Limitations	66
A Proofs	67
B Reproducibility	68

Chapter 1

Introduction

1.1 The problem

Many questions about linear dynamical systems reduce to finding the values of a complex variable s where a scalar function vanishes. The modes of a system are the roots of its characteristic function $\Delta(s)$; the poles and zeros of a transfer function $G(s) = N(s)/D(s)$ are the roots of D and N that survive cancellation. When Δ , N and D are polynomials, the problem is solved by computing matrix eigenvalues, as MATLAB's `roots`, `pole` and `zero` do.

Two large classes of models are not rational:

- systems with *time delays*, whose characteristic functions contain terms such as e^{-sT} [5, 15, 16, 21];
- *distributed-parameter* systems, described by partial differential equations, whose transfer functions contain $\sinh$, $\cosh$, $\sqrt{s}$ and similar functions [10];
- *aeroelastic* systems, whose unsteady aerodynamics involve Theodorsen's function, a ratio of Bessel functions [13].

Their characteristic functions typically have infinitely many roots, and no finite matrix contains all of them. A common practice is to replace the nonrational terms by rational approximations, for example Padé approximations of e^{-sT} , rational approximations of unsteady aerodynamics, or truncated modal expansions. The approximation makes the finite-dimensional toolbox available again, but it changes the question: one then computes the roots of a different function. Chapter 7 shows, on simple examples, that the answer can change qualitatively.

ContourRoots answers the original question in a bounded region: *which roots does the function have inside this rectangle?* It evaluates the function as it is given, and uses the argument principle to count how many roots each part of the rectangle contains, which makes it possible to tell whether the search was complete.

1.2 What the toolbox provides

- `croots`, `cpoles`, `czeros` and `cpzmap`, a MATLAB-style interface for roots, poles, zeros and pole-zero maps of scalar functions given as function handles, coefficient vectors, numerator/denominator pairs (`ndpair`), symbolic expressions or SISO LTI models.
- Detection of pole-zero cancellations, including partial ones.
- Diagnostics for every search: a status, the multiplicity and residual of each location, cancelled locations and unresolved regions.
- For scalar retarded time-delay systems $D(s) + N(s)e^{-sT} = 0$: all critical delays and crossing directions up to a given delay, without approximation, and a count of unstable roots in a

region that provably contains all of them.

- Examples, reproducible studies and tests.

1.3 What the toolbox does not claim

The word *exact* is used in this manual only in one sense: the original function is evaluated, without rational approximation or truncation. The computed roots are floating-point numbers. A search that ends with the status `numerically_complete` has passed careful numerical checks under the analyticity assumption, but it is not a proof in the sense of interval arithmetic (Section 4.7). The toolbox does not treat matrix-valued characteristic functions, does not enumerate infinite spectra, and does not discover singularities hidden inside opaque function handles. No claim of superiority over other root finders is made; related methods and software are discussed in Section 4.9.

1.4 How to read this manual

Chapter 2 is a short tour of the interface. Chapters 3–5 give the mathematical background, the algorithm, and the precise meaning of each input form and of the analyticity assumption. Chapter 6 is a compact reference. Chapters 7–10 are case studies that can be read independently: time-delay systems and the limits of Padé approximations; distributed-parameter systems; a beam coupled to an oscillator; and aeroelastic flutter with exact Theodorsen aerodynamics. Chapter 11 computes impulse, step and forced time responses of the same transfer functions by numerical Laplace inversion. Chapter 12 combines both for a continuous flexible wing: poles, flutter and time responses with neither the structure nor the aerodynamics discretized or approximated. Chapter 15 summarizes the validation, and the appendices contain proofs and reproducibility information.

The Markdown documentation distributed with the toolbox covers the same material in a tutorial style, with runnable code for every step.

Chapter 2

Quick start

2.1 Installation

ContourRoots is plain MATLAB code; the core needs no additional toolbox. Download ContourRoots.mltbx from the release page and open it to install it as an Add-On, or download ContourRoots.zip, unzip it and run `setup_contourroots`, which adds the toolbox to the path for the current session. From the Command Window:

```
url = "https://github.com/flavioluiz/ContourRoots/releases/latest/download/
      ContourRoots.zip";
unzip(websave("ContourRoots.zip", url), "ContourRoots");
run("ContourRoots/setup_contourroots.m")
```

2.2 Roots of a characteristic function

The characteristic function $F(s) = s^2 + s + 1 + (2s + 3)e^{-s}$ has infinitely many roots. Those in $-8 < \operatorname{Re} s < 2$, $-20 < \operatorname{Im} s < 20$ are obtained with

```
F = @(s) s.^2 + s + 1 + (2*s + 3).*exp(-s);
[r, info] = croots(F, [-8 2 -20 20], 'AssumeAnalytic', true);
```

The vector `r` contains seven roots, ordered by decreasing real part; the first two, $0.36201 \pm 1.8195i$, lie in the right half-plane. `info.status` is `'numerically_complete'`. The option `'AssumeAnalytic', true` is the user's statement that F is analytic on a neighborhood of the closed rectangle; it is justified here because F is built from polynomials and the exponential. Without it the search is exploratory (Section 4.6).

2.3 Poles and zeros

A transfer function is given as a pair of analytic functions:

```
G = ndpair(@(s) sinh(s/2), @(s) sinh(s));
p = cpoles(G, [-1 1 -10 10], 'AssumeAnalytic', true); % +-i*pi, +-3i*pi
z = czeros(G, [-1 1 -10 10], 'AssumeAnalytic', true); % empty
cpzmap(G, [-1 1 -10 10], 'AssumeAnalytic', true)
```

The denominator vanishes at $i\pi k$ for every integer k ; for even k the numerator vanishes as well and the candidate cancels. Figure 2.1 shows both results.

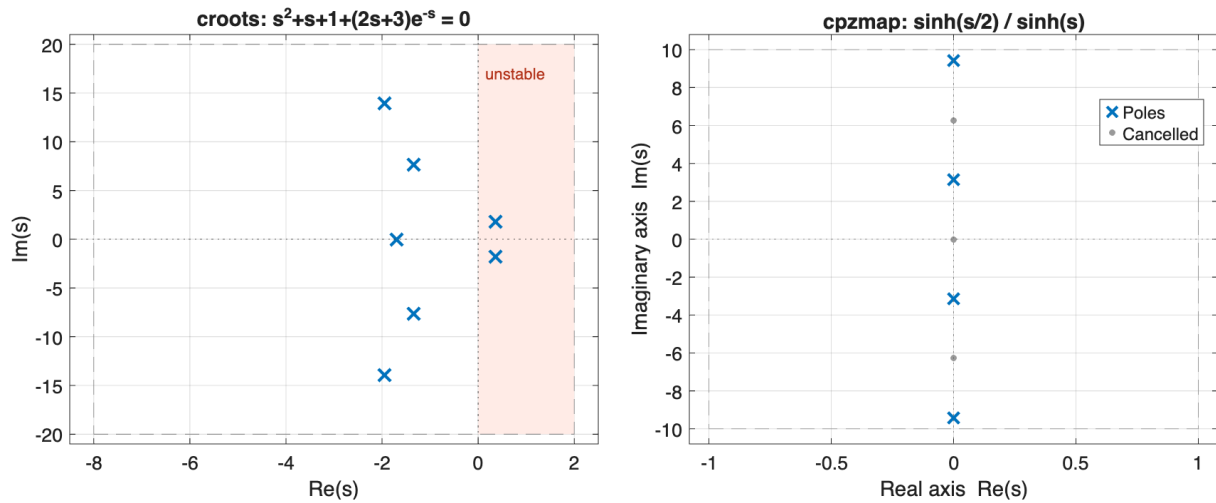


Figure 2.1: Left: roots of $s^2 + s + 1 + (2s + 3)e^{-s}$ in the rectangle $[-8, 2] \times [-20, 20]$ (dashed); the shaded area is the right half-plane. Right: cpzmap of $\sinh(s/2)/\sinh(s)$; grey dots are cancelled candidates.

2.4 Time-delay stability

For $D(s) + N(s)e^{-sT} = 0$ with coefficient vectors $\mathbf{N}$ and $\mathbf{D}$:

```
crit = critical_delays(2, [1 1], 10);           % s + 1 + 2 e^{-sT} = 0
Z = unstable_root_count(2, [1 1], 1.5);        % 2 unstable roots
```

The first critical delay is $2\pi/(3\sqrt{3}) \approx 1.2092$ (Chapter 7).

Chapter 3

Mathematical background

This chapter collects the facts from complex analysis on which the method relies; see Ahlfors [1] for proofs of the classical results.

3.1 Analytic and meromorphic functions

A function F of the complex variable $s = x + iy$ is *analytic* (holomorphic) on an open set U if it has a complex derivative at every point of U . It is *entire* if it is analytic on the whole plane. Polynomials, e^s , $\sin s$, $\cos s$, $\sinh s$, $\cosh s$, and all sums, products and compositions of entire functions are entire. A function is *meromorphic* on U if it is analytic on U except at isolated poles.

Two properties of analytic functions are essential:

1. The zeros of an analytic function that is not identically zero are isolated, and each has a finite *multiplicity* $m \geq 1$: $F(s) = (s - s_0)^m g(s)$ with g analytic and $g(s_0) \neq 0$.
2. If F is analytic on a neighborhood of a closed bounded region and not identically zero, that region contains finitely many zeros. The hypothesis on the closure matters: zeros may accumulate at a singular point on the boundary of a region (see Section 8.4).

Neither holds for functions that are not analytic: $|s|^2 - 1$ vanishes on a whole circle. Functions with branch points (such as $\sqrt{s}$ or $\log s$) are analytic only on a domain that excludes a cut.

Some expressions that contain square roots are nevertheless entire. For instance $\cosh \sqrt{q} = \sum_k q^k / (2k)!$ and $\sinh(\sqrt{q}) / \sqrt{q} = \sum_k q^k / (2k + 1)!$ contain only integer powers of q : the choice of the branch of $\sqrt{q}$ does not change their value. Such *even functions of a root* appear in the transfer functions of Chapters 8 and 9.

3.2 The argument principle

Theorem 3.1 (Argument principle). *Let C be a simple closed piecewise-smooth curve, oriented counterclockwise, and let F be meromorphic on a neighborhood of C and its interior, with no zero or pole on C . Then*

$$\frac{1}{2\pi i} \oint_C \frac{F'(s)}{F(s)} ds = Z - P = \frac{1}{2\pi} \Delta_C \arg F(s), \quad (3.1)$$

where Z and P are the numbers of zeros and poles inside C , counted with multiplicity, and $\Delta_C \arg F$ is the total change of a continuous branch of $\arg F(s)$ as s travels once around C .

The right-hand form is the one used numerically: the change of argument is the sum of the phase increments of F between consecutive samples of C , each taken in $(-\pi, \pi]$. It equals 2π times the *winding number* of the image curve $F(C)$ around the origin (Figure 3.1).

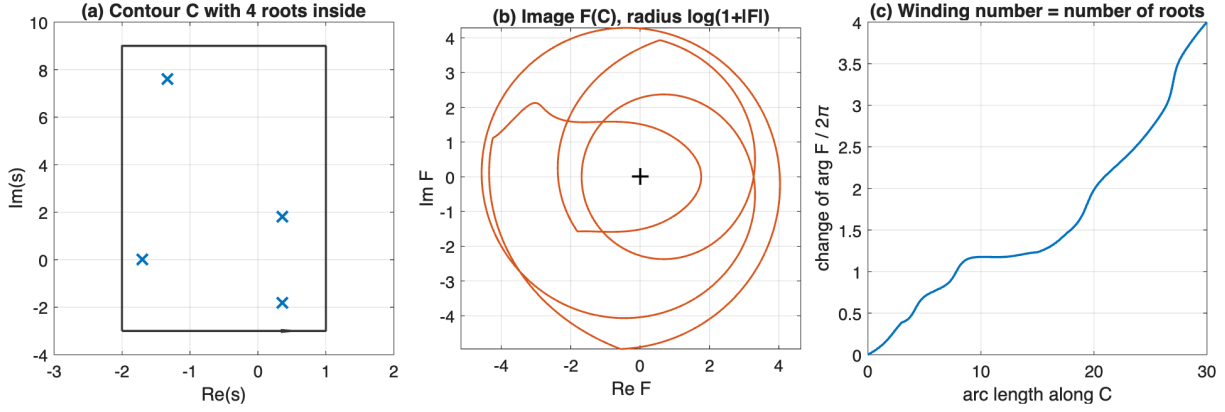


Figure 3.1: The argument principle for $F(s) = s^2 + s + 1 + (2s + 3)e^{-s}$ on the rectangle $[-2, 1] \times [-3, 9]$. (a) Four roots lie inside C . (b) The image curve $F(C)$, drawn with radius $\log(1 + |F|)$, winds four times around the origin. (c) The unwrapped argument of F along C grows by $4 \times 2\pi$.

Three consequences shape the design of ContourRoots.

Analytic functions. If F is analytic, $P = 0$ and the integral counts the zeros. This is the case for characteristic functions such as $D(s) + N(s)e^{-sT}$.

Quotients. For a transfer function $G = N/D$ with N and D analytic, the integral of G'/G counts $Z_N - Z_D$ after cancellation. A count of zero does not mean that the region is empty. This is why ContourRoots asks for N and D separately (`ndpair`) and counts the zeros of each.

Multiplicity and the first moment. If F is analytic with zeros $s_1, \dots, s_Z$ (repeated according to multiplicity) inside C ,

$$\frac{1}{2\pi i} \oint_C s \frac{F'(s)}{F(s)} ds = \sum_{j=1}^Z s_j. \quad (3.2)$$

Divided by Z it gives the centroid of the zeros; when $Z = 1$ it gives the zero itself. Higher moments lead to the classical methods of Delves and Lyness [12] and Kravanja and Van Barel [19], which compute all zeros inside C from a small eigenvalue problem. ContourRoots uses only the first moment, as a starting point for Newton's method.

3.3 Cancellation of poles and zeros

Let N and D be analytic near s_0 , with zeros of orders $m_N \geq 0$ and $m_D \geq 0$ at s_0 . Then $G = N/D$ has at s_0

- a pole of order $m_D - m_N$ if $m_D > m_N$;
- a zero of order $m_N - m_D$ if $m_N > m_D$;
- neither (a removable point, after cancellation) if $m_N = m_D$.

Counting the zeros of N and of D separately in a small disk around each candidate determines m_N and m_D , and therefore the type and order of the point. This is the cancellation test of Section 4.5.

3.4 Modes and transfer poles

For a linear system, the roots of the characteristic function are the *modes*; they decide internal stability. A transfer function from one input to one output may not contain every mode: a mode that the input cannot excite, or that the output cannot observe, appears as a common zero of N and D and cancels. Both computations are available: `croots` on the characteristic function returns the modes, and `cpoles` on N/D returns the transfer poles. Examples of hidden modes appear in Sections 8.2 and 9.6.

Chapter 4

Algorithm and numerical safeguards

The solver, `complex_spectrum`, isolates the zeros of an analytic function f in a rectangle $\Omega = [x_0, x_1] \times [y_0, y_1]$. In zero mode $f = N$ (or the given function); in pole mode $f = D$. This chapter describes each component and the conditions under which a result is declared complete.

4.1 Contour counts

The boundary of a rectangle B is sampled at n equally spaced points per edge, $4n$ points in total, traversed counterclockwise. With $v_j = f(z_j)$, the phase and log-modulus increments are

$$\delta_j = \text{Arg} \frac{v_{j+1}}{v_j} \in (-\pi, \pi], \quad \mu_j = \log |v_{j+1}| - \log |v_j|,$$

and the discrete winding number is $w = \frac{1}{2\pi} \sum_j \delta_j$. The number of samples starts at $n = n_0$ (`ContourPoints`, default 32) and is doubled up to `ContourRefinements` times (default 7, i.e. up to 4096 points per edge). A count is *accepted* when

1. the rounded winding number $\text{round}(w)$ is the same at three consecutive sampling levels,
2. at the last two levels $|w - \text{round}(w)| < 10^{-7}$, and
3. at the last two levels every increment satisfies $|\delta_j| < \pi/3$ and $|\mu_j| < 2$.

Condition 3 is the practical form of the requirement that the argument of f vary slowly between samples; without it, a fast phase rotation could be aliased and a whole turn lost. If any sample is zero, infinite or NaN (a zero on the boundary, or overflow), or if the conditions are not met at the finest level, the count fails and the rectangle is *unresolved*.

The same samples give the first-moment estimate (3.2), computed as $\sum_j \frac{z_j + z_{j+1}}{2} (\mu_j + i\delta_j) / (2\pi i Z)$, used as a Newton starting point.

4.2 Newton's method

From a starting point z inside a cell B with count m , the solver applies the modified Newton iteration

$$z \leftarrow z - \alpha m \frac{f(z)}{f'(z)},$$

which converges quadratically to a zero of multiplicity m . Safeguards:

- the step is limited to half the larger side of B ;
- the damping factor α is halved (at most 12 times) until $|f|$ does not increase; if it cannot be decreased, the start is abandoned;

- the iteration stops if it leaves B , and converges when the step is below $\text{RootTolerance} \cdot (1 + |z|)$.

The derivative is the user-supplied `Derivative`, the exact derivative for symbolic input and coefficient vectors, or otherwise the four-point complex difference

$$f'(z) \approx \frac{f(z+h) - f(z-h) - i[f(z+ih) - f(z-ih)]}{4h}, \quad h = 10^{-4} \max(1, |z|),$$

whose error is $O(h^4)$ for analytic f (the h^2 terms of the real and imaginary central differences cancel).

4.3 Local confirmation

A Newton limit z^* is accepted only if a contour count on the small square of half-width $\rho = \text{RootTolerance} \cdot (1 + |z^*|)$ centered at z^* equals the count m of the cell. This confirms both that z^* is a zero and that no other zero of the cell was missed; m becomes the multiplicity of z^* , and $\sqrt{2}\rho$ is reported as `locationRadius`. Consequently, zeros closer than about ρ are reported as one location whose multiplicity is the size of the cluster.

4.4 Recursive subdivision

Algorithm 1 — recursive isolation of the zeros of f in a cell B with count m

1. If $m = 0$, return.
2. For each start in (first-moment center of B , geometric center of B , user seeds inside B): run Newton (Section 4.2); if the limit is confirmed with count m (Section 4.3), record it and return.
3. If the depth exceeds `MaxDepth` or the number of visited cells exceeds `MaxCells`, mark B unresolved and return.
4. Split B across its longer side at the fraction 0.5 of the side. Count the two halves. If both counts are accepted and add up to m , recurse into both halves and return.
5. Otherwise retry with the split fractions 0.438... and 0.573..., then with the other side. If no split conserves the count, mark B unresolved.

The *conservation check* of step 4 detects zeros lying on the split line (the counts of the halves then fail or do not add up) and moves the line. The irrational-looking fractions make it unlikely that a second attempt hits a zero again. Unresolved cells are never discarded: they are returned in `info.unresolvedBoxes`.

4.5 Cancellation

In pole mode with a pair N/D , the zeros of D are isolated with Algorithm 1. For each location z^* with multiplicity m_D , the zeros of N are counted on the same small square, giving m_N . The reported multiplicity is $\max(m_D - m_N, 0)$; if it is zero, z^* is moved to `cancelledLocations`, and the removed order $\min(m_D, m_N)$ is recorded. Zero mode is symmetric. If the count of the other factor fails, the location is kept and flagged in `cancellationUncertain`, and the result is not complete. The test has the resolution of the small square: a pole and a zero closer than ρ are indistinguishable from an exact cancellation.

4.6 Exploratory mode

When the function is not known to be analytic (an opaque handle without `AssumeAnalytic`, or a single quotient handle), contour counts may measure $Z - P$ instead of Z and cannot be trusted. The solver then starts Newton from a `GridSize` grid of points and from the seeds, and keeps the limits confirmed by a positive local count. The result has status `exploratory` and is never declared complete.

4.7 The completeness contract

A search is `numerically_complete` when (i) the count on $\partial\Omega$ was accepted, (ii) every cell was resolved, (iii) the multiplicities of the located zeros add up to that count, and (iv) every cancellation test was resolved. Under the analyticity assumption, and if the accepted counts are correct, all zeros in Ω are then reported, each within its `locationRadius` as a cluster of the stated multiplicity.

What this is not

It is not a proof. The counts are computed from finitely many floating-point samples; a function that oscillates between samples faster than the safeguards of Section 4.1 can detect could in principle fool them. Analyticity of a handle is the user's declaration. Residuals depend on the scaling of the searched factor and are not bounds on the location error. `info.certified` is always false. Rigorous enclosures would require interval arithmetic, which is outside the scope of this version.

4.8 Cost

The cost is dominated by function evaluations on contours: $4n$ per count, with n doubling until acceptance, for the outer rectangle, for every cell of the subdivision and for every local confirmation. Vectorized handles are evaluated on all samples of a contour at once. Tall rectangles over oscillatory functions (e.g. e^{-sT} with large $T|\operatorname{Im} s|$) need more samples per edge; smaller rectangles or larger `ContourRefinements` help. Seeds from a previous parameter value reduce the number of subdivisions in parameter sweeps.

4.9 Related methods and software

Contour-integral root finders go back to Delves and Lyness [12]; see Kravanja and Van Barel [19] for a thorough treatment including moment-based eigenvalue formulations. `cxroots` [23] implements such a method in Python with subdivision. GRPF [18] analyzes the phase of a single meromorphic function on an adaptive triangulation and locates both zeros and poles. For quasi-polynomials and time-delay systems, QPmR [27] maps the zero-level curves of the real and imaginary parts, and spectral discretization methods [6, 7, 21] approximate the rightmost roots of matrix delay equations. ContourRoots combines well-known ingredients; its emphasis is on a MATLAB-style interface, explicit cancellation for N/D pairs, and diagnostics that make incomplete searches visible.

Chapter 5

Model input and the analyticity contract

The algorithm of Chapter 4 is only as good as its assumption that the searched factor is analytic on a neighborhood of the closed rectangle. This chapter lists the accepted input forms and states, for each, how that assumption is established.

5.1 Input forms

Input	Treatment
Coefficient vector	Polynomial in descending powers, exact derivative; analytic by construction. <code>croots(c,region)</code> behaves like <code>roots(c)</code> restricted to the rectangle.
Function handle	Evaluated as given. Analytic only if the user passes <code>AssumeAnalytic</code> ; otherwise exploratory. Vectorized handles are evaluated on whole contours; scalar-only handles are evaluated point by point.
<code>ndpair(N,D)</code>	Each factor is a handle, a coefficient vector or a symbolic expression. Polynomial and checked symbolic factors are analytic automatically; handles need <code>AssumeAnalytic</code> , which then applies to <i>both</i> factors.
Symbolic (<code>sym</code> , <code>symfun</code>)	Split with <code>numden</code> ; each factor is checked by a conservative grammar (below) and differentiated exactly.
<code>tf</code> , <code>zpk</code> , <code>ss</code>	Continuous-time SISO only. Rational models and <code>tf</code> models with input, output or transport delays become an analytic pair (delay in the numerator). Models with internal delays are evaluated with <code>evalfr</code> in exploratory mode.

5.2 The symbolic analyticity check

A symbolic factor is accepted as entire if it is a number, the variable, or is built from entire factors by sums, products, exp, sin, cos, sinh, cosh and powers with constant nonnegative integer exponents. The check is *sufficient, not necessary*: $\cosh \sqrt{s}$ is entire but is rejected, with the error `complex_spectrum:AnalyticContract`. The remedy is to provide the factor as a handle that evaluates the entire function directly (with a series near removable points, Chapter 8) and to declare `AssumeAnalytic`. All parameters other than the complex variable must be substituted numerically before the call.

5.3 What a declaration of analyticity means

`'AssumeAnalytic',true` asserts that the searched factor(s) are analytic on an open set containing the closed rectangle. It is a statement about mathematics, not about the code, and must be checked by the user:

- removable singularities (e.g. $\sinh(s)/s$ at 0) must be removed in the implementation, otherwise the evaluation returns `NaN`;
- roots such as $\sqrt{s}$ or $s^{1/4}$ are acceptable only inside combinations invariant under the change of branch;
- for a *single* handle in pole mode, the declaration concerns $1/G$: it asserts that G has no zeros in the rectangle. For transfer functions with zeros, use `ndpair`.

A false declaration makes the counts meaningless and can produce wrong results without any warning.

5.4 Singular points

Some models have points where the function is not meromorphic: branch points of genuinely multivalued functions, or accumulation points of poles (Section 8.4). No finite contour computation can resolve a neighborhood of such a point. The option `Singularities` declares them; a rectangle that contains one is rejected with the error `complex_spectrum:SingularityInRegion`. The solver does not discover such points in opaque handles.

5.5 Scaling

The argument principle is invariant under multiplication of f by a nonzero constant, and the regression tests check that roots do not change when f is scaled by $10^{\pm 100}$. Overflow is a different matter: a term such as e^{-sT} grows like $e^{T|\operatorname{Re} s|}$ in the left half-plane, and samples that overflow make a count fail. Rectangles extending far to the left may need to be split, or the function rescaled by an analytic nonvanishing factor.

Chapter 6

Interface reference

The complete reference, with examples for every function, is in `docs/api/` and in the MATLAB help (`help croots`). This chapter summarizes it.

6.1 Functions

Syntax	Purpose
<code>[r,info] = croots(F,region,...)</code>	roots of F in the rectangle
<code>[p,info] = cpoles(G,region,...)</code>	poles of G after cancellation
<code>[z,info] = czeros(G,region,...)</code>	zeros of G after cancellation
<code>cpzmap(G,region,...)</code>	pole-zero map; <code>[p,z,infoP,infoZ]</code> without plot
<code>G = ndpair(N,D)</code>	transfer function from two analytic factors
<code>[x,info] = complex_spectrum(...)</code>	core solver, 'Mode' 'zeros'/'poles'
<code>crit = critical_delays(N,D,Tmax)</code>	crossings of $D + Ne^{-sT}$, $0 \leq T \leq T_{\max}$
<code>Z = unstable_root_count(N,D,T)</code>	roots with $\operatorname{Re} s \geq 0$ (bound-based count)
<code>R = rhp_root_bound(N,D)</code>	radius containing all such roots, all T
<code>delay_roots, delay_root_count</code>	grid/Newton solver and count for one delay
<code>pade_delay, pade_characteristic</code>	diagonal Padé coefficients and polynomial
<code>pade_critical_delays</code>	critical delays predicted by Padé models
<code>[g,t,info] = cimpulse(G,t,...)</code>	ordinary impulse response; Dirac terms in <code>info</code>
<code>[y,t,info] = cstep(G,t,...)</code>	unit-step response
<code>[y,t,info] = clsim(G,u,t,...)</code>	response to a sampled input (ZOH/FOH)
<code>K = ckernel(G,t,...)</code>	reusable kernels for <code>clsim(K,u,t,...)</code>
<code>[f,t,info] =</code> <code>cinvlaplace(F,t,...)</code>	inverse Laplace transform of $F(s)$
<code>setup_contourroots, contourroots</code>	path setup; overview and version

The region is always `[xmin xmax ymin ymax]`. When `info` is not requested and a search of `croots`, `cpoles`, `czeros` or `cpzmap` is not complete, a warning (`ContourRoots:Exploratory` or `ContourRoots:Incomplete`) is issued; `'Warn',false` silences it.

6.2 Options

Option	Default	Meaning
<code>AssumeAnalytic</code>	false	declare the searched factor(s) analytic
<code>SeedPoints</code>	<code>[]</code>	extra Newton starts
<code>Derivative</code>	<code>[]</code>	F' for a single handle
<code>Singularities</code>	<code>[]</code>	points that must not lie in the region
<code>RootTolerance</code>	1e-7	relative half-width of confirmation boxes
<code>ContourPoints</code>	32	initial samples per edge
<code>ContourRefinements</code>	7	maximum doublings of the sampling
<code>MaxDepth</code>	24	maximum subdivision depth
<code>MaxCells</code>	5000	maximum number of cells
<code>MaxIterations</code>	80	Newton iterations per start
<code>GridSize</code>	[25 41]	start grid in exploratory mode
<code>Plot, Display</code>	false	figure or table of results
<code>Warn</code>	true	incompleteness warning (wrappers only)

6.3 Diagnostics

Field	Meaning
<code>status</code>	<code>numerically_complete</code> , <code>unresolved</code> or <code>exploratory</code>
<code>complete</code>	conditions (i)–(iv) of Section 4.7 hold
<code>certified</code>	always false
<code>count</code>	total multiplicity returned, after cancellation
<code>targetCountBeforeCancellations</code>	outer count of the searched factor
<code>multiplicity</code>	multiplicity (cluster size) of each location
<code>locationRadius</code>	resolution of each location (not an error bound)
<code>residuals</code>	$ f $ of the searched factor at each location
<code>cancelledLocations,</code> <code>cancellationOrders</code>	removed candidates and orders
<code>cancellationUncertain</code>	cancellation test unresolved at a location
<code>unresolvedBoxes</code>	cells that could not be resolved
<code>contourResolved, cellsVisited</code>	outer count accepted; cells used
<code>exploratory, region, mode,</code> <code>analyticSource</code>	context of the search

Chapter 7

Time-delay systems and the limits of Padé approximations

This chapter studies the scalar characteristic equation

$$F(s, T) = D(s) + N(s) e^{-sT} = 0, \quad T \geq 0, \quad (7.1)$$

with real polynomials D of degree m and N of degree p . It shows that the stability of such systems can be decided for every delay without approximating the delay, and then examines precisely what a Padé approximation of e^{-sT} preserves and what it loses.

7.1 Formulation

The simplest example is the delay-differential equation

$$\dot{x}(t) = -a x(t) - b x(t - T). \quad (7.2)$$

Its solution from $t = 0$ is determined only by a whole *history* $x(\theta) = \varphi(\theta)$, $\theta \in [-T, 0]$: on $[0, T]$ the delayed term is known and (7.2) is an ordinary differential equation; its solution determines the delayed term on $[T, 2T]$, and so on (the *method of steps* [5]). The state is a function, and the system is infinite-dimensional. Exponential solutions e^{st} exist when $s + a + be^{-sT} = 0$. More generally, the loop of Figure 7.1, a rational plant N/D with delayed unit feedback, has the characteristic equation (7.1).

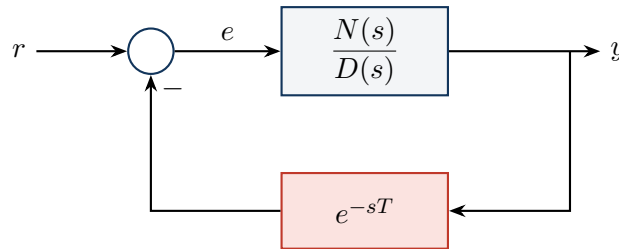


Figure 7.1: Loop with a delay in the feedback path. The loop transfer function is $L(s) = \frac{N(s)}{D(s)} e^{-sT}$; $1 + L = 0$ is equivalent to (7.1). The delay has unit gain at every frequency and unbounded phase $-\omega T$.

Definition 7.1. The system is *retarded* if $p < m$ and *neutral* if $p = m$.

Throughout this chapter $p < m$. Three cases illustrate the typical behaviors (Table 7.1).

Table 7.1: Case studies; all are stable at $T = 0$.

Case	Equation	Model	Behavior as T grows
P1	$s + 1 + 2e^{-sT} = 0$	$\dot{x} = -x(t) - 2x(t - T)$	loses stability at $T \approx 1.209$
P2	$s + 2 + e^{-sT} = 0$	$\dot{x} = -2x(t) - x(t - T)$	stable for every $T \geq 0$
P3	$s^2 + 0.8s + 4 + 3e^{-sT} = 0$	$\ddot{x} + 0.8\dot{x} + 4x = -3x(t - T)$	stability switches

7.2 Structure of the spectrum

$F(\cdot, T)$ is entire and, for $T > 0$, has infinitely many roots. For retarded systems they form chains that move to the left: balancing $|D(s)| \sim |s|^m$ with $|N(s)e^{-sT}| \sim |s|^p e^{-T \operatorname{Re} s}$ gives

$$\operatorname{Re} s \approx -\frac{m-p}{T} \ln |s|, \quad |s| \rightarrow \infty.$$

Hence every half-plane $\operatorname{Re} s > c$ contains finitely many roots, the *spectral abscissa* $\alpha(T) = \max\{\operatorname{Re} s : F(s, T) = 0\}$ is well defined, and the zero solution of a linear retarded system is exponentially stable if and only if $\alpha(T) < 0$ [16, 21]. Figure 7.2 shows the spectra of the three cases together with the poles of Padé models (Section 7.7).

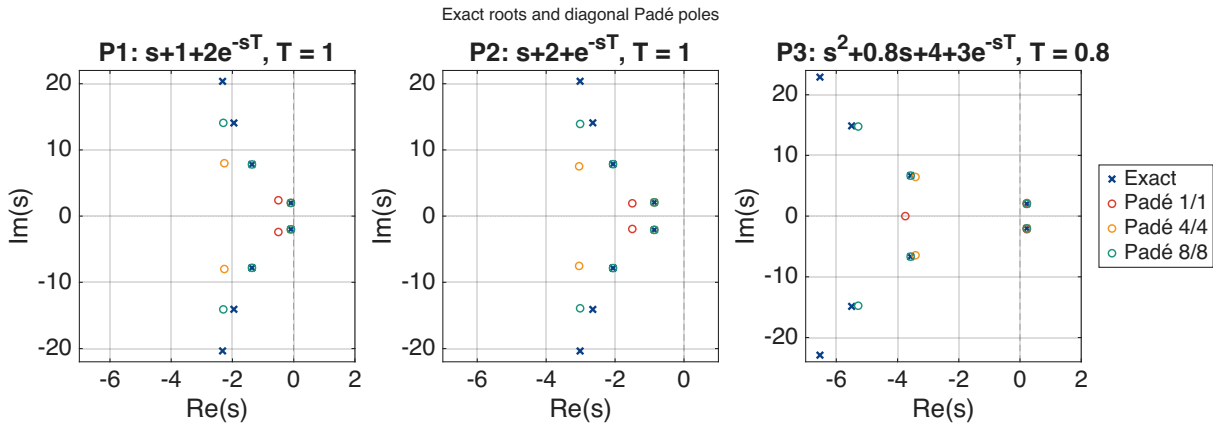


Figure 7.2: Roots of (7.1) ($\times$) and poles of diagonal Padé models ($\circ$) at $T = 1$ (P1, P2) and $T = 0.8$ (P3). The exact chains continue to the left; the rational spectra are finite.

Proposition 7.2 (A region containing all unstable roots). *Let $p < m$, D monic, $D(s) = s^m + \sum_{k < m} d_k s^k$, $N(s) = \sum_{k \leq p} n_k s^k$, and let $R > 0$ be the positive root of*

$$r^m - \sum_{k < m} |d_k| r^k - \sum_{k \leq p} |n_k| r^k = 0.$$

Then for every $T \geq 0$, every root of (7.1) with $\operatorname{Re} s \geq 0$ satisfies $|s| \leq R$.

The proof is in Appendix A. For P1, $|s + 1| \leq 2$ gives $R = 3$; for P3, $R \approx 3.076$. The function `rhp_root_bound` computes R , and `unstable_root_count` applies the argument principle to the rectangle $[0, R + 1] \times [-(R + 1), R + 1]$: the number $Z(T)$ of unstable roots is obtained without guessing a search window and without approximation (the count itself is a floating-point computation with adaptive sampling; since the left edge is the imaginary axis, it is inconclusive, and returns NaN, when roots lie on or extremely close to the axis — near critical delays, for persistent imaginary roots, or for large delays, where many roots approach the axis. $Z(T) = 0$ means stability only when no root lies on the imaginary axis).

7.3 Roots for a given delay

The study of this chapter locates roots with the delay-specific solver `delay_roots` (damped Newton from a grid, audited by an argument-principle count); `croots` gives the same roots with the subdivision algorithm of Chapter 4. Table 7.2 shows that the number of roots found agrees with the independent count and that the residuals are near machine precision.

Table 7.2: Solver diagnostics at the nominal delay.

Case	Roots	Count	Max residual	Abscissa
P1	14	14	1.13e-13	-0.09248
P2	14	14	4.46e-14	-0.86098
P3	12	12	2.18e-12	0.22028

7.4 Critical delays

As T varies the roots move continuously, and new roots appear only from $\operatorname{Re} s = -\infty$. Stability can therefore change only when a root crosses the imaginary axis. At a crossing $s = i\omega$, $\omega > 0$,

$$e^{-i\omega T} = -\frac{D(i\omega)}{N(i\omega)}. \quad (7.3)$$

This complex equation splits into two real equations with very different roles.

Modulus: where. The left side has unit modulus for every T , so the crossing frequencies do not depend on the delay and solve

$$G(\omega) = |D(i\omega)|^2 - |N(i\omega)|^2 = 0, \quad (7.4)$$

an even polynomial in ω , i.e. a polynomial of degree m in $x = \omega^2$. Its positive roots give a *finite* set of frequencies.

Phase: when. For each frequency ω_i , with $\theta_i = \operatorname{mod}(-\arg[-D(i\omega_i)/N(i\omega_i)], 2\pi)$, the phase condition gives *infinitely many* critical delays

$$T_{i,k} = \frac{\theta_i + 2\pi k}{\omega_i}, \quad k = 0, 1, 2, \dots \quad (7.5)$$

This elimination is the scalar form of the direct method [21, 22, 28]. `critical_delays` computes all $T_{i,k}$ in an interval and refines each pair (ω, T) by a two-variable Newton correction.

Example 7.3 (P1). $G(\omega) = 1 + \omega^2 - 4$ gives $\omega_c = \sqrt{3}$. Since $-D(i\sqrt{3})/N = e^{-2\pi i/3}$, $\theta = 2\pi/3$ and $T_k = (2\pi/3 + 2\pi k)/\sqrt{3}$: $T_0 = 2\pi/(3\sqrt{3}) \approx 1.2092$, $T_1 \approx 4.8368$, $T_2 \approx 8.4644$.

Example 7.4 (P3). $G(\omega) = \omega^4 - 7.36\omega^2 + 7$ has the roots $\omega_1 \approx 2.4976$ and $\omega_2 \approx 1.0593$; each generates its own sequence of critical delays, and the two sequences interleave.

For P2, $G(\omega) = \omega^2 + 3 > 0$: there is no crossing, and since P2 is stable at $T = 0$ it is stable for every delay.

7.5 Direction of the crossings

Differentiating $F(s(T), T) = 0$ gives

$$\frac{ds}{dT} = -\frac{F_T}{F_s} = \frac{s e^{-sT} N(s)}{D'(s) + e^{-sT} [N'(s) - TN(s)]}. \quad (7.6)$$

The crossing is destabilizing if $\text{Re}(ds/dT) > 0$ at $s = i\omega$. The following classical result [9] makes the direction a property of the frequency alone.

Proposition 7.5 (Direction depends only on the frequency). *At a simple crossing $s = i\omega$, $\omega > 0$, $\text{sign Re } \frac{ds}{dT} = \text{sign } G'(\omega)$.*

All branches k of a frequency therefore cross in the same direction. In P1, $G'(\sqrt{3}) > 0$: every T_k adds an unstable pair. In P3, crossings at ω_1 always destabilize and crossings at ω_2 always stabilize (Table 7.3, Figure 7.3).

Table 7.3: Imaginary-axis crossings in the studied delay range.

Case	T_c	ω_c	branch	$\text{Re}(ds/dT)$	direction
P1	1.20919958	1.73205081	0	+3.237e-01	destabilizing
P2			no crossing		
P3	0.29182687	2.49756112	0	+1.025e+00	destabilizing
P3	2.69528299	1.05933396	0	-6.200e-02	stabilizing
P3	2.80755521	2.49756112	1	+2.239e-01	destabilizing

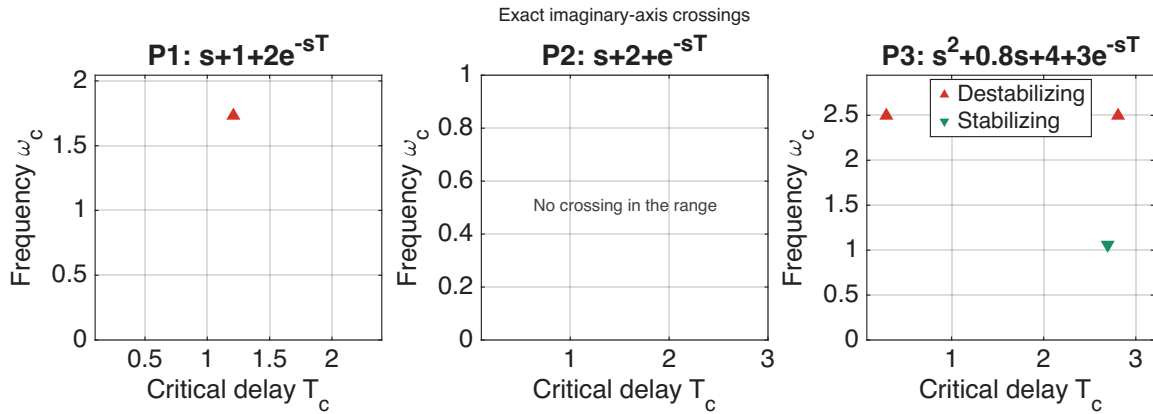


Figure 7.3: Frequencies and directions of the exact crossings (up: into the right half-plane; down: back to the left half-plane).

7.6 Stability map

Each simple crossing moves one conjugate pair, so the number of roots in the right half-plane is the step function

$$Z(T) = Z(0) + 2 \sum_{T_{i,k} < T} \text{sign } G'(\omega_i), \quad (7.7)$$

where $Z(0)$ is the number of unstable roots of $D + N$ (the roots created when T becomes positive come from $\operatorname{Re} s = -\infty$). For P3 on $[0, 10]$:

$$Z(T) = \begin{cases} 0, & 0 \leq T < 0.2918, \\ 2, & 0.2918 < T < 2.6953, \\ 0, & 2.6953 < T < 2.8076 \quad (\text{stability window}), \\ 2, 4, 6, 4, & \text{after } 2.8076, 5.3233, 7.8390, 8.6265. \end{cases}$$

The window has width 0.112: a sweep with step 0.5 misses it entirely. Formula (7.7) was checked independently with `unstable_root_count` at 80 delays, with full agreement (black curve and circles in Figure 7.12). Figure 7.4 shows the spectral abscissa computed by a sweep, refined near the window.

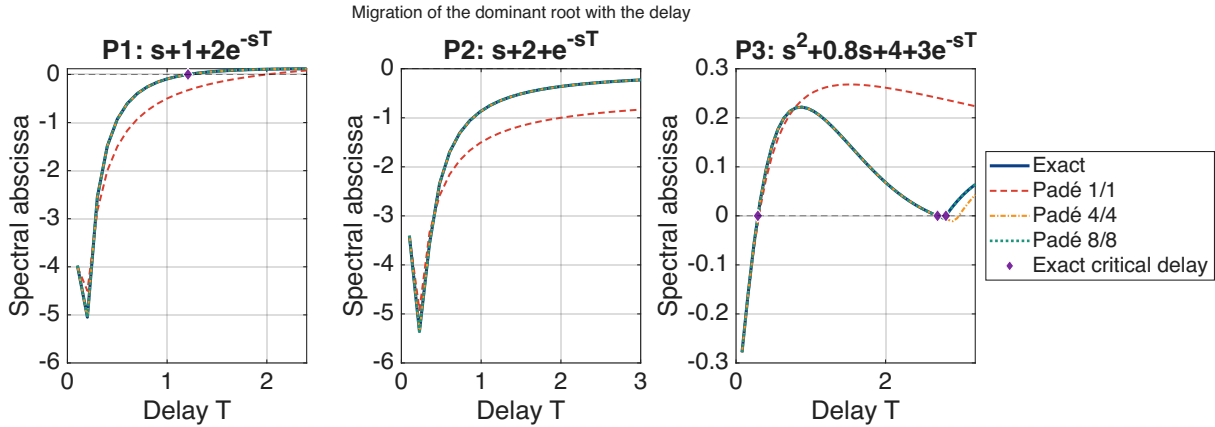


Figure 7.4: Spectral abscissa versus delay, exact and with Padé models of orders 1, 4 and 8; diamonds are the exact critical delays of (7.5). In P3 the exact curve touches zero at $T \approx 2.695$ and crosses again at $T \approx 2.808$; the $[1/1]$ model misses the window and the $[4/4]$ model widens it.

7.7 The Padé approximation

The diagonal Padé approximation of e^{-z} of order n is the rational function $P_n(z)/Q_n(z)$ whose Taylor series agrees with that of e^{-z} up to z^{2n} [3]:

$$e^{-z} - \frac{P_n(z)}{Q_n(z)} = O(z^{2n+1}), \quad Q_n(z) = \sum_{k=0}^n \frac{(2n-k)!n!}{(2n)!k!(n-k)!} z^k, \quad P_n(z) = Q_n(-z). \quad (7.8)$$

With $z = sT$ the characteristic equation becomes the polynomial $D(s)Q_n(sT) + N(s)P_n(sT)$ of degree $m + n$. Three structural properties determine its behavior.

(i) **Exact gain.** Since $P_n(iy) = \overline{Q_n(iy)}$,

$$\left| \frac{P_n(iy)}{Q_n(iy)} \right| = 1 \quad (y \in \mathbb{R}), \quad \left| \frac{P_n(z)}{Q_n(z)} \right| \leq 1 \quad (\operatorname{Re} z \geq 0), \quad (7.9)$$

the second by the maximum principle, as all roots of Q_n lie in the left half-plane. The error of the approximation is entirely in the phase.

(ii) **Bounded phase.** Writing $P_n(iy)/Q_n(iy) = e^{i\phi_n(y)}$, $\phi_n = -2 \arg Q_n(iy)$ decreases monotonically and

$$\phi_n(y) = -y + O(y^{2n+1}) \quad (y \rightarrow 0), \quad \phi_n(y) \searrow -n\pi \quad (y \rightarrow \infty), \quad (7.10)$$

whereas the exact phase $-y$ is unbounded (Figure 7.5). This behavior depends only on $y = \omega T$.

(iii) **Right-half-plane zeros.** The n zeros of $P_n(sT)$ have positive real part. The step response of the approximation jumps to $(-1)^n$ at $t = 0^+$ and oscillates before $t = T$, while the true delay stays at zero (Figure 7.6).

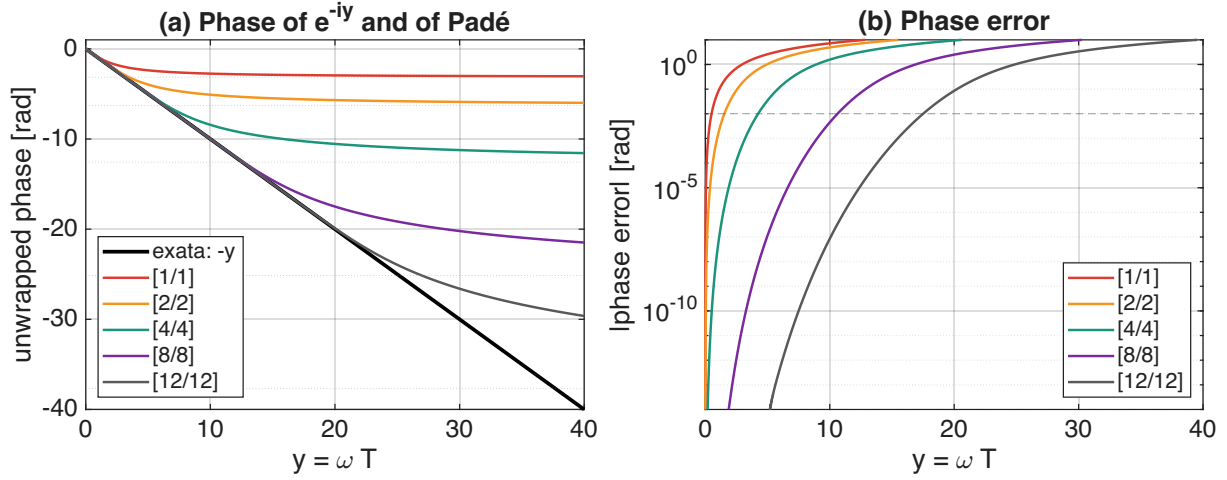


Figure 7.5: (a) Unwrapped phase of e^{-iy} and of $[n/n]$ approximations; dotted lines mark $-n\pi$. (b) Phase error; the dashed line marks 10^{-2} rad.

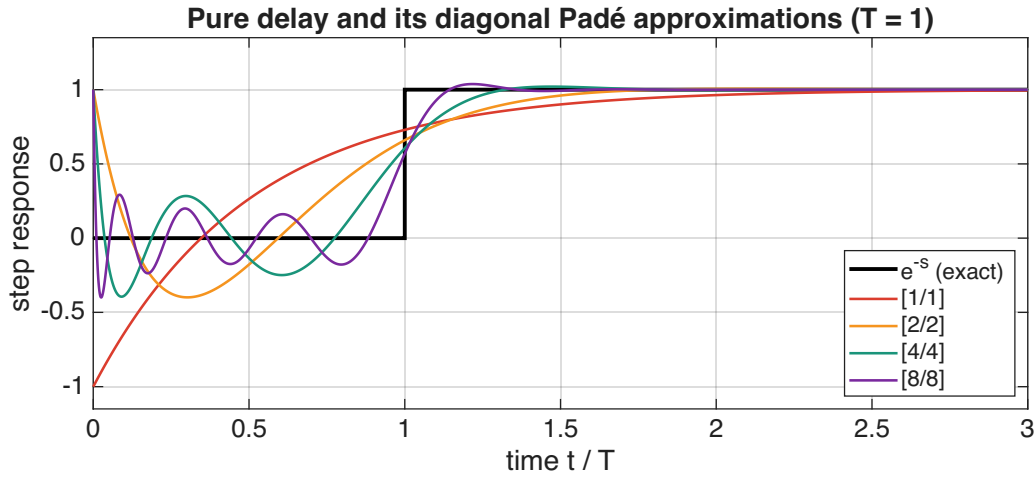


Figure 7.6: Step response of e^{-s} and of its Padé approximations.

Table 7.4 quantifies property (ii): the largest y for which the phase error is below a tolerance. For 0.01 rad and $n \geq 4$, $y_{0.01} \approx 1.6n - 2$.

Table 7.4: Largest $y = \omega T$ with Padé phase error below 0.1, 0.01 and 10^{-6} rad, and asymptotic phase.

n	$y_{0.1}$	$y_{0.01}$	$y_{10^{-6}}$	limit phase $-n\pi$
1	1.124	0.499	0.022	-3.14
2	2.561	1.528	0.235	-6.28
3	4.149	2.813	0.723	-9.42
4	5.819	4.240	1.442	-12.57
6	9.294	7.335	3.369	-18.85
8	12.877	10.621	5.720	-25.13
10	16.526	14.022	8.341	-31.42
12	20.218	17.502	11.149	-37.70

7.8 What Padé preserves

Proposition 7.6 (Frequencies and directions are inherited). *Assume simple crossings, $G \neq 0$ and no common factor of N and D on the imaginary axis. For the rational equation $D + NP_n/Q_n = 0$:*

- (a) *imaginary roots $i\omega$, $\omega > 0$, can occur only at the candidate frequencies ω_i of (7.4) (whether one occurs for a finite delay is decided by (b));*
- (b) *at ω_i , the critical delays solve $\phi_n(\omega_i T) = -(\theta_i + 2\pi k)$, which has a (unique) solution if and only if $\theta_i + 2\pi k < n\pi$;*
- (c) *every crossing has the direction sign $G'(\omega_i)$.*

Diagonal Padé never invents crossing frequencies nor reverses a direction; but a model of order n has at most $\lceil (n\pi - \theta_i)/(2\pi) \rceil$ crossings per frequency for *all* $T \geq 0$, about $n/2$, where the true system has infinitely many. Conclusions that depend only on the modulus are preserved: for P2, $|s + 2| \geq 2 > 1 \geq |P_n/Q_n|$ in the right half-plane, and every Padé model is stable for every delay, as the true system.

In a fixed condition the dominant roots converge very fast with n (Table 7.5, Figure 7.7): faster than geometrically, consistent with the $O(y^{2n+1})$ error, until round-off. Far from the origin, however, rational poles appear that have no exact counterpart (two for $n = 4$, four for $n = 8$, six for $n = 12$ in the window of Figure 7.2).

Table 7.5: RMS error of the dominant pair at the nominal delay.

Case	$n = 1$	$n = 2$	$n = 4$	$n = 6$	$n = 8$	$n = 10$	$n = 12$
P1	5.71e-01	2.90e-02	1.42e-05	1.04e-09	2.16e-14	8.99e-16	3.83e-15
P2	6.53e-01	5.00e-02	4.08e-05	4.73e-09	1.58e-13	5.55e-16	8.71e-15
P3	1.25e-01	5.57e-03	1.20e-06	3.83e-11	1.38e-15	1.54e-15	3.24e-15

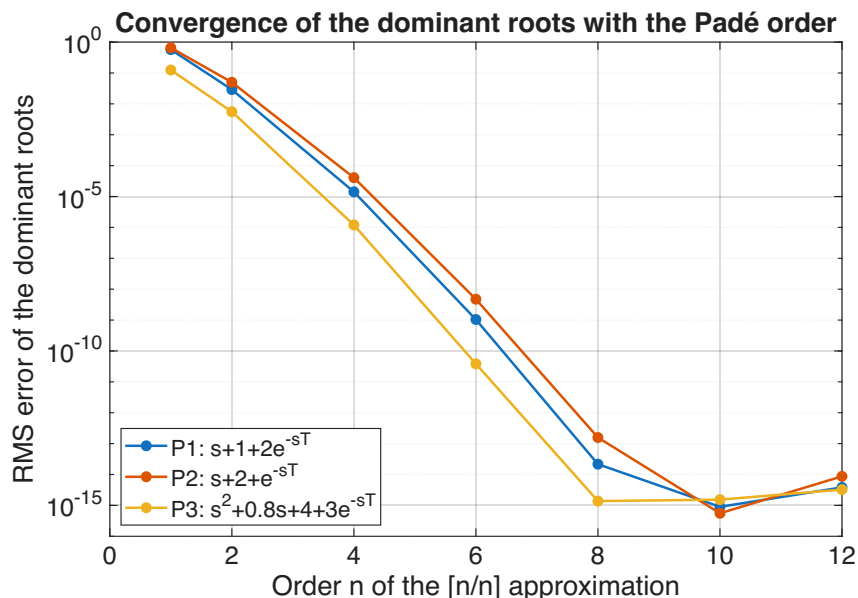


Figure 7.7: RMS error of the dominant pair versus Padé order.

7.9 What Padé loses

7.9.1 A non-conservative delay margin

For P1, the $[1/1]$ phase is $-2 \arctan(\omega T/2)$; equating it to $-2\pi/3$ at $\omega_c = \sqrt{3}$ gives $T_c^{[1/1]} = 2$, against the exact 1.2092: a 65 % error (Table 7.6). The rational polynomial $\frac{T}{2}s^2 + (1 - \frac{T}{2})s + 3$ is Hurwitz if and only if $T < 2$. The error *overestimates* the tolerable delay: at $T = 1.5$, the $[1/1]$ model is stable (abscissa $-1/6$) while the true system is unstable (abscissa $+0.0656$). Figure 7.8 confirms it in the time domain, with the delay equation integrated directly by dde23 [25]. The first crossing occurs at $y = 2\pi/3 \approx 2.09$, beyond the useful range of orders 1 and 2 and within that of order 4 (Table 7.4).

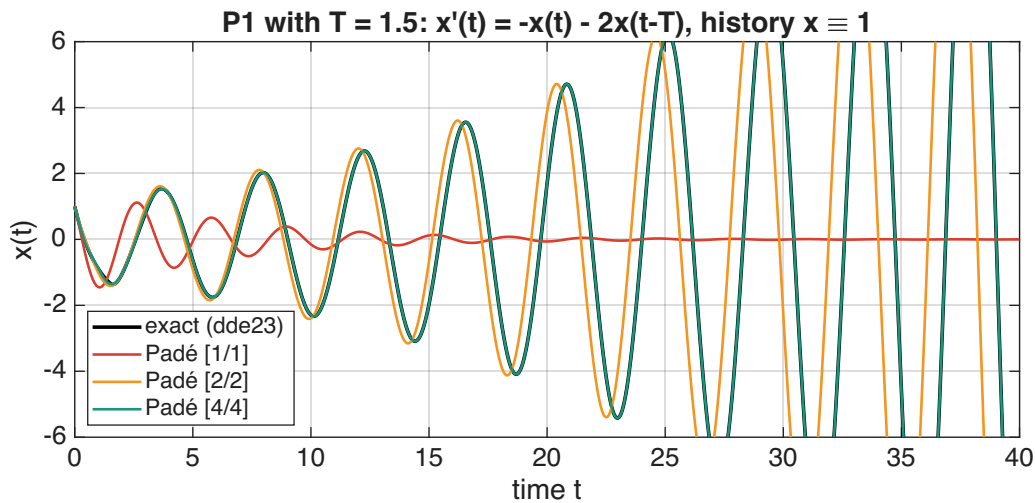


Figure 7.8: P1 at $T = 1.5$, above the exact critical delay 1.209: the delay equation diverges; the $[1/1]$ model decays; $[4/4]$ coincides with the exact solution.

Remark. The rational model also needs an initial state for the Padé filter. The true initial condition is a function on $[-T, 0]$; a model of order n holds only n numbers. The constant history of Figure 7.8 can be represented consistently; general histories cannot.

7.9.2 Shifted and missing branches

For larger delays the branches $k \geq 1$ of (7.5) matter, with larger values of y . Figure 7.9 shows the mechanism for P1: every intersection of a phase curve with a horizontal line $-(\theta + 2\pi k)$ is a critical delay; the Padé phases saturate and reach only the first lines, later and later. Table 7.7 and Figures 7.10–7.11 separate two notions of quality: *local accuracy* of the first critical delay converges fast with n ; *global coverage* of the events grows slowly. In P1 there are three events up to $T = 10$; $[1/1]$ and $[2/2]$ represent one, $[4/4]$ two and $[8/8]$ three. Even when an event is represented, its position may be far off: $[4/4]$ places the second crossing of P1 at 5.71 instead of 4.84.

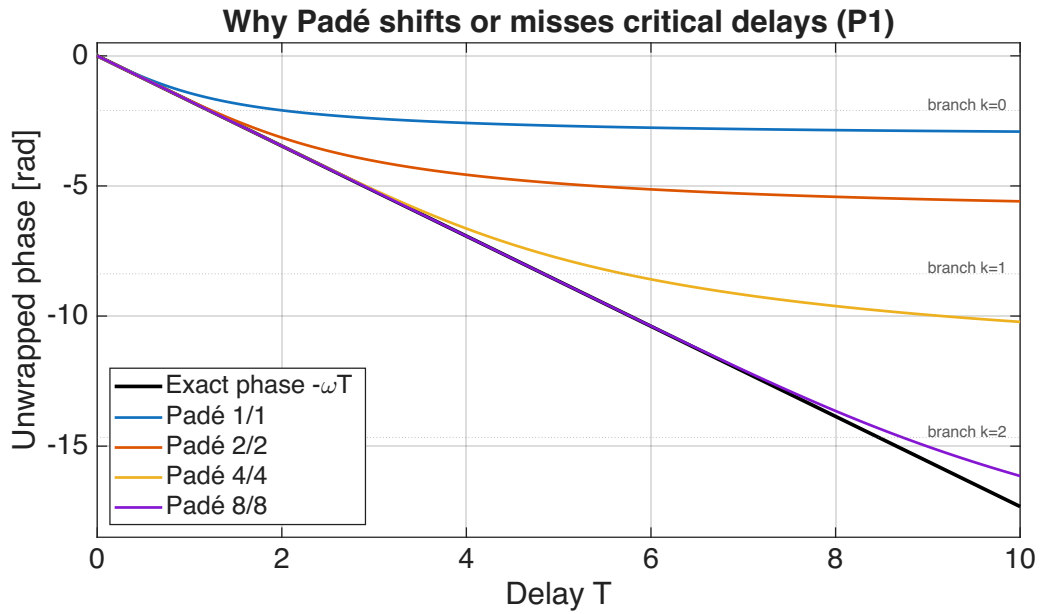


Figure 7.9: P1: exact and Padé phases versus delay; intersections with the branch lines are critical delays.

Table 7.6: First critical delay: exact value and Padé prediction.

Case and order	predicted T_c	absolute error	relative error
P1 exact	1.209199576	—	—
Padé 1/1	2.000000000	7.908e-01	6.540e-01
Padé 2/2	1.236067977	2.687e-02	2.222e-02
Padé 4/4	1.209215058	1.548e-05	1.280e-05
Padé 8/8	1.209199576	3.419e-14	2.828e-14
Padé 12/12	1.209199576	2.220e-16	1.836e-16
P3 exact	0.291826867	—	—
Padé 1/1	0.305471089	1.364e-02	4.675e-02
Padé 2/2	0.291937845	1.110e-04	3.803e-04
Padé 4/4	0.291826868	9.008e-10	3.087e-09
Padé 8/8	0.291826867	5.551e-17	1.902e-16
Padé 12/12	0.291826867	7.772e-16	2.663e-15

Table 7.7: Coverage of the exact events up to $T = 10$.

Order	events P1	coverage P1	events P3	coverage P3
1	1/3	33.3%	1/6	16.7%
2	1/3	33.3%	2/6	33.3%
3	1/3	33.3%	3/6	50.0%
4	2/3	66.7%	3/6	50.0%
5	2/3	66.7%	4/6	66.7%
6	2/3	66.7%	5/6	83.3%
8	3/3	100.0%	5/6	83.3%
10	3/3	100.0%	6/6	100.0%
12	3/3	100.0%	6/6	100.0%

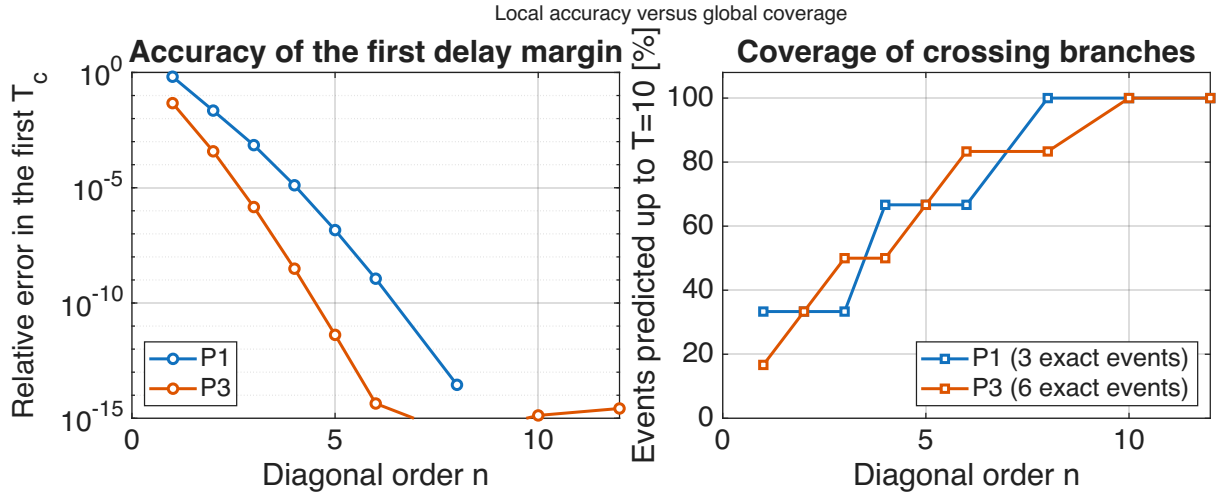


Figure 7.10: Left: relative error in the first critical delay. Right: fraction of the exact crossings in $[0, 10]$ represented by each order.

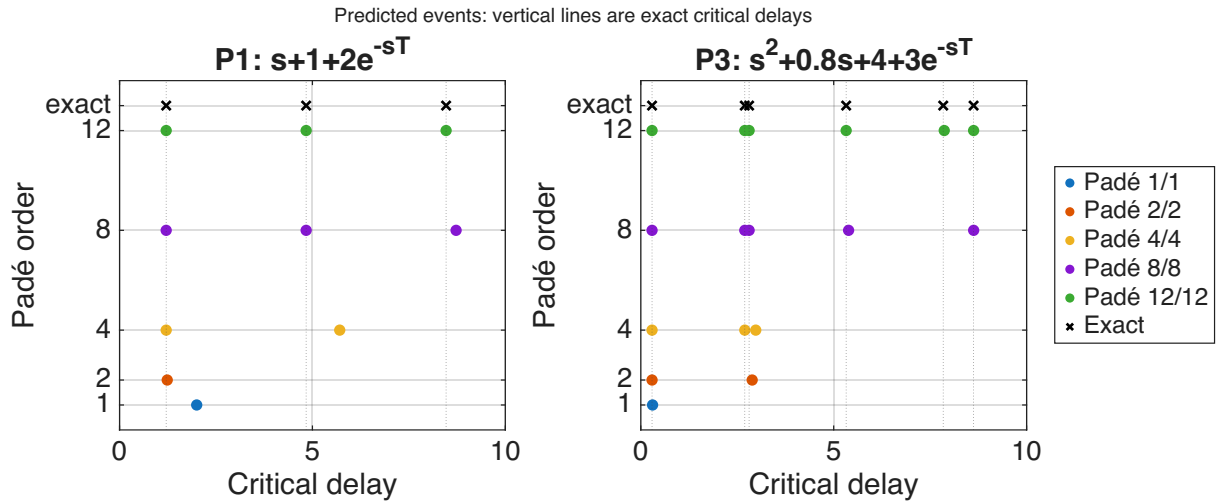


Figure 7.11: Critical delays predicted by each order (dots) and exact (vertical lines and crosses).

7.9.3 Stability windows erased, shifted and invented

In P3 stability depends on the order of events at two frequencies, so a shifted event changes the qualitative conclusion. Figure 7.12 compares $Z(T)$ from (7.7) with the rational models (their counts computed with Proposition 7.6 and checked by eigenvalues of a balanced state-space realization), and Table 7.8 lists the predicted stable intervals.

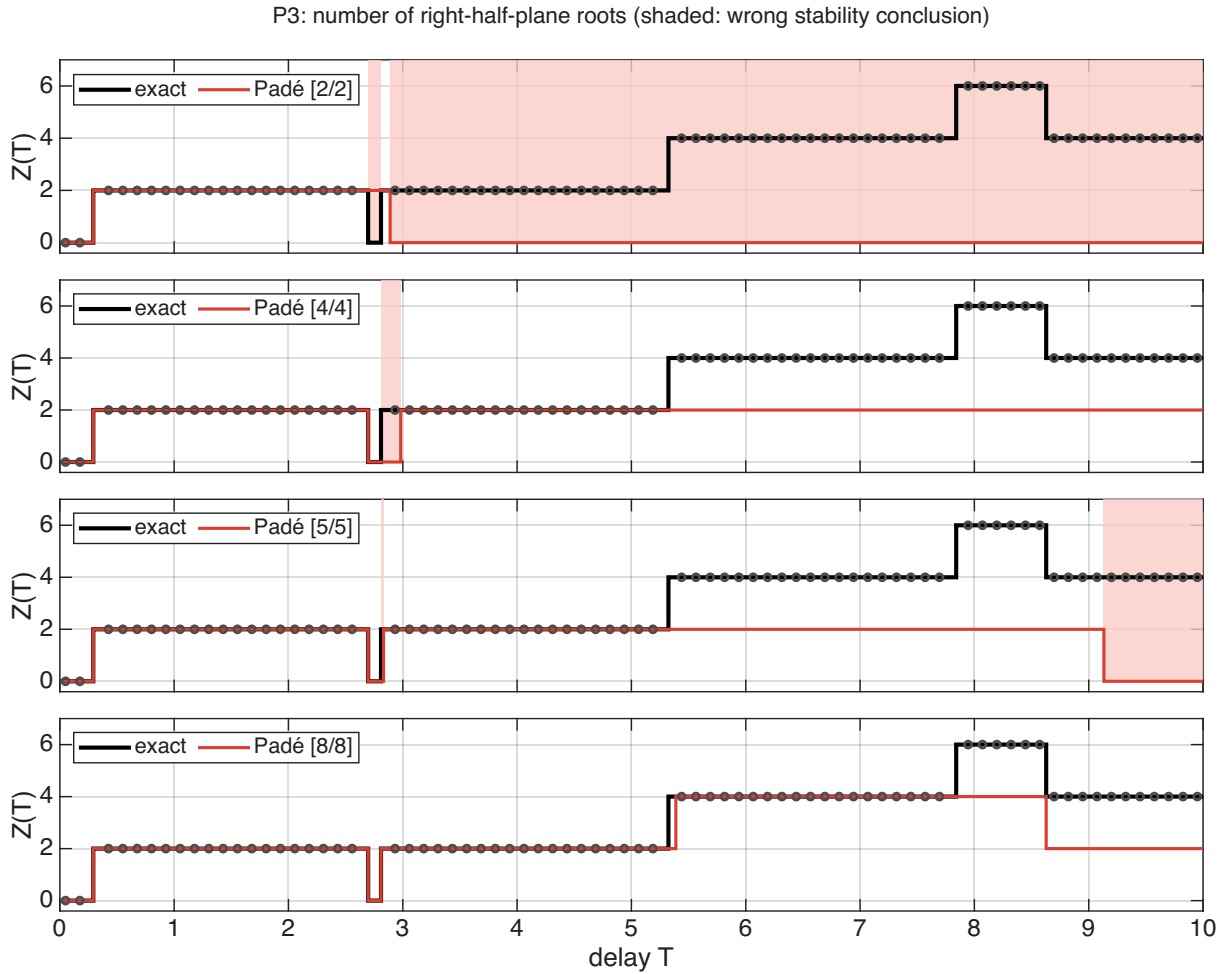


Figure 7.12: P3: number of right-half-plane roots. Black: exact (7.7); circles: independent bound-based count (Proposition 7.2); red: Padé model. Shaded: the rational model reaches the wrong stability conclusion.

Table 7.8: P3: delay intervals in $[0, 10]$ where each model predicts stability, and the number of unstable roots at $T = 10$.

Model	stable intervals in $[0, 10]$	$Z(10)$
exact	$[0, 0.2918), [2.6953, 2.8076)$	4
Padé $[1/1]$	$[0, 0.3055)$	2
Padé $[2/2]$	$[0, 0.2919), (2.8872, 10]$	0
Padé $[3/3]$	$[0, 0.2918), [2.7059, 4.0358)$	2
Padé $[4/4]$	$[0, 0.2918), [2.6957, 2.9797)$	2
Padé $[5/5]$	$[0, 0.2918), [2.6953, 2.8317), (9.1311, 10]$	0
Padé $[6/6]$	$[0, 0.2918), [2.6953, 2.8101)$	2
Padé $[8/8]$	$[0, 0.2918), [2.6953, 2.8076)$	2
Padé $[10/10]$	$[0, 0.2918), [2.6953, 2.8076)$	4
Padé $[12/12]$	$[0, 0.2918), [2.6953, 2.8076)$	4

Four kinds of error appear:

- **Erased window.** $[1/1]$ has no stabilizing event of ω_2 in $[0, 10]$ and misses the window.
- **Unbounded fictitious stability.** $[2/2]$ has the stabilizing event at $T = 2.887$ but not the second branch of ω_1 (which needs $y \approx 7.0 > 2\pi$); it predicts stability for *every* $T > 2.887$, whereas the system is unstable for every $T > 2.808$.

- **Shifted and widened window.** $[3/3]$ and $[4/4]$ predict windows 12 and 2.5 times wider than the true one.
- **Invented stable region.** $[5/5]$ reproduces the window but, lacking the branches $k = 2, 3$ of ω_1 , turns the stabilizing event at $T = 9.13$ into a stable region $(9.13, 10]$ that does not exist.

Only from order 10 on is the stability map in $[0, 10]$ correct, and any fixed order fails again on a long enough interval. High orders are also numerically fragile: the roots of the expanded polynomial $DQ_n + NP_n$ gave a wrong unstable-root count for 24 of 99 small delays with $n = 10$, and 64 of 99 with $n = 12$; a balanced realization avoids this.

7.10 Guidelines

Using Padé safely

1. Decide stability on the original equation: frequencies from (7.4), delays from (7.5), the map (7.7), and a bound-based count as a check.
2. Use Padé as a local tool: design near an operating point, low-order simulation models, or Newton seeds.
3. Choose the order from the largest relevant ωT with Table 7.4; an order n represents no phase beyond $n\pi$.
4. Distrust stability windows and “stable for all larger delays” predicted by rational models.
5. Validate in the time domain near critical delays.
6. Avoid high orders through expanded polynomials; for larger problems use methods designed for delay equations [6, 7, 27].

7.11 Limitations

The direct method above assumes a single constant delay, real coefficients, simple crossings and a retarded system. Tangential or multiple crossings require higher-order analysis [9, 21]. For neutral systems ($p = m$) Proposition 7.2 fails, the root chains approach vertical lines, and stability can be sensitive to infinitesimal changes of the delay [16, 21]. With several delays, crossings form curves and surfaces [20, 22]; matrix systems lead to nonlinear eigenvalue problems [7, 21].

Chapter 8

Distributed-parameter systems

Transfer functions of systems governed by partial differential equations are ratios of entire functions that often look multivalued. This chapter shows how to write them as analytic pairs and computes their poles for the heat, string, duct and beam examples of Curtain and Morris [10]. The models are implemented in `examples/models/distributed_model.m`; the study is `examples/distributed_systems/run_nonrational_examples.m`.

8.1 Removing apparent branch points

For the heat equation the natural expressions contain $\sqrt{s}$. Define

$$C(q) = \cosh \sqrt{q} = \sum_{k \geq 0} \frac{q^k}{(2k)!}, \quad H(q) = \frac{\sinh \sqrt{q}}{\sqrt{q}} = \sum_{k \geq 0} \frac{q^k}{(2k+1)!}, \quad H(0) = 1.$$

Both are entire: only integer powers of q appear. The implementation evaluates H by its series for $|q| < 10^{-4}$ and by the closed form elsewhere, which avoids the $0/0$ at $q = 0$ and does not introduce an artificial pole. The same idea applies to the models below: each factor is evaluated through a regularized formula and is analytic in the search regions used here, so `AssumeAnalytic` is justified for both factors. The heat, string and duct factors are entire; the Kelvin–Voigt beam factors are analytic except at $s = -1/c_d$ (Section 8.4), which the regions must exclude.

8.2 Heat equation: the sensor position

With the normalizations $L = \alpha = K_0 = 1$ of the paper, boundary control and a point sensor at x_0 , the Neumann, Dirichlet and mixed problems give

$$G_N = \frac{C(x_0^2 s)}{s H(s)}, \quad G_D = \frac{x_0 H(x_0^2 s)}{H(s)}, \quad G_M = \frac{x_0 H(x_0^2 s)}{C(s)}.$$

With $x_0 = 1/2$ and $-100 < \operatorname{Re} s < 2$, the Neumann model keeps only 0 and $-4\pi^2$: the candidates $-\pi^2$ and $-9\pi^2$ cancel. The Dirichlet model keeps $-\pi^2$ and $-9\pi^2$ and cancels $-4\pi^2$. The mixed model keeps $-\pi^2/4$, $-9\pi^2/4$ and $-25\pi^2/4$. With the Dirichlet model and $x_0 = 1$, $G \equiv 1$ and every candidate cancels. These are modes of the PDE that the sensor cannot see (Figure 8.1).

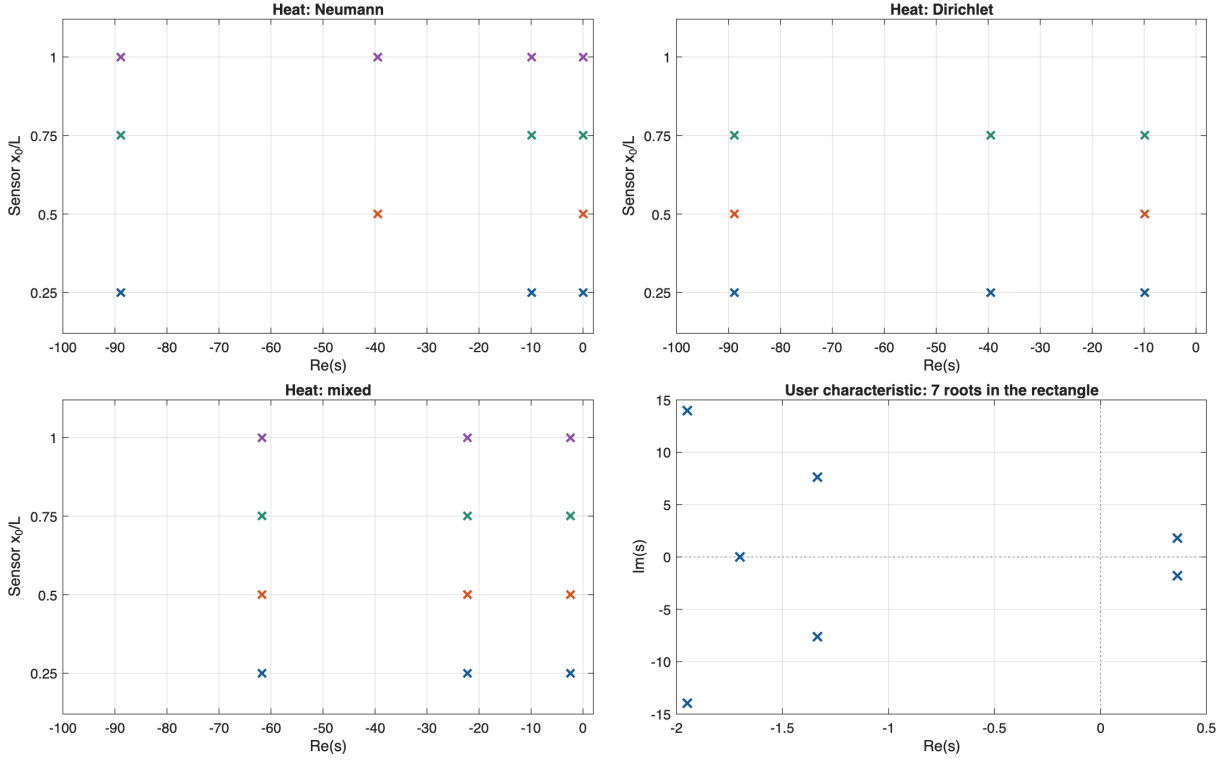


Figure 8.1: Poles retained for four sensor positions and three boundary conditions (the Dirichlet row $x_0 = 1$ is empty because $G = 1$). The last panel shows the characteristic example of Chapter 2.

8.3 String and duct

For the string of Section 2 of the paper, with actuator distribution $b(x) = 1$ on $[0, 1/2]$ and $L = c = 1$, the modal coefficients are proportional to $1 - \cos(k\pi/2)$, which vanishes for k multiple of 4; those modes are not poles. The damped model is the output feedback $G/(1 + \varepsilon G)$ of the paper, with $\varepsilon = 0.5$, and the removable factor at $s = 0$ is regularized by a series.

For the acoustic duct with constant reflectance $a = 0.5$ and $L = c = 1$, the poles lie on the vertical line $\ln(a)/2 + ik\pi$. With the radiation impedance of Section 3.4 (Fig. 5 parameters of the paper: $L = 3.54$ m, $c = 341$ m/s, $\rho = 1.20$ kg/m³, radius 0.101 m, sensor at $L/2$), the reflectance depends on frequency and the poles bend to the left. Its rational denominator is cleared from N and D together, so that no artificial poles are introduced (Figure 8.2).

Visible poles of the damped string in the left half-plane do not prove internal stability of modes hidden from this input/output pair.

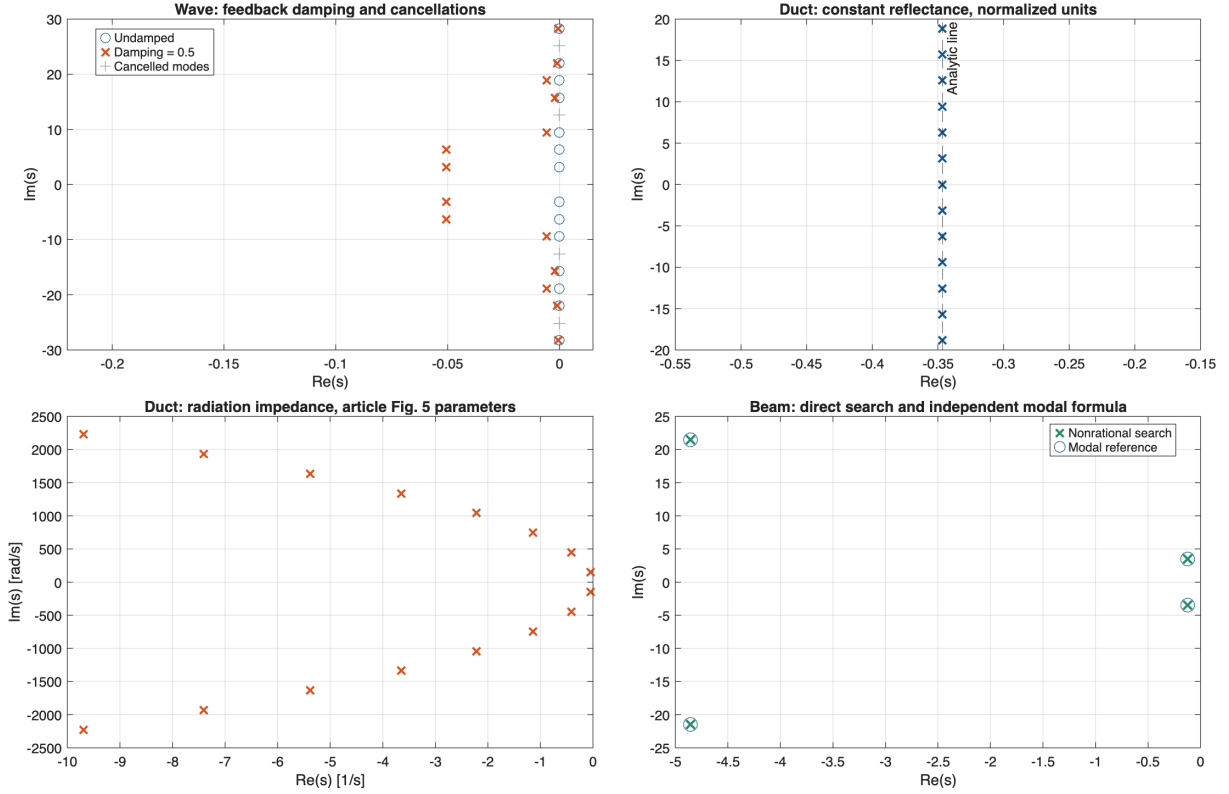


Figure 8.2: String (undamped, damped, and cancelled modes), duct with constant reflectance and with radiation impedance, and beam poles compared with an independent modal formula.

8.4 Kelvin–Voigt beam: an accumulation point

For the shear-force to tip-velocity transfer function of Section 4.2 of the paper, with $L = EI = I = 1$ and Kelvin–Voigt damping $c_d = 0.02$, let

$$q(s) = \frac{-s^2}{1 + c_d s}, \quad m = q^{1/4}, \quad G(s) = \frac{s [\cosh m \sin m - \sinh m \cos m]/m^3}{(1 + c_d s)[1 + \cosh m \cos m]}.$$

The factors containing m are invariant under $m \mapsto im$, so the choice of fourth root disappears except at $s = -1/c_d$; the numerator ratio has the removable value $2/3$ at $m = 0$. With the positive roots β_k of $1 + \cosh \beta \cos \beta = 0$, the poles solve $s^2 + c_d \beta_k^4 s + \beta_k^4 = 0$, an independent reference. In $[-25, 2] \times [-160, 160]$ the search returns $-0.12362363 \pm 3.51384128i$ and $-4.85518819 \pm 21.49292828i$, within 10^{-14} of the reference.

As $\beta_k \rightarrow \infty$, one branch of these poles converges to $-1/c_d = -50$ (Figure 8.3). Every neighborhood of -50 contains infinitely many poles: G is not meromorphic there, and no finite contour computation can resolve a region containing that point. The model reports it in `meta.singularities`, and the option `Singularities` rejects any region that contains it.

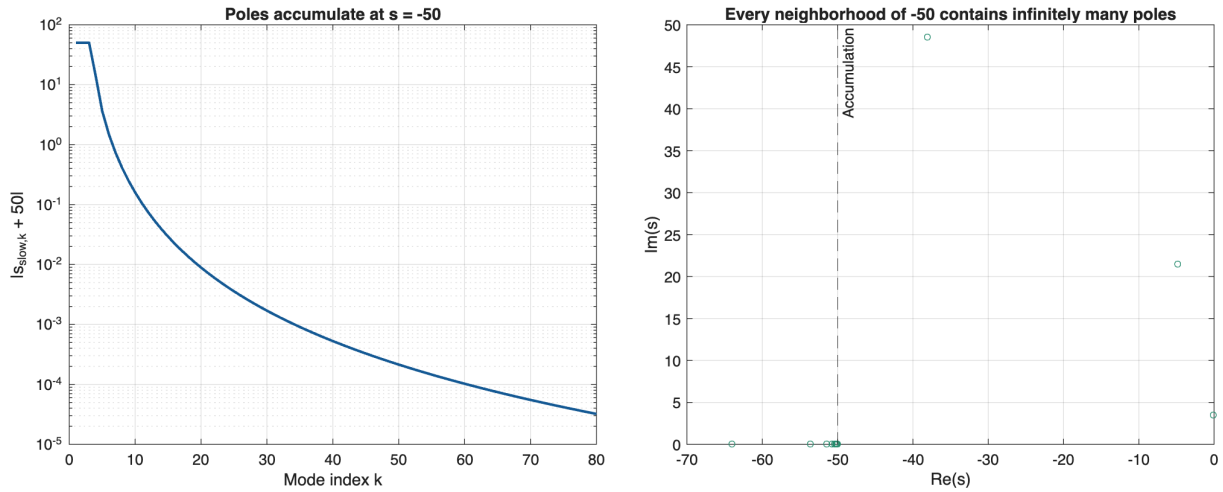


Figure 8.3: Modal reference with 80 modes: one branch of poles converges to $s = -1/c_d = -50$, the other moves to the left.

8.5 Results

Table 8.1: Poles in the stated rectangles; “cancelled” counts locations. The reference errors compare with closed-form or modal formulas that are never used as seeds.

Model	$[\text{Re}_{\min}, \text{Re}_{\max}]$	$ \text{Im} <$	Poles	Cancelled	Max. error
Heat, Neumann	$[-100, 2]$	3	2	2	5×10^{-14}
Heat, Dirichlet	$[-100, 2]$	3	2	1	1×10^{-14}
Heat, mixed	$[-100, 2]$	3	3	0	7×10^{-15}
String, undamped	$[-2, 1]$	30	14	4	3×10^{-21}
String, $\varepsilon = 0.5$	$[-2, 1]$	30	14	4	—
Duct, constant reflectance	$[-2, 1]$	20	13	0	2×10^{-12}
Duct, radiation impedance	$[-400, 30]$	2500	16	0	—
Beam, Kelvin–Voigt	$[-25, 2]$	160	4	0	7×10^{-15}

For the damped string and the radiating duct there is no independent closed form; they are verified by the contour counts and the residuals of the original equations.

Chapter 9

A beam coupled to an oscillator

This case study couples a partial differential equation (a beam, with infinitely many modes) to an ordinary one (a mass-spring-damper), derives an entire characteristic function and transfer functions, and computes the poles without modal truncation. The results are validated with an independent finite-element model.

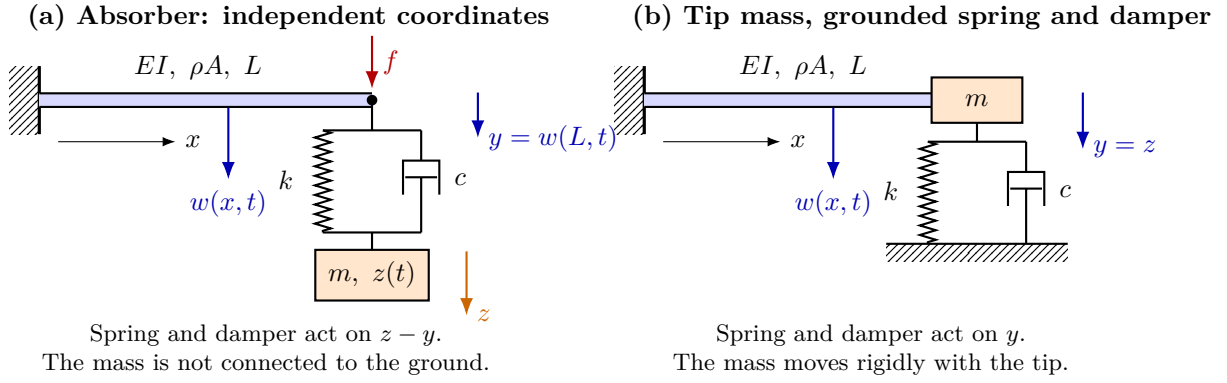


Figure 9.1: The two topologies. The connection transmits transverse force but no moment; displacements are positive downwards.

9.1 Model

A uniform Euler–Bernoulli beam of length L , bending stiffness EI and mass per length $\mu_b = \rho A$ is clamped at $x = 0$. In topology (a), a mass m with displacement $z(t)$ is connected to the tip displacement $y(t) = w(L, t)$ by a spring k and a damper c in parallel; the mass is not grounded (a tuned-mass absorber). The beam is elastic; all dissipation is in c . Shear deformation, rotary inertia and gravity (already balanced) are neglected:

$$\mu_b w_{tt} + EI w_{xxxx} = 0, \quad 0 < x < L, \quad w(0, t) = w_x(0, t) = 0, \quad (9.1)$$

$$EI w_{xx}(L, t) = 0, \quad -EI w_{xxx}(L, t) = F(t) + f(t), \quad (9.2)$$

$$f = k(z - y) + c(\dot{z} - \dot{y}), \quad m\ddot{z} + c(\dot{z} - \dot{y}) + k(z - y) = u(t), \quad (9.3)$$

where f is the force of the connection on the beam and F, u are external forces at the tip and on the mass.

9.2 The bare-beam receptance

With $w = W(x)e^{st}$ and $\theta^4 = q = -\mu_b L^4 s^2 / EI$, the clamped solution is $W(x) = a[\cosh bx - \cos bx] + d[\sinh bx - \sin bx]$, $b = \theta/L$. The zero-moment condition fixes d/a , and the shear condition gives the tip receptance

$$H_b(s) = \frac{y}{f} = \frac{L^3}{EI} \frac{A(q)}{B(q)}, \quad A(q) = \frac{\cosh \theta \sin \theta - \sinh \theta \cos \theta}{\theta^3}, \quad B(q) = 1 + \cosh \theta \cos \theta.$$

A and B are entire in q : they are invariant under $\theta \mapsto i\theta$. Near $q = 0$, $A(q) = \frac{2}{3} - \frac{q}{315} + \frac{q^2}{623700} - \frac{q^3}{5108103000} + \dots$ and $B(0) = 2$, so $H_b(0) = L^3/(3EI)$, the static compliance of a cantilever. Write $H_b = N_b/D_b$ with $N_b = (L^3/EI)A$, $D_b = B$.

9.3 Characteristic function and transfer functions

With $Z(s) = k + cs$ the Laplace-domain equations are

$$\begin{bmatrix} D_b + N_b Z & -N_b Z \\ -Z & ms^2 + Z \end{bmatrix} \begin{bmatrix} y \\ z \end{bmatrix} = \begin{bmatrix} N_b F \\ u \end{bmatrix},$$

whose determinant is the entire characteristic function

$$\Delta(s) = D_b(s)(ms^2 + cs + k) + N_b(s)ms^2(cs + k). \quad (9.4)$$

The coupled poles are not the union of the beam and oscillator poles, and $1 + H_b Z_{\text{eff}}$, with $Z_{\text{eff}} = ms^2(cs + k)/(ms^2 + cs + k)$, is not entire; clearing its denominators gives (9.4). The transfer functions are

$$\frac{y}{F} = \frac{N_b(ms^2 + Z)}{\Delta}, \quad \frac{y}{u} = \frac{N_b Z}{\Delta}, \quad \frac{z}{u} = \frac{D_b + N_b Z}{\Delta}.$$

With zero inputs, the energy $E = \frac{1}{2} \int_0^L (\mu_b w_t^2 + EI w_{xx}^2) dx + \frac{1}{2} m \dot{z}^2 + \frac{1}{2} k (z - y)^2$ satisfies $\dot{E} = -c(\dot{z} - \dot{y})^2 \leq 0$, which confirms the sign conventions and excludes growing modes for positive parameters. In topology (b) the mass moves with the tip and $\Delta_{\text{tip}} = D_b + N_b(ms^2 + cs + k)$; it is a different system, with potential energy $ky^2/2$.

9.4 Implementation

With $\tau = \sqrt{\mu_b L^4 / EI}$, $\lambda = \tau s$, $\mu = m/(\mu_b L)$, $\kappa = kL^3/EI$ and $\gamma = cL^3/(EI\tau)$, the function evaluated is $\Delta = B(-\lambda^2)(\mu\lambda^2 + \gamma\lambda + \kappa) + A(-\lambda^2)\mu\lambda^2(\gamma\lambda + \kappa)$, equal to (9.4) times L^3/EI . In normalized units the complete computation needs only function handles:

```
m = 0.1; k = 1.2362363368; c = 0.05; % L = EI = rho*A = 1
beta = @(s) (-s.^2).^(1/4);
Nb = @beam_numerator; % series near 0, see text
Db = @(s) 1 + cosh(beta(s)).*cos(beta(s));
Zc = @(s) k + c*s;
N = @(s) Nb(s).*(m*s.^2 + Zc(s));
D = @(s) Db(s).*(m*s.^2 + Zc(s)) + Nb(s).*m*s.^2.*Zc(s);
[p, info] = cpoles(ndpair(N, D), [-60 1 -150 150], 'AssumeAnalytic', true);
```

This excerpt is not self-contained: the local function `beam_numerator` (the series-regularized $A(q)$) is defined in the full script `examples/coupled_beam/beam_direct_nd.m`; the helper `examples/models/coupled_beam_model.m` builds the dimensional model and the three transfer functions.

9.5 Reference case and parameter studies

The reference case uses $m = 0.1$, $k = m\omega_{b1}^2 = 1.23623634$ with the first bare-beam frequency $\omega_{b1} = 3.51601527$, and $c = 0.05$. The ten poles in $[-60, 1] \times [-150, 150]$ are (upper half-plane):

Re(s)	Im(s)	ζ
-0.04655950	2.57427003	0.018084
-0.29481789	4.76361120	0.061771
-0.10690219	22.14678239	0.004827
-0.10105104	61.73606590	0.001637
-0.10032435	120.92154746	0.000830

For $c = 0$ the two lowest frequencies are 2.5703 and 4.7801: the coupling splits the original resonance; in the first mode the mass follows the tip, in the second it moves against it (Figure 9.2).

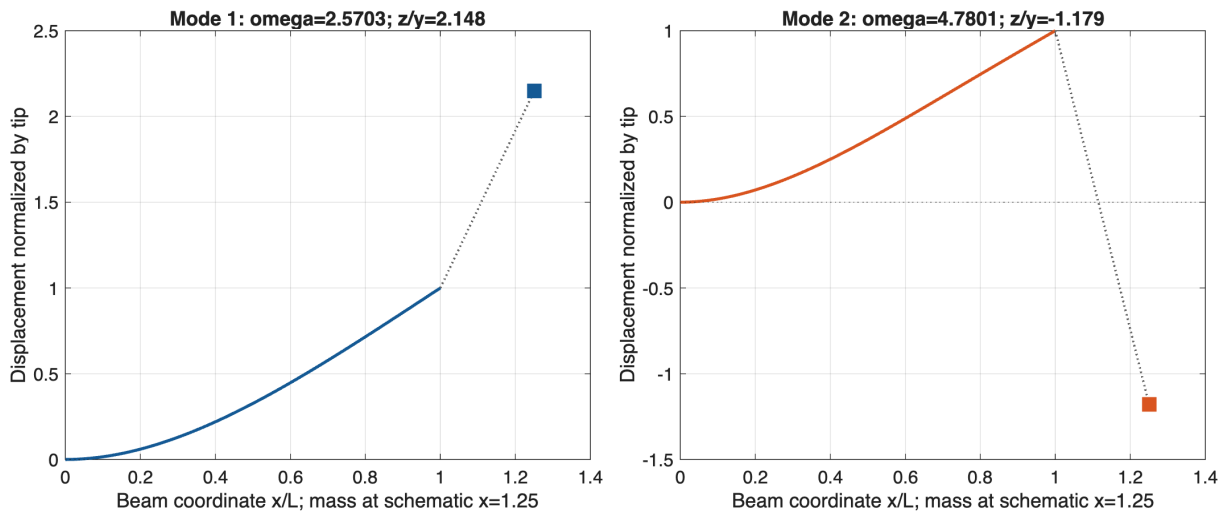


Figure 9.2: Conservative coupled modes from the exact spatial solution, normalized by the tip displacement.

One parameter at a time was varied: $m \in [0.02, 0.5]$ (25 values), $k \in [0.05, 5]$ (36 values) and $c \in [0, 1.5]$ (43 values). Every one of the 104 cases has ten poles in the window, each found with a fresh contour count (previous poles were only seeds).

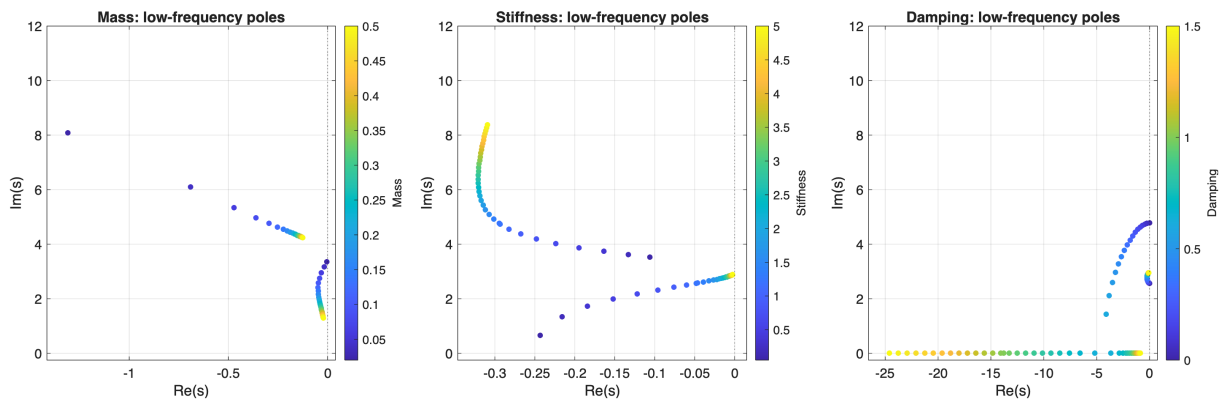


Figure 9.3: Low-frequency poles as the mass, the stiffness and the damping vary.

c	$\text{Re}(s)$	$\text{Im}(s)$	ζ
0.00	-0.000000	2.570269	0.0000
0.00	0.000000	4.780068	-0.0000
0.05	-0.046560	2.574270	0.0181
0.05	-0.294818	4.763611	0.0618
0.15	-0.134523	2.606584	0.0515
0.15	-0.892649	4.629622	0.1893
0.30	-0.224003	2.710806	0.0824
0.30	-1.850766	4.152388	0.4071
0.60	-0.192357	2.884138	0.0665
0.60	-4.116381	1.428956	0.9447
1.00	-1.429161	0.000000	1.0000
1.00	-0.123947	2.938350	0.0421
1.00	-13.808315	-0.000000	1.0000
1.50	-0.873874	-0.000000	1.0000
1.50	-0.083993	2.954900	0.0284
1.50	-24.605705	-0.000000	1.0000

Increasing c can make the second pair real. Among the sampled values the rightmost pole is leftmost at $c = 0.375$ (abscissa -0.234798); at $c = 1.5$ the dominant pair returns to $-0.0840 \pm 2.9549i$. Damping in the connection restrains the very relative motion it dissipates, so more damping is not always better. This is a statement about the sampled values and the stated window, not a global optimum.

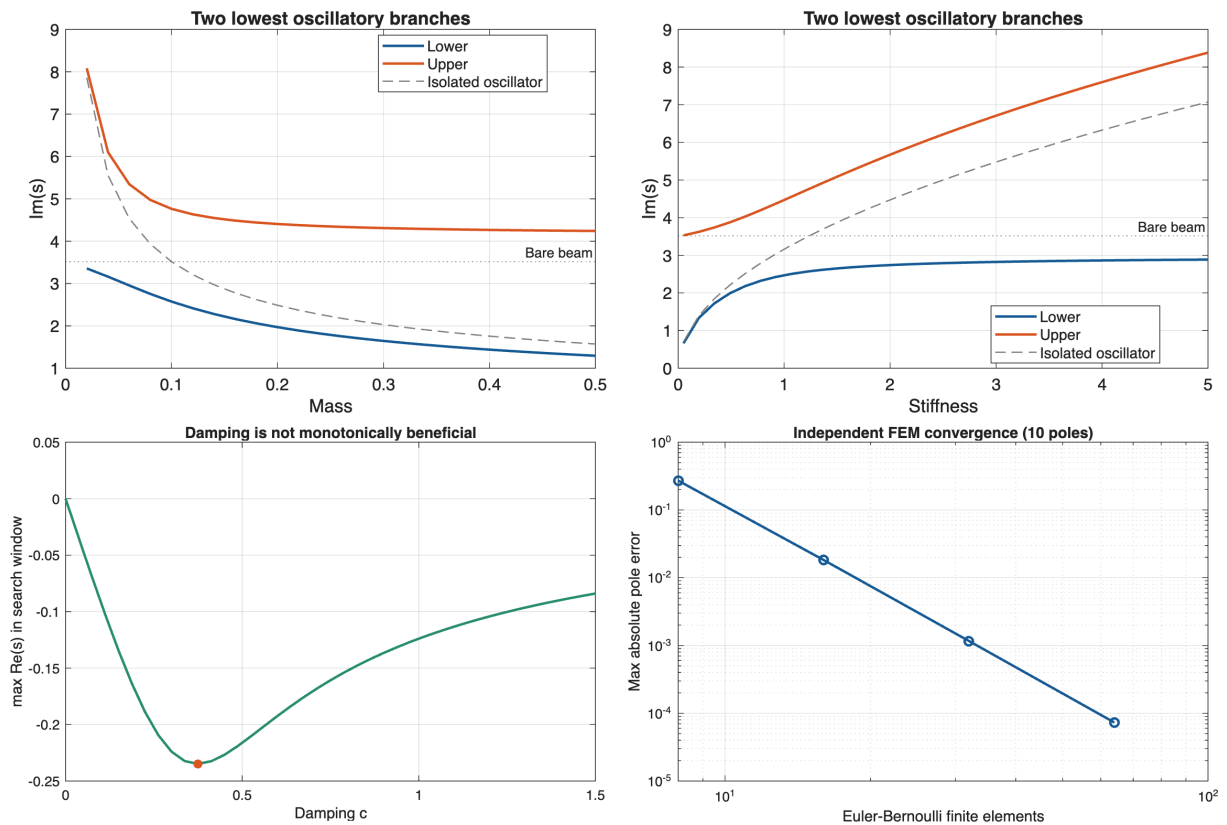


Figure 9.4: Lowest branches versus mass and stiffness, window abscissa versus damping, and convergence of the finite-element model.

9.6 Independent validation

A cubic-Hermite finite-element model with consistent mass matrix and an extra degree of freedom for the absorber mass (`coupled_beam_fem.m`) is solved as a quadratic eigenvalue problem, without using any result of the nonrational solver. The largest error among the ten reference poles decreases with the mesh:

Elements	Max. absolute error
8	0.270955
16	0.0182861
32	0.00116392
64	7.30819e-05

Further checks: the static compliance $H_b(0) = 1/3$; invariance of the dimensionless characteristic function under dimensional rescaling; the bare-beam limit; and the decoupled limit $k = c = 0$, where the characteristic function has a double root at 0 (the free mass) that is *not* a pole of the tip transfer function y/F — a hidden mode. Enlarging the window to $[-90, 1] \times [-300, 300]$ at three damping values leaves the rightmost pole unchanged. The beam is elastic and has no finite accumulation point, unlike the Kelvin–Voigt beam of Section 8.4.

Chapter 10

Aeroelastic flutter without rational approximations

Flutter is a dynamic instability of a flexible structure in an airflow: above a critical speed the aerodynamic forces feed energy into a structural mode, which then grows. Its analysis is a feedback-stability problem with an *irrational* feedback path, Theodorsen's function. The standard practice approximates that function by a rational one, so that the aeroelastic system becomes a finite state-space model [13, 17]. This chapter shows how to decide stability and compute the flutter speed with the exact function instead, using the two-degree-of-freedom typical section and two published benchmarks [17, 24]. The code is in `examples/aeroelasticity/` and `examples/models/`.

10.1 The typical section as a feedback system

The section (Figure 10.1) has semichord b , the elastic axis at ab from midchord and the centre of gravity at $x_\alpha b$ behind the elastic axis. With plunge h (positive downwards) and pitch α (positive nose-up) about the elastic axis, $q = [h, \alpha]^T$, $S_\alpha = m x_\alpha b$ and $I_\alpha = m r_\alpha^2 b^2$, the equations per unit span are

$$M\ddot{q} + Kq = \begin{bmatrix} -L \\ M_a \end{bmatrix}, \quad M = \begin{bmatrix} m & S_\alpha \\ S_\alpha & I_\alpha \end{bmatrix}, \quad K = \begin{bmatrix} k_h & 0 \\ 0 & k_\alpha \end{bmatrix}, \quad (10.1)$$

with the lift L (upwards) and the nose-up aerodynamic moment M_a about the elastic axis.

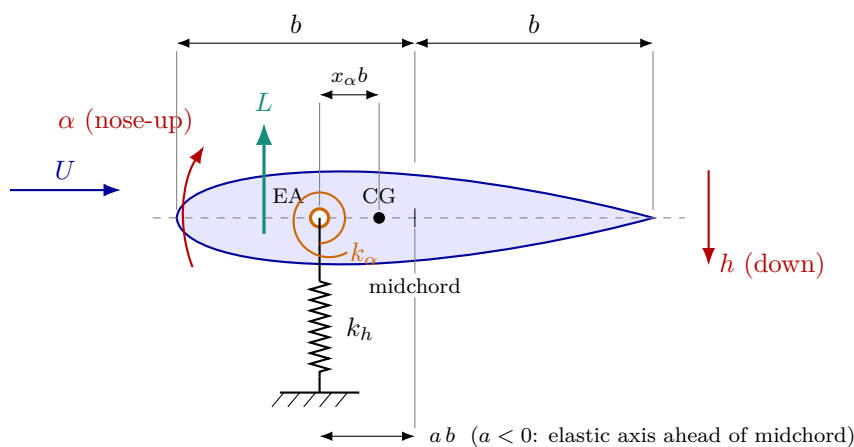


Figure 10.1: Typical section. The springs k_h and k_α act at the elastic axis (EA); L is the lift.

For a motion e^{st} , Theodorsen's theory (incompressible, inviscid, two-dimensional flow, small

motions, Kutta condition) gives

$$L = \pi \rho b^2 [s^2 h + U s \alpha - a b s^2 \alpha] + 2 \pi \rho U b C(p) w, \quad (10.2)$$

$$M_a = \pi \rho b^2 [a b s^2 h - U b (\frac{1}{2} - a) s \alpha - b^2 (\frac{1}{8} + a^2) s^2 \alpha] + 2 \pi \rho U b^2 (a + \frac{1}{2}) C(p) w, \quad (10.3)$$

where $w = s h + [U + b(\frac{1}{2} - a)s]\alpha$ is the downwash at three-quarter chord and $p = s b / U$. The first terms are non-circulatory (added mass and damping of the air); the circulatory terms carry the memory of the wake through Theodorsen's function C . Writing the loads as $A(s, U)q$ gives the dynamic stiffness matrix and the characteristic function

$$D(s, U) = s^2 M + K - A(s, U), \quad \Delta(s, U) = \det D(s, U), \quad (10.4)$$

whose roots are the aeroelastic modes, in the same way as the eigenvalues of a state-space matrix are the roots of $\det(sI - A)$. Explicitly, $A = \pi \rho U^2 (p^2 A_2 + p A_1 + A_0)$ with

$$A_2 = \begin{bmatrix} -1 & ab \\ ab & -(\frac{1}{8} + a^2)b^2 \end{bmatrix}, \quad A_0 = \begin{bmatrix} 0 & -2Cb \\ 0 & 2C(\frac{1}{2} + a)b^2 \end{bmatrix},$$

$$A_1 = \begin{bmatrix} -2C & [-1 - 2C(\frac{1}{2} - a)]b \\ 2C(\frac{1}{2} + a)b & (\frac{1}{2} - a)[2C(\frac{1}{2} + a) - 1]b^2 \end{bmatrix},$$

in agreement with Kaiser and Quero [17, Eq. (33)]. At $U = 0$ the limit is $D = s^2(M + M_{\text{air}}) + K$ with the still-air added mass $M_{\text{air}} = \pi \rho b^2 \begin{bmatrix} 1 & -ab \\ -ab & (1/8 + a^2)b^2 \end{bmatrix}$.

10.2 Theodorsen's function in the Laplace domain

The generalized Theodorsen function is [13]

$$C(p) = \frac{K_1(p)}{K_0(p) + K_1(p)}, \quad (10.5)$$

with the modified Bessel functions K_0, K_1 . On the imaginary axis, $p = ik$ with the reduced frequency $k = \omega b / U$, it reduces to the classical $C(k) = H_1^{(2)}(k) / [H_1^{(2)}(k) + i H_0^{(2)}(k)]$; $C(0) = 1$ and $C \rightarrow \frac{1}{2}$ at high frequency. The implementation uses exponentially scaled Bessel functions, whose common factor cancels in the ratio.

C is analytic in the plane cut along the negative real axis; $s = 0$ is a logarithmic branch point, and $C(0) = 1$ is a limit, not an analytic value. Therefore $\Delta(\cdot, U)$ is analytic on any region that does not touch $(-\infty, 0]$, and in particular on

- the right half-plane $\text{Re } s > 0$, which contains the positive real axis — this window counts the unstable roots, including real ones;
- the upper half-plane $\text{Im } s > 0$ — this window shows the stable oscillatory modes; since $\Delta(\bar{s}) = \overline{\Delta(s)}$ the lower half-plane holds the conjugates.

The branch cut also produces a non-exponential contribution to transient responses [13]; the stability statements below concern the roots of (10.4).

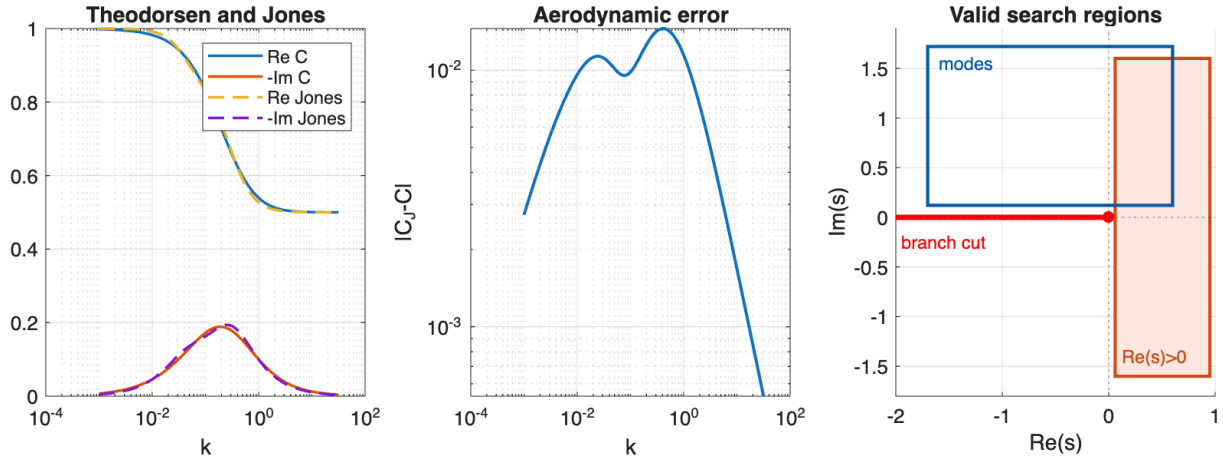


Figure 10.2: Left: real and imaginary parts of $C(ik)$ and of Jones' approximation. Centre: the approximation error. Right: the branch cut and the two valid search regions.

10.3 Stability in three steps

The function Δ is passed to ContourRoots as a handle, with `AssumeAnalytic` justified by the preceding section. The scaled determinant $\det(S^{-1}DS^{-1})$, $S = \text{diag}(\sqrt{k_h}, \sqrt{k_a})$, is used for conditioning; functions such as $|\det D|$ or the smallest singular value are not analytic and must not be used.

1. Stability at a given speed. The number of roots with $\text{Re } s > 0$, the analogue of `any(real(eig(A))>0)`:

```
model = aeroelastic_section('nasa');
Delta = @(s,U) aeroelastic_delta(s,U,model);
[p,info] = croots(@(s) Delta(s,170), [1e-3 60 -250 250], 'AssumeAnalytic', true);
```

At $U = 170$ ft/s the count is zero (stable); at 180 ft/s it is two, a pair $2.132 \pm 74.78i$. For large $|s|$ the inertia terms dominate, $D \approx s^2(M + M_{\text{air}})$, so a window of a few times the structural frequencies suffices; enlarging it is a cheap check. When a mode is very close to the imaginary axis — at very low speed, where the air adds little damping, or within a fraction of a percent of flutter — the count needs more contour samples; it is then reported as `unresolved` until `ContourRefinements` is increased.

2. Modes. The window $[-100, 50] \times [0.5, 180]$ gives, at 170 ft/s, the modes $-1.1504 + 75.8683i$ ($\zeta = 0.015$) and $-31.4233 + 63.4831i$ ($\zeta = 0.444$).

3. Flutter speed. The spectral abscissa $\alpha(U) = \max \text{Re } s$ over the modes crosses zero at the flutter speed, which `fzero` locates:

```
upper = [-100 50 0.5 180];
alpha = @(U) max(real(croots(@(s) Delta(s,U), upper, 'AssumeAnalytic', true)));
Uf = fzero(alpha, [150 200]); % 173.2624 ft/s
```

Alternatively, `aeroelastic_flutter` solves $\text{Re } \Delta(i\omega, U) = \text{Im } \Delta(i\omega, U) = 0$ for (ω, U) by Newton's method and returns $\text{Re}(ds/dU) = -\text{Re}(\Delta_U/\Delta_s) > 0$ at the crossing.

Divergence. With $C(0) = 1$, static divergence occurs at $U_d = \sqrt{k_a/[2\pi\rho b^2(a + \frac{1}{2})]}$ for $a > -\frac{1}{2}$, when a real root crosses into the right half-plane through $s = 0$. Real roots are invisible to the

upper window but not to the right-half-plane count: at $1.05 U_d$ the count finds the real root $s = 1.888$ (NASA case).

10.4 Benchmarks

The NASA standard case [24, Appendix C] recomputes the flexure–torsion example of NACA Report 496: $\pi \rho b^2/m = 0.1$, $a = -0.4$, $x_\alpha = 0.2$, $r_\alpha^2 = 0.25$, uncoupled frequencies $\omega_h = \sqrt{k_h/m} = 50$ and $\omega_\alpha = \sqrt{k_\alpha/I_\alpha} = 100$ rad/s, $b = 1$ ft. The dimensional case of Kaiser and Quero [17, Table 1] is in SI units. Table 10.1 compares the results; the published values are never used as starting points.

Table 10.1: Flutter of the two benchmarks.

Case	Published U_f	Computed U_f	ω_f [rad/s]	k_f	U_d
NASA	173.26 ft/s, $k_f = 0.4355$	173.2624 ft/s	75.462	0.43554	353.55 ft/s
Kaiser–Quero	212.2 m/s	212.1729 m/s	58.438	0.27542	394.69 m/s

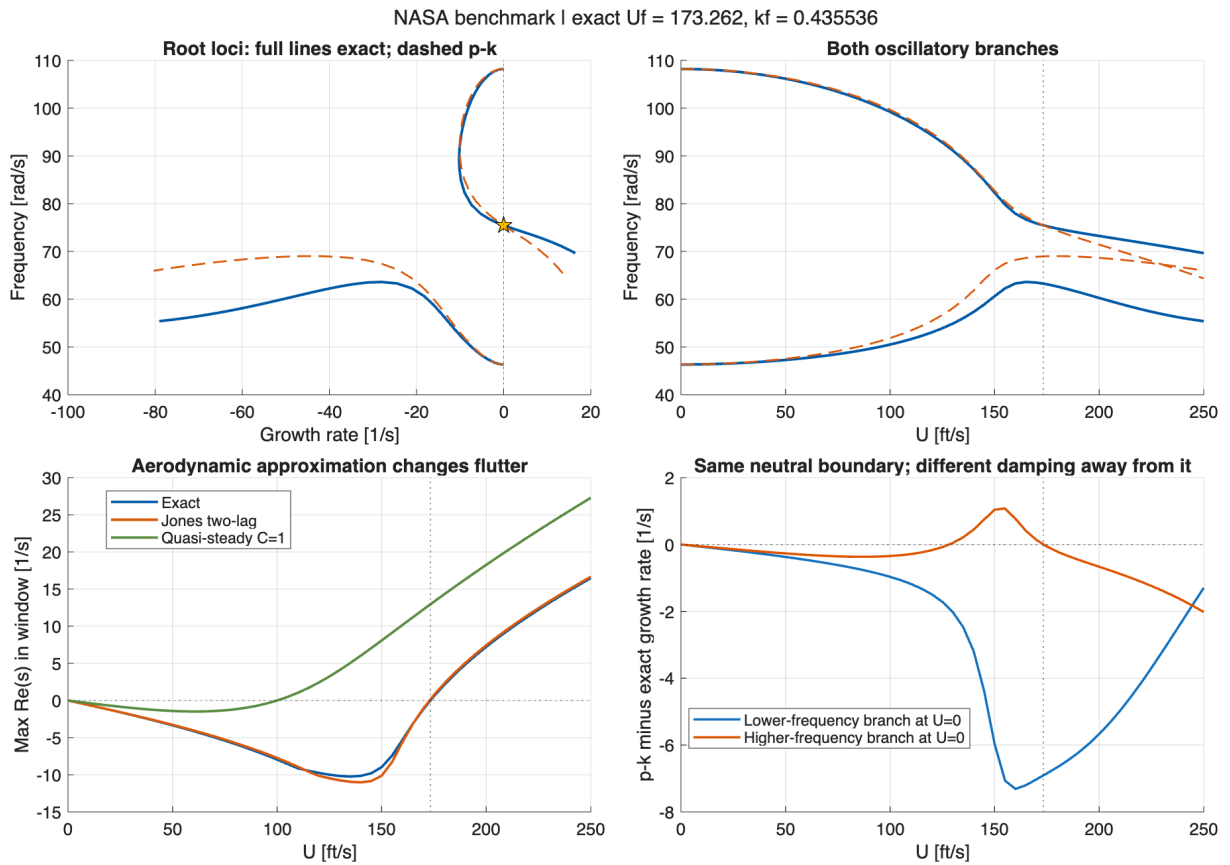


Figure 10.3: NASA case. Top: root loci (exact, full lines; p-k, dashed) and frequencies versus speed. Bottom left: largest real part of the modes for exact, Jones and quasi-steady aerodynamics. Bottom right: difference in growth rate between p-k and exact.

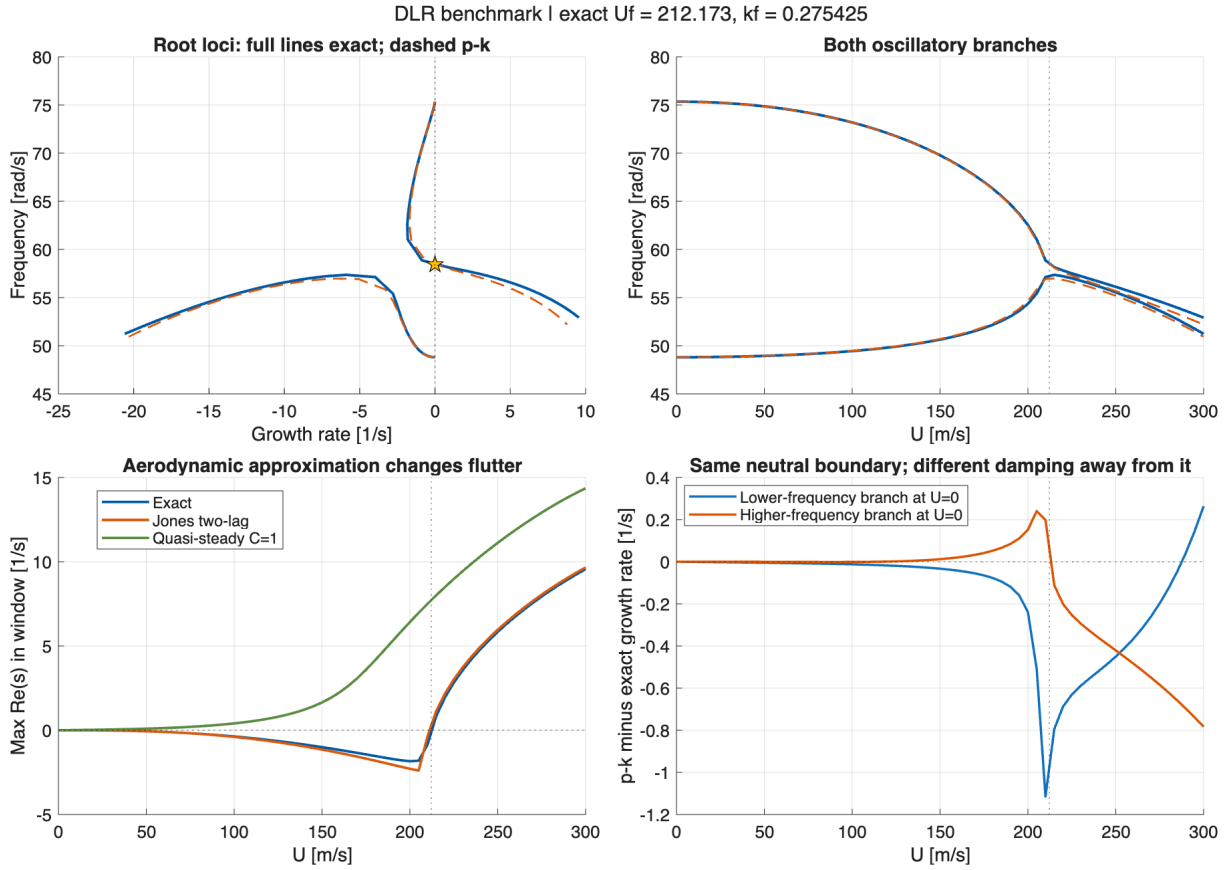


Figure 10.4: The dimensional benchmark of Kaiser and Quero [17], same layout as Figure 10.3.

10.5 What rational approximations change

Jones' approximation. The two-lag approximation

$$C_J(p) = 1 - \frac{0.165 p}{p + 0.0455} - \frac{0.335 p}{p + 0.3}$$

is rational; with two aerodynamic lag states the system becomes a six-state model whose eigenvalues are computed with `eig(aeroelastic_rational_roots)`. At 170 ft/s it gives $-1.0467 \pm 75.0313i$ and $-32.4827 \pm 66.1864i$, plus two real lag roots (-7.12 and -34.96), against the exact $-1.1504 \pm 75.8683i$ and $-31.4233 \pm 63.4831i$: the critical mode has about 9 % less damping. The flutter speeds are close (Table 10.2); a small error in the flutter speed does not imply small errors in damping elsewhere.

Quasi-steady aerodynamics. Setting $C = 1$ while keeping the non-circulatory terms ignores the wake memory. For the NASA case it predicts flutter at 100.01 ft/s; the dimensional case is already unstable at 0.01 m/s.

The p-k method. Evaluating the whole aerodynamic matrix at $p = i \operatorname{Im}(s)b/U$ while keeping the complex s in the structural terms [17, Eq. (4)] gives the same equation as the exact method on the imaginary axis, hence the same flutter speed, but different damping away from it (Figure 10.3). The p-k target is not analytic in s and is solved by Newton's method in two real variables, never by contour counting.

Table 10.2: Flutter speed with approximate aerodynamics.

Case	Exact	Jones (difference)	Quasi-steady
NASA [ft/s]	173.2624	172.8573 (−0.23 %)	100.01
Kaiser–Quero [m/s]	212.1729	210.9854 (−0.56 %)	unstable at 0.01

10.6 Parameter study

Varying only the plunge stiffness of the NASA case, so that ω_h/ω_α goes from 0.30 to 0.80, lowers the flutter speed from 197.8 to 135.7 ft/s: flutter is easier as the uncoupled frequencies approach each other (Figure 10.5). Every point is confirmed by contour counts at 98 % and 102 % of its flutter speed.

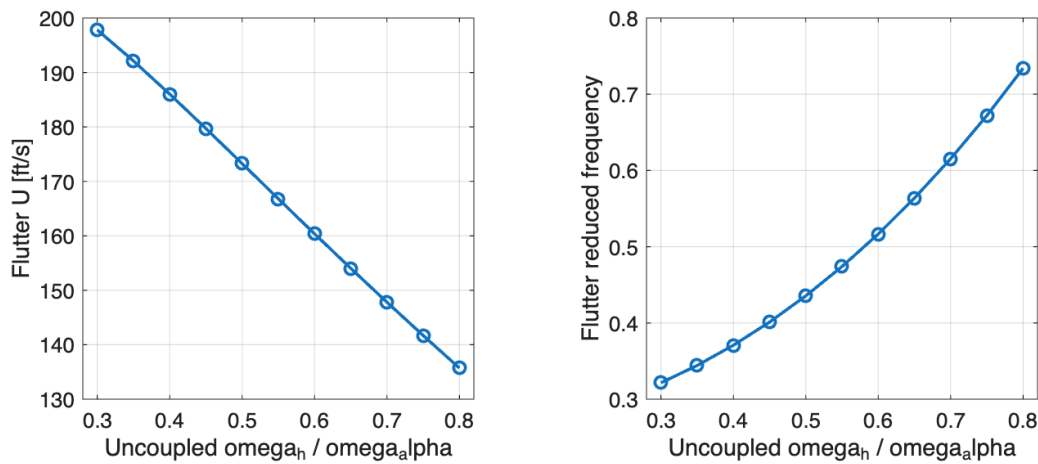


Figure 10.5: Flutter speed and reduced frequency versus the uncoupled frequency ratio (NASA case).

10.7 Validation and limitations

The study `examples/aeroelasticity/run_aeroelastic_study.m` and the regression tests check: the published flutter values; the Bessel form of C against the Hankel form, its limits and conjugate symmetry; the lift and moment formulas assembled independently of the matrix implementation; the still-air added mass; complete contour counts with two modes in the upper window at every speed of the sweeps, with enlarged windows and doubled sampling at selected speeds; zero unstable roots below and two above flutter wherever the count is conclusive, and a real unstable root above divergence; an independent harmonic flutter formulation (eigenvalues in U^2 of $Kq = U^2 B(k)q$ with Hankel functions), which agrees to 10^{-6} in speed; and the Jones state-space eigenvalues against the Jones characteristic function.

The model is linear, incompressible and two-dimensional; “exact” refers to evaluating it without rational approximation. The counts are complete within the stated rectangles; flutter candidates from Newton’s method are confirmed by the sweeps and counts, not by a global search in the (ω, U) plane.

Remark. This example required one change in the solver: Newton steps and finite-difference stencils are now kept inside the current cell, so that a function is never evaluated outside the search region. Before, a right-half-plane search could step onto the branch cut of C .

Chapter 11

Time responses without rational approximation

The previous chapters located poles and zeros. This chapter computes *time responses* — impulse, step and forced — of the same nonrational transfer functions, with the functions `cimpulse`, `cstep`, `clsim` and `cinvlaplace`.

11.1 Motivation

For a rational model, MATLAB's `impz`, `step` and `lsim` integrate a state-space realization in time. A transfer function with a delay, a diffusion term or unsteady aerodynamics has no finite realization, and the usual practice simulates a rational approximation instead: a Padé model of the delay, a truncated modal expansion of the PDE, or aerodynamic lag states. Chapter 7 showed that such approximations can change stability conclusions; they can change transients as well.

The response of a causal linear system at rest is the inverse Laplace transform of $G(s)U(s)$,

$$y(t) = \frac{1}{2\pi i} \int_{\sigma-i\infty}^{\sigma+i\infty} e^{st} G(s)U(s) ds, \quad (11.1)$$

where the Bromwich line $\operatorname{Re} s = \sigma$ lies to the right of every singularity of $G(s)U(s)$. Equation (11.1) involves only values of G : it can be evaluated numerically for any transfer function that can be evaluated. What is discretized is this integral, not the physical model, and the discretization is refined until it converges.

11.2 Problem statement and the domain assumption

The module computes zero-state responses of real, causal, single-input single-output systems: all internal states, including the memory of a delay line, of a PDE or of an aerodynamic wake, are zero at $t = 0$. A transfer function does not describe an arbitrary internal initial state, so no analogue of MATLAB's `initial` is provided.

The one piece of information that cannot be extracted from a function handle is where G is analytic. The user supplies it as `SingularityBound a`: the assertion that G is analytic for $\operatorname{Re} s > a$ (or directly as the line, `Abcissa`). For coefficient vectors and `tf/zpk/ss` models it is computed from the poles. Table 11.1 gives typical justifications.

Table 11.1: Justifying the inversion half-plane.

Transfer function	Bound	Justification
$1/(s + 1 + 0.5e^{-s})$	0	for $\operatorname{Re} s \geq 0$, $ s + 1 \geq 1 > 0.5 \geq 0.5e^{-s} $: no zeros of the denominator
$e^{-a\sqrt{s}}$ (diffusion)	0	only singularity: the branch cut $(-\infty, 0]$
$e^{-\tau s}/(s + 1)$	-1	pole at -1; $e^{-\tau s}$ is entire
unstable system	growth rate	the unstable poles must lie left of the line
aeroelastic section, 180 ft/s	5	the right-half-plane count of Chapter 10 gives the pair $2.13 \pm 74.8i$

A bounded root search helps to choose the bound but cannot prove it: a singularity outside the searched rectangle would be missed. The bound is therefore recorded as an assertion (`info.domainSource`). Choosing a line to the left of an unstable pole does not fail loudly; it computes a different, non-causal function (Figure 11.1, right).

11.3 Inversion on a Bromwich line

Discretization. On $s = \sigma + i\omega$, (11.1) is the Fourier integral of $f_\sigma(t) = e^{-\sigma t}f(t)$, with $F = GU$. Sampling F at $\omega_k = 2\pi k/P$ and applying the inverse FFT of length N gives, at $t_n = nP/N$,

$$\frac{1}{\Delta t} \text{IFFT}(F(\sigma + i\omega_k))_n = \sum_{m \geq 0} f_\sigma(t_n + mP) + E_{\text{band}}, \quad \Delta t = \frac{P}{N}, \quad (11.2)$$

where the sum is the periodic repetition of f_σ (aliasing in time; only $m \geq 0$ appears because f is causal) and E_{band} is the error of truncating the frequency band at $\pm\pi N/P$. Multiplying by $e^{\sigma t_n}$ recovers $f(t_n)$ plus the aliasing terms $e^{-m\sigma P}f(t_n + mP)$, $m \geq 1$. If $|f(t)| \lesssim Ce^{at}$, the first of them is of relative order $e^{-(\sigma-a)P}$. The solver chooses

$$\sigma = \max(a, 0) + \frac{\max\{12, -\ln(0.01 \min(\text{AbsTol}, \text{RelTol}))\}}{P},$$

which makes the aliasing contribution smaller than the requested tolerance by a factor of about 100. The price is the amplification $e^{\sigma t}$ of rounding errors when the weighting is undone, reported in `info.amplification`; very long horizons for growing responses are refused.

Refinement. Each sample is computed four times: with the reference period and resolution, with the period doubled, with the bandwidth doubled, and with the line shifted to the right by $0.5/P$. The three differences — period, bandwidth and shift — estimate the three error sources separately. A sample is accepted when they are below $\text{AbsTol} + \text{RelTol}|y|$ in two successive rounds; otherwise period and bandwidth are refined. Figure 11.1 (left) shows the three differences for a delayed first-order system. Zero-padding a previously sampled spectrum would add no information; each refinement evaluates G at new frequencies.

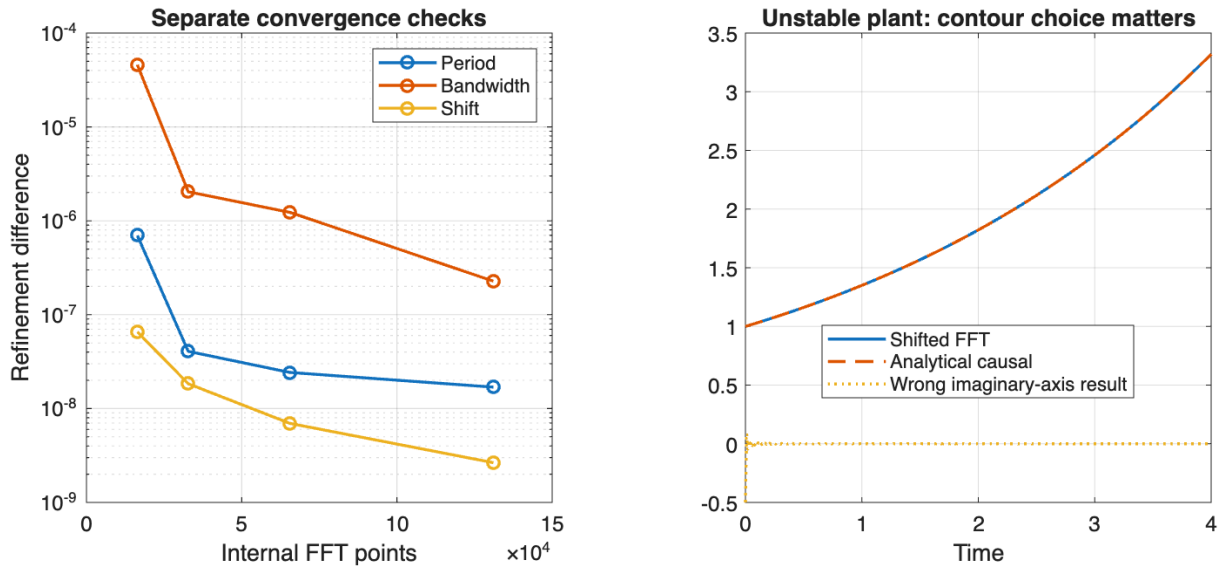


Figure 11.1: Left: period, bandwidth and shift differences during refinement ($e^{-0.713s}/(s+1)$). Right: for the unstable $1/(s-0.3)$, inversion on a line right of the pole gives the causal response $e^{0.3t}$; an inverse FFT on the imaginary axis gives a different function.

Independent methods. Two further methods accept arbitrary positive times and serve as cross-checks. `Method='dehoog'` sums the Fourier series of (11.2) with the continued-fraction acceleration of de Hoog et al. [11]; its quotient-difference table is a Padé-type device for summing the series, not a rational approximation of G . It checks the degree, the period and the line shift; each degree check doubles the degree, hence the sampled bandwidth, so that consecutive degrees cannot share a plateau that omits lightly damped poles just above the band (Chapter 12). `Method='quadrature'` integrates a truncated Bromwich integral with adaptive Gauss–Kronrod quadrature and checks the cutoff, the quadrature tolerance and the line; it avoids the conditioning limits of the quotient-difference table in double precision for late oscillatory responses. There is no automatic switching between methods.

11.4 Dirac terms, the origin and jumps

Direct terms. If $G = D + G_r$ with a constant D , the impulse response contains $D\delta(t)$ and, with a transport delay τ of an LTI model, $D\delta(t - \tau)$. A vector of samples cannot represent these; the returned impulse response is the ordinary part g_r , and the Dirac terms are listed in `info.singularTerms`. For function handles no hidden Dirac term can be excluded from finite samples: `cimpulse` requires the assertion `RegularImpulse=true`, with an optional known `Feedthrough`.

The origin. An impulse response with $g(0^+) = h_0 \neq 0$ jumps at $t = 0$, which would slow the convergence of the inversion. When h_0 is known — exactly for rational models, or through `InitialValue` — the solver inverts $G_r(s) - h_0/(s + \beta)$, whose inverse is continuous at zero, and adds $h_0 e^{-\beta t}$ back exactly; $\beta > 0$ is chosen so that the auxiliary pole lies left of the line. When h_0 is unknown, the value at $t = 0$ is returned as `NaN` and marked unresolved; positive times are unaffected. Kernels that are infinite at zero, such as $1/\sqrt{\pi t}$ for $G = s^{-1/2}$, are computed at positive times.

Jumps. At a jump of f the inversion integral converges to the midpoint of the jump. Known jumps — the direct term of a step, the delay of an LTI model — are placed exactly with the

right-continuous convention. A jump unknown to the solver, such as a delay inside a function handle, leaves the single sample at that time unresolved, while the other samples converge.

11.5 Forced response and input reconstruction

`clsim` computes the response to an input given by samples u_j at uniform times t_j (or a handle sampled there). The sampled input must be completed between samples; the user chooses a zero-order hold (ZOH) or a first-order hold (FOH), and the response is computed exactly for that reconstructed input. Convolution of sampled impulse responses would require $g(0)$, which may be infinite; instead the solver uses integrated kernels. With $G = D + G_r$, define the step and ramp responses $S_r = \mathcal{L}^{-1}\{G_r/s\}$ and $R_r = \mathcal{L}^{-1}\{G_r/s^2\}$, both zero before $t = 0$. A held input is a sum of delayed steps, one per jump $u_j - u_{j-1}$ (with $u_{-1} = 0$):

$$y(t) = Du(t) + \sum_{t_j \leq t} (u_j - u_{j-1}) S_r(t - t_j). \quad (11.3)$$

A piecewise-linear input with slopes $m_j = (u_{j+1} - u_j)/(t_{j+1} - t_j)$ is a step u_0 plus delayed ramps, one per change of slope:

$$y(t) = Du(t) + u_0 S_r(t) + m_0 R_r(t) + \sum_{j \geq 1, t_j \leq t} (m_j - m_{j-1}) R_r(t - t_j). \quad (11.4)$$

On a uniform grid these sums are discrete linear convolutions. They are computed with zero-padded FFTs (length at least $2n - 1$, so that the convolution is linear, not circular), after the exact rescaling $\sum_j w_j K(t_n - t_j) = e^{\sigma t_n} \sum_j (w_j e^{-\sigma t_j}) (K(t_n - t_j) e^{-\sigma(t_n - t_j)})$, which limits the dynamic range for growing kernels. The kernel error estimates are propagated through the same sums. For rational models the result agrees with MATLAB's `lsim` with the same hold to about 10^{-10} (ZOH) and 10^{-9} (FOH). With an `InputDelay` T and $u_0 \neq 0$, `lsim`'s FOH lets the delayed input rise linearly from zero over the sample interval ending at T , while `clsim` keeps the jump of the interpolant at T ; the two then differ by about $g(0^+) u_0 \Delta t/2$ just after T . The delay-free response shifted by T confirms `clsim` to 10^{-8} .

11.5.1 Several inputs with the same model

In (11.3) and its FOH analogue, only the weights depend on the input; the kernels S and R depend on the system alone. When many inputs are applied to the same system, such as a sine sweep, a family of reference signals or random disturbances, the inversions can be done once. `ckernel` performs them and returns a read-only object `K`; `clsim` accepts `K` in place of G and only evaluates the convolution. The example below “measures” the frequency response of the delayed loop by simulation, fitting a sinusoid to the steady part of each output:

```
G = @(s) 1./(s + 1 + 0.5*exp(-s));
t = (0:0.02:20).';
K = ckernel(G, t, 'SingularityBound', 0);    % inversions, once
w = 0.5:0.25:2; late = t > 10;
for k = 1:numel(w)
    [y,~,info] = clsim(K, sin(w(k)*t), t);    % convolution only
    ab = [sin(w(k)*t(late)) cos(w(k)*t(late))] \ y(late);
    gain(k) = norm(ab);                       % compare with abs(G(1i*w(k)))
end
```

The seven gains agree with $|G(i\omega)|$ to about 10^{-4} , the size of the transient left at $t = 10$ (the rightmost characteristic root has $\text{Re } s \approx -1.10$). Every `info.evaluations` is zero, and the script takes about 1.2s against 2.8s for seven ordinary calls.

What is stored. FOH preparation stores R_r and S_r , so a later input may start at $u_0 \neq 0$; ZOH stores S_r only. The direct term and a known transport delay are kept. `K` holds numbers for one model, one grid and one hold, not the function handle: changing a parameter, the time samples or the hold requires a new preparation, and `clsim` rejects a mismatched grid, hold or model option. Each call is an independent zero-state experiment.

Two tolerances. The error of the stored kernels is amplified by each input. With a step-kernel error estimate ϵ_S , the ZOH output error is estimated as

$$\epsilon_y(t_n) = \sum_{j=0}^n |u_j - u_{j-1}| \epsilon_S(t_n - t_j), \quad (11.5)$$

and FOH propagates ϵ_R through the slope changes and adds $|u_0| \epsilon_S$. The kernels are therefore prepared at tighter tolerances (10^{-8} absolute, 10^{-6} relative) than the output (10^{-6} , 10^{-4}), and ϵ_y is recomputed and checked for every input. A large or rapidly varying input may fail that check even though the preparation converged; the samples are then marked unresolved and the kernels are *not* refined automatically. The user either prepares tighter kernels or relaxes an unnecessarily strict output tolerance. The estimate is conservative (true errors near 10^{-9} when it first exceeds 10^{-6} in the tests), and it retains the finite-bandwidth caveat of the inversion: it is not a certified bound.

11.6 Validation

The study `examples/time_response/run_time_response_study.m` compares step responses computed from transfer functions with references obtained by different means (Table 11.2, Figure 11.2). The finite-element and modal truncations belong to the references, not to the inversion.

Table 11.2: Maximum absolute errors of step responses on $0 \leq t \leq 6$.

Transfer function	Independent reference	Error
$1/(s + 1 + 0.5e^{-s})$	method-of-steps series	1.2×10^{-8}
$e^{-0.7\sqrt{s}}$	$\text{erfc}(0.7/(2\sqrt{t}))$	4.6×10^{-8}
heated rod (Chapter 8)	separation of variables, 150 modes	1.1×10^{-7}
beam and oscillator (Chapter 9)	finite elements, 32 elements	2.5×10^{-5}
aeroelastic pitch, 160 and 180 ft/s	quadrature versus FFT	$< 10^{-10}$

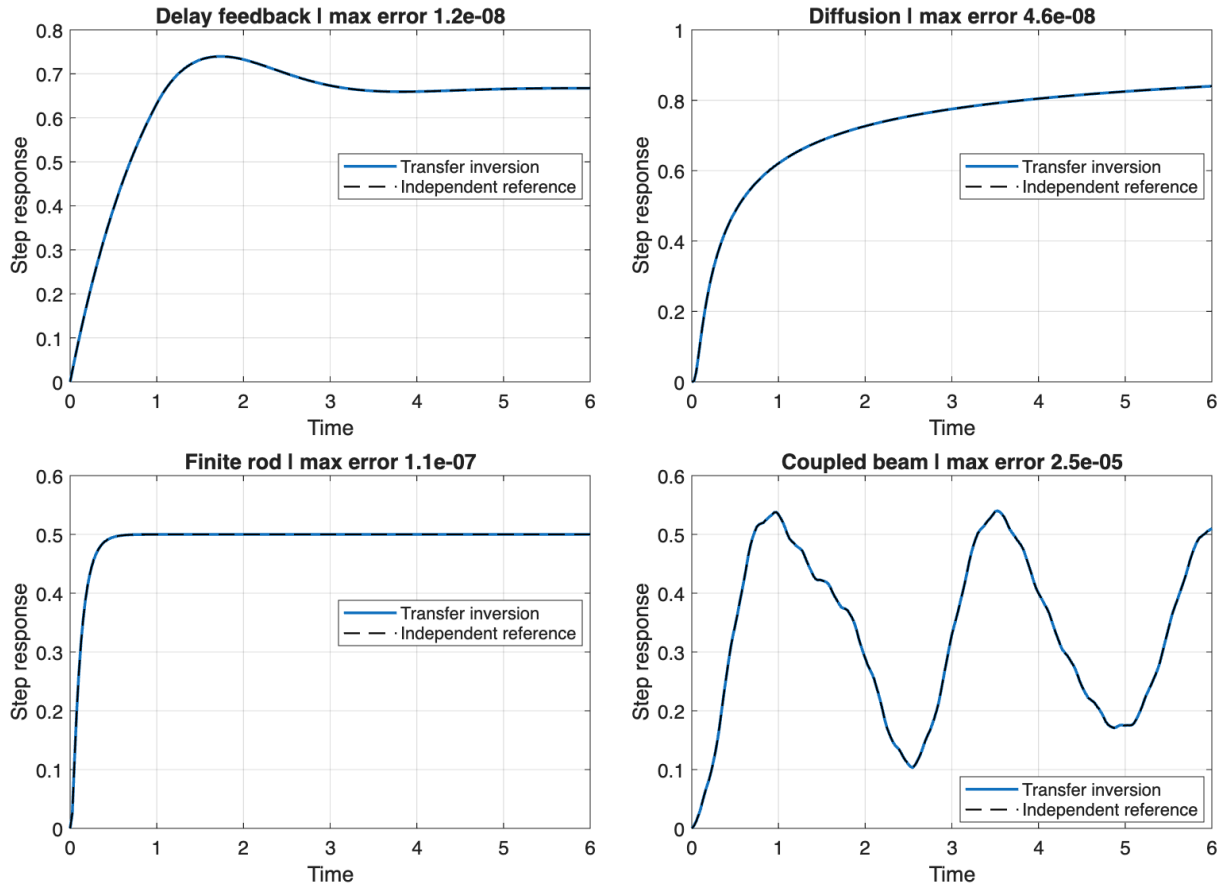


Figure 11.2: Step responses computed from the transfer functions (solid) and independent references (dashed).

The aeroelastic example of Chapter 10 gives the pitch response to a pitch moment: the transfer function is the second entry of $D(s, U)^{-1}[0, 1]^T$ — an input/output channel, not the characteristic determinant — with exact Theodorsen aerodynamics (Figure 11.3). Below flutter the oscillation decays to the static deflection; above flutter it grows at the rate predicted by the right-half-plane count.

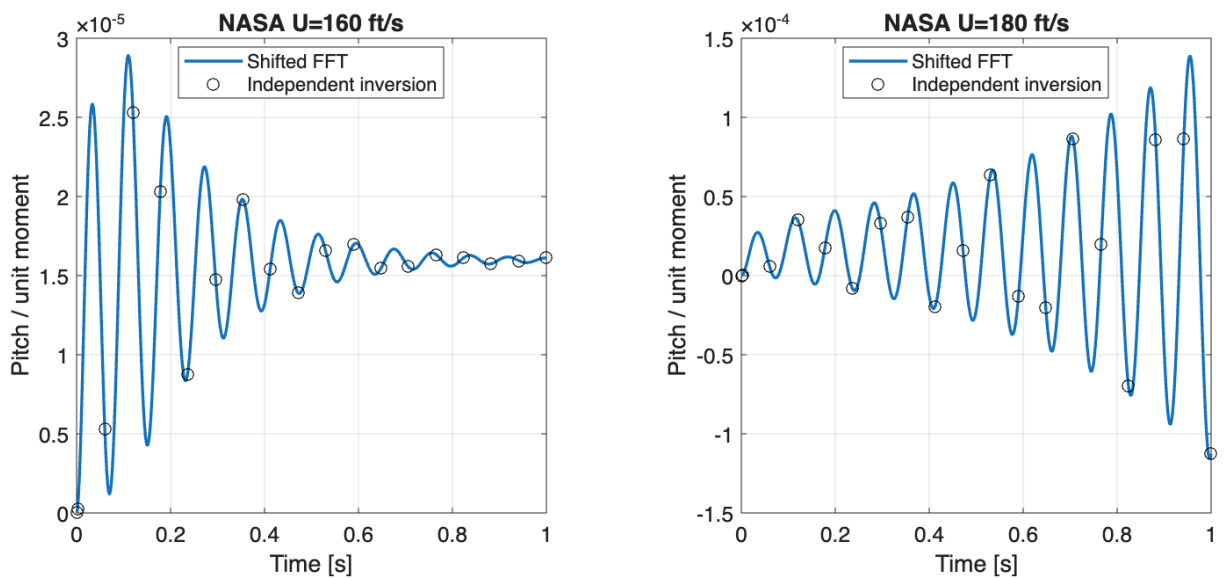


Figure 11.3: Pitch response to a unit pitch-moment step of the NASA section, below and above flutter. Circles: independent Bromwich quadrature.

The regression tests add analytical first-order, delayed, unstable, marginal and fractional cases, MATLAB's `lsim` with both holds, direct terms, unknown initial limits, scalar-only evaluators and resource limits.

11.7 Limitations

- Zero-state, real, single-input single-output responses; for matrix models, a chosen input/output channel.
- The inversion half-plane is an assertion for function handles.
- The input is the chosen ZOH or FOH interpolant of its samples.
- Jumps and corners unknown to the solver leave the sample at that time unresolved.
- Convergence is numerical (successive refinements agree); a feature outside the sampled band cannot be excluded, and `info.certified` is always false.

Chapter 12

Flutter of a continuous wing with nothing discretized

Chapter 10 treated a wing *section*: two degrees of freedom and exact Theodorsen aerodynamics. A wing is a flexible beam with infinitely many modes. The standard route to its flutter involves two independent approximations:

1. a spatial discretization, usually finite elements, followed by a truncation to N vibration modes;
2. a rational fit of the unsteady aerodynamic forces, tabulated at a few reduced frequencies, which turns the system into a large state-space model [13, 17].

Each approximation raises its own convergence question, and the fit can introduce spurious poles. This chapter computes the poles, the stability, the flutter speed and the time responses of the classical Goland wing [14] with *neither* approximation. The bending and torsion equations are solved exactly along the span, and Theodorsen's function is evaluated from Bessel functions at every complex s . The only modelling assumption on the aerodynamics is strip theory. The code is in `examples/continuous_wing/` and `examples/models/wing_*.m`.

12.1 Model

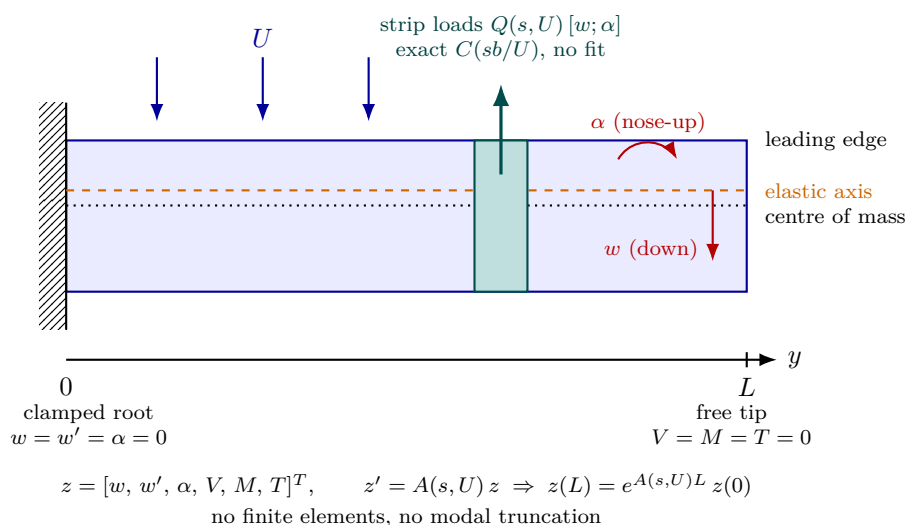


Figure 12.1: The continuous wing. Each spanwise strip carries the exact two-dimensional Theodorsen load; the state $z(y)$ is propagated exactly from the clamped root to the free tip.

The wing (Figure 12.1) is a straight cantilever of span L . Its section at position y moves with the plunge $w(y, t)$, positive downwards at the elastic axis, and the nose-up pitch $\alpha(y, t)$. Bending follows Euler–Bernoulli theory (stiffness EI) and torsion Saint-Venant theory (stiffness GJ). The mass per unit span is μ , the pitch inertia I_α and the static moment $S = \mu x_\alpha b$; the static moment couples the two motions through inertia. For a motion e^{st} , the section equations are

$$EI w'''' + s^2(\mu w + S\alpha) = Q_{11}w + Q_{12}\alpha, \quad -GJ \alpha'' + s^2(Sw + I_\alpha \alpha) = Q_{21}w + Q_{22}\alpha, \quad (12.1)$$

with $' = d/dy$. $Q(s, U)$ is the strip load matrix of Chapter 10, $Q = \pi \rho U^2 (p^2 A_2 + p A_1 + A_0)$ with $p = sb/U$ and Theodorsen's function $C(p) = K_1(p)/(K_0(p) + K_1(p))$ [17, 26]. The example uses the Goland parameters as tabulated by Cal [8, Table 2.4]:

- $L = 6.096$ m, $b = 0.9144$ m;
- $EI = 9.773 \times 10^6$ N m², $GJ = 9.876 \times 10^5$ N m²;
- $\mu = 35.717$ kg/m, $I_\alpha = 8.642$ kg m;
- elastic axis at 33 % of the chord, centre of mass at 43 %;
- $\rho = 1.225$ kg/m³.

12.2 An exact characteristic function

With s as a parameter, (12.1) is a linear ordinary differential equation in y . Collect displacement, slope, twist, shear force, bending moment and torque in the state vector

$$z = [w, w', \alpha, V, M, T]^T, \quad M = EI w'', \quad V = -M', \quad T = GJ \alpha'. \quad (12.2)$$

Then $z' = A(s, U)z$, where the only nonzero entries of A are

$$A_{12} = 1, \quad A_{25} = \frac{1}{EI}, \quad A_{36} = \frac{1}{GJ}, \quad A_{54} = -1, \quad [A_{41} \ A_{43}] = D_{1,:}, \quad [A_{61} \ A_{63}] = D_{2,:}, \quad (12.3)$$

with $D(s, U) = s^2 \begin{bmatrix} \mu & S \\ S & I_\alpha \end{bmatrix} - Q(s, U)$. On a span of constant properties, the solution is exact: $z(L) = e^{A(s, U)L} z(0)$. For a wing whose properties vary, the span is divided into N strips of constant properties, with one propagator $E_j = e^{A_j \ell_j}$ each.

Let $x = [z_0; \dots; z_N]$ be the (fixed-scaled) states at the strip boundaries. The clamped root, the N propagation relations and the free tip give a square homogeneous system $K(s, U)x = 0$:

$$w_0 = w'_0 = \alpha_0 = 0, \quad z_j - E_j z_{j-1} = 0 \quad (j = 1, \dots, N), \quad V_N = M_N = T_N = 0. \quad (12.4)$$

The wing has a mode at s exactly when $K(s, U)$ is singular, so the characteristic function is

$$\Delta(s, U) = \det K(s, U). \quad (12.5)$$

Three properties make (12.5) suitable for the argument principle:

- The matrix exponential is an entire function of the entries of A . Δ is therefore entire in vacuum, and analytic except on the negative real axis (the branch cut of C) with air. So **AssumeAnalytic** holds in any rectangle that avoids that axis.
- Nothing is inverted. Write $E = \prod_j E_j$ in blocks acting on $q = [w, w', \alpha]$ and $p = [V, M, T]$. The conditions $q_0 = 0, p_N = 0$ give $E_{pp}(s)p_0 = 0$, a reduced characteristic function $\det E_{pp}$ obtained without inversion; it equals $\det K$. What must be avoided are condensations that invert s -dependent blocks, such as hybrid or dynamic-stiffness element matrices. They create artificial poles where those blocks are singular. K is preferred to E_{pp} for conditioning: its multiple-shooting structure keeps each exponential moderate.
- For identical strips $e^{AL} = (e^{AL/N})^N$, so Δ does not depend on N . The study confirms this to 1.4×10^{-14} for $N = 1, 2, 4, 8$. For a uniform wing the strips are bookkeeping, not discretization.

The state is scaled by constants, independent of s , to improve the conditioning of K .

12.3 Stability, root locus and flutter

In vacuum and without inertial coupling, the zeros of Δ found by `croots` reproduce the closed-form cantilever frequencies ($\beta_n^2 \sqrt{EI/\mu}/L^2$ in bending, $(2n-1)\pi\sqrt{GJ/I_\alpha}/(2L)$ in torsion). The eight frequencies below 900 rad/s agree to 4×10^{-14} . A finite-element model reaches this accuracy only in the limit of a fine mesh.

Stability is decided, as in Chapter 10, by counting the zeros of Δ in a rectangle of the right half-plane. This rectangle includes the real axis, so a divergence root would also be counted. In $[10^{-3}, 60] \times [-400, 400]$ the count is 0 at 120 m/s and 2 at 150 m/s, where the unstable pair is $3.70 \pm 68.2i$. Each count takes a fraction of a second.

Figure 12.2 follows the four lowest modes from 0 to 180 m/s. The first torsion branch (93 rad/s in still air) moves down towards the first bending frequency, turns, and crosses the imaginary axis near 70 rad/s: classical bending–torsion flutter. The flutter speed is the zero of the spectral abscissa, found by `fzero`. A local Newton solution of $\Delta(i\omega, U) = 0$ in the real unknowns (U, ω) agrees to 10^{-9} :

$$U_f = 136.983977 \text{ m/s}, \quad \omega_f = 70.033013 \text{ rad/s}. \quad (12.6)$$

Larger windows up to 700 rad/s at $0.95 U_f$ and $1.05 U_f$ contain six modes in the upper half-plane, with 0 and 1 unstable, respectively.

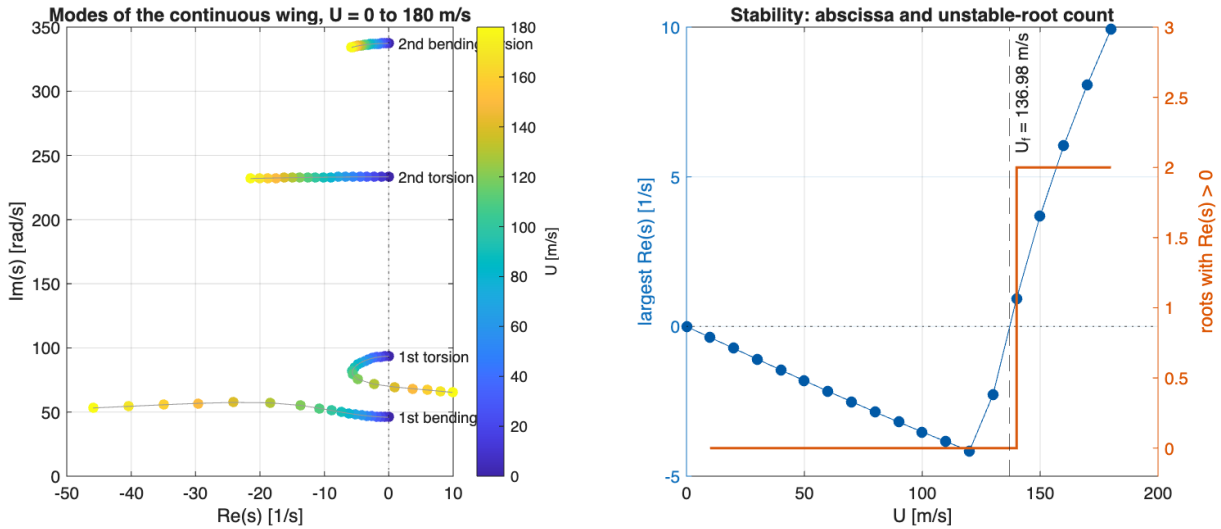


Figure 12.2: Left: modes of the continuous Goland wing from 0 to 180 m/s, labelled by their still-air character. Right: spectral abscissa and number of roots counted in the right half-plane.

12.4 A finite-element model converges to the continuous answer

An independent calculation follows the usual route:

- cubic Hermite elements for bending and linear elements for torsion;
- projection onto the vacuum modes;
- air loads assembled from Theodorsen's lift and moment formulas, with $C(k)$ written in Hankel functions rather than the modified Bessel functions of the main model.

Its flutter speed decreases towards the continuous value, and the error falls by a factor of four each time the mesh is doubled (Table 12.1, Figure 12.3). This is second-order convergence to the limit that ContourRoots computes directly.

Table 12.1: Flutter speed of the finite-element model versus the continuous result.

Elements	Modes	U_f (m/s)	$ U_f^{\text{FE}}/U_f - 1 $
8	8	137.308301	2.4×10^{-3}
16	12	137.065051	5.9×10^{-4}
32	20	137.004259	1.5×10^{-4}
64	28	136.989048	3.7×10^{-5}
128	36	136.985245	9.3×10^{-6}
continuous (ContourRoots)		136.983977	—

For a tapered wing the strips do approximate the geometry, since the properties are replaced by constant values within each strip. For a synthetic wing whose chord decreases linearly by 30 %, the flutter speed takes the values 151.58, 155.81, 156.81, 157.05 and 157.11 m/s for 2, 4, 8, 16 and 32 strips. The solution within each strip remains exact, and there is still no modal truncation and no aerodynamic fit.

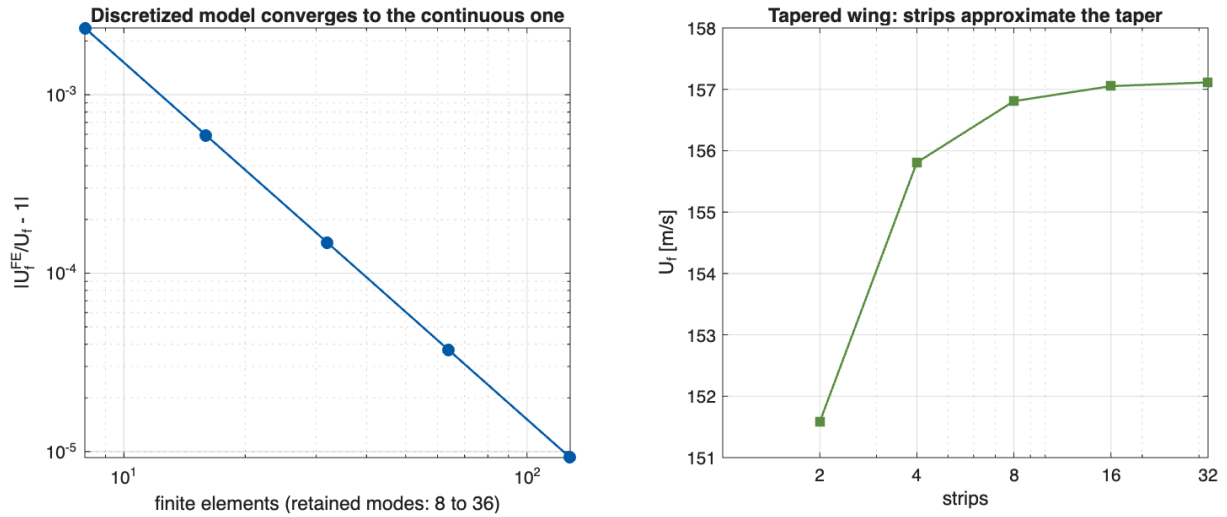


Figure 12.3: Left: convergence of the finite-element flutter speed to the continuous value. Right: strip refinement of a tapered wing.

12.5 Time responses

The tip transfer function, for example tip force to tip deflection, follows from solving $K(s, U)x = f$ with the load in the tip rows. It is evaluated at any complex s , so `cstep` and `clsim` (Chapter 11) give the response directly. There is no modal ordinary differential equation and no aerodynamic lag state.

At the high frequencies sampled by the inversion, a bending wave grows or decays like $e^{\kappa y}$ with $\kappa = (|s|^2 \mu / EI)^{1/4}$. At 10^4 rad/s, $\kappa L \approx 27$. The transfer evaluation therefore subdivides each strip into identical pieces with $\kappa \ell \leq 3$. This is multiple shooting [2], and it is exact for constant properties: results with n and $2n$ pieces agree to 10^{-12} .

The inversion line is placed with `SingularityBound=5`. This is an assumption supported, not proved, by the counts: it lies to the right of the unstable pair found at 150 m/s, but the counts cover a finite rectangle, and excluding singularities outside it would need an additional argument about the rest of the spectrum. The output is expressed in mm per kN, so that the absolute tolerance 10^{-3} is one micrometre.

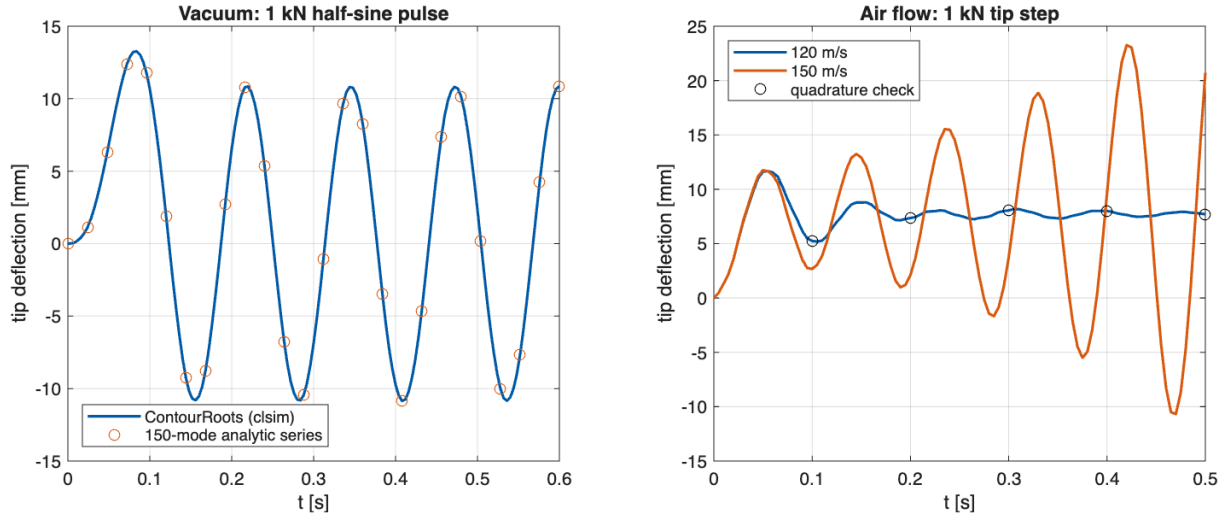


Figure 12.4: Left: vacuum response to a 1 kN half-sine tip pulse against an analytic 150-mode series. Right: 1 kN tip step below and above flutter; circles: adaptive quadrature.

Figure 12.4 shows the results. In vacuum, the pulse response agrees with the analytic series to 1.9×10^{-6} mm on a 13 mm response. Below flutter, the tip step settles near its static deflection; above flutter it grows at the rate $\text{Re } s = 3.70$ predicted by the root count. The FFT and adaptive quadrature agree to 7×10^{-4} mm. In version 0.6.0 the de Hoog method converged here to a plateau 0.04 mm (0.5 %) off, with all its internal checks in agreement. Its truncated series omitted lightly damped higher modes, and successive degrees only 1.4 times apart could not reveal them. Since 0.6.1 every degree check doubles the sampled bandwidth. De Hoog then stays within the requested tolerance of the FFT (largest difference 6.7×10^{-3} mm at five times; tolerance about 9×10^{-3} mm). A regression test keeps the case.

12.6 Scope

The aerodynamics is two-dimensional strip theory: incompressible, with no spanwise interaction and no stall. The structure is linear and undamped, with no shear deformation or warping. These are modelling choices, and the solver adds no approximation to them. Root counts are complete in the chosen rectangles, conditional on analyticity, and time responses start from rest. Δ is the determinant of a small dense matrix. Matrix-valued methods, which would count characteristic values without forming a determinant, are discussed in the development notes of the toolbox. The same exact dynamic-stiffness idea underlies the wing analyses of Banerjee [4].

Chapter 13

Building a wing from parts; MIMO responses

Chapter 12 treated the Goland wing as a single object, with one characteristic function and one tip transfer function. This chapter addresses two further needs. The first is *assembly from parts*: describe each substructure by its ports and connect the parts, the idea of the transfer-matrix and dynamic-stiffness methods for beams [4]. The second is *several inputs at once*: the response to simultaneous loads, the analogue of `lsim` for a multi-input, multi-output system. Nothing is discretized. Each block is the exact solution of the beam equations with exact Theodorsen strip loads; the matrices are small because they describe ports, not truncated modes. The code is `examples/continuous_wing/hybrid_quickstart.m`.

13.1 A strip as a hybrid two-port

A strip of constant properties maps its left state to its right state exactly, with displacements $q = [w, w', \alpha]$ and internal loads $p = [V, M, T]$:

$$\begin{bmatrix} q_R \\ p_R \end{bmatrix} = E(s) \begin{bmatrix} q_L \\ p_L \end{bmatrix}, \quad E = e^{A(s)\ell} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}. \quad (13.1)$$

A neighbour imposes a displacement at one end and a load at the other. The strip is therefore rewritten with mixed inputs: q_L and the load $f_R = p_R$ are given, and $f_L = -p_L$ and q_R are computed. Solving $p_R = cq_L + dp_L$ for p_L gives

$$\begin{bmatrix} f_L \\ q_R \end{bmatrix} = H(s) \begin{bmatrix} q_L \\ f_R \end{bmatrix}, \quad H = \begin{bmatrix} d^{-1}c & -d^{-1} \\ a - bd^{-1}c & bd^{-1} \end{bmatrix}. \quad (13.2)$$

These are the hybrid (h-) parameters of a two-port, familiar from circuit theory, with three displacements and three loads per port. H is 6×6 because the beam has six state variables, and its entries are transcendental functions of s (Figure 13.1).

13.2 Connection is not multiplication

At a junction, the displacements are shared and the mutual loads balance: $q_R^A = q_L^B = q_I$ and $f_R^A + f_L^B = 0$. With H^A, H^B partitioned into 3×3 blocks,

$$(I + H_{22}^A H_{11}^B) q_I = H_{21}^A q_L - H_{22}^A H_{12}^B f_R, \quad (13.3)$$

and eliminating q_I yields the hybrid matrix H^{AB} of the pair, with the same port structure (`wing_hybrid_join`). The product $H^A H^B$ would feed the loads of A into the displacement

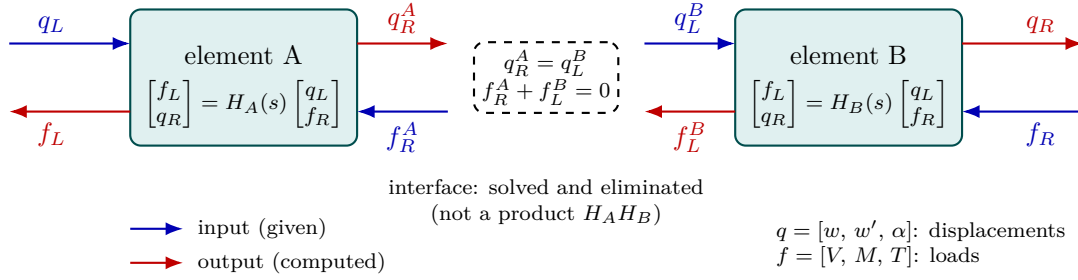


Figure 13.1: Two hybrid elements. Inputs (blue) are given and outputs (red) computed. The interface conditions are solved and eliminated.

inputs of B and is meaningless. Two joined half-span strips reproduce one full-span strip to 3×10^{-16} . With a clamped root ($q_L = 0$) and tip loads $u = f_R$, the tip displacements are $q_R = H_{22}^{AB} u$: H_{22}^{AB} is the 3×3 tip transfer matrix.

The implicit assembly of Chapter 12 keeps the interface states as unknowns and gives $K(s)x = Bu$, $y = Cx$, that is $G = CK^{-1}B$. The structured model `cdyn` represents it, and `ceval` evaluates all nine channels with one factorization of K . The two assemblies impose the same conditions and differ only in when the interface unknowns are eliminated. At $s = 3 + 70i$ and 120 m/s they agree to 2×10^{-15} .

13.3 Artificial poles of the hybrid form

The hybrid form inverts d and $I + H_{22}^A H_{11}^B$, which depend on s . d is singular exactly when the piece alone, clamped at its left end and free at its right end, has a mode. That is a property of the piece, not of the assembled wing. In vacuum, a half-span piece has such a mode at $4 \times 49.49 = 197.97$ rad/s. The whole wing has none there (its nearest mode is at 261 rad/s), and its implicit transfer is finite, while the hybrid matrix of the piece does not exist. These artificial poles are the reason why the characteristic function must be built from the implicit matrix: $\det H$ has poles that belong to no physical mode. The functions report a singular pivot (`available=false`, or an error from the transfer evaluators) and never regularize it. With air, the artificial poles leave the imaginary axis; along $\text{Re } s = 6$ the two assemblies agree to 1.6×10^{-9} up to 2×10^4 rad/s.

13.4 Matrix models and simultaneous inputs

`cmimo` builds a transfer matrix, from channels, a constant matrix or a matrix-valued handle; `cdyn` builds a structured model $H(s)x = B(s)u$, $y = C(s)x + D(s)u$. `clsim`, `cstep`, `cimpulse` and `ckernel` accept both. `clsim` takes one input column per input and returns one column per output, with all inputs acting together. `cstep` and `cimpulse` return $N_t \times n_y \times n_u$ arrays, one experiment per input. Each channel uses the scalar inversion of Chapter 11. Evaluations of the matrix model are shared between channels within one call, and a kernel bank from `ckernel` stores the kernels of every channel.

Version 0.9.0 adds `'SharedGrid', true` to matrix `ckernel` preparation with FFT or de Hoog. All channels and the step/ramp kernels share spectral nodes and refine until the worst channel meets its own output tolerance at two successive levels. Each structured node uses one factorization with all inputs as right-hand sides. Delays remain exact time shifts and direct terms retain their analytical treatment. The shared line uses the tightest output tolerance and the largest singularity bound. Independent channel preparation remains the default: a common grid can cost more for cheap channels and requires matrix sample storage. Shared quadrature and

parallel evaluation are not implemented. The numeric snapshot schema and cancellation-aware `clsim` error propagation are unchanged; saved 0.8.0 banks still load.

Figure 13.2 shows the tip response of the wing at 120 m/s to a force $1 + 0.2 \sin 10t$ kN and a torque $0.3 \cos 5t$ kNm. One bank is prepared (about 30 s), and three `clsim` calls give the response to both loads, to the force alone and to the torque alone. Each call takes milliseconds and evaluates the wing model zero times. The coupling is strong in both directions. The force twists the wing about as much as the torque, and the torque raises the tip by up to 4 mm, partly cancelling the deflection due to the force. The coupling is inertial (centre of mass behind the elastic axis) and aerodynamic. The combined response equals the sum of the two contributions to 9×10^{-16} . The inversion bound 5 is an assumption supported by the root counts of Chapter 12, not a proof.

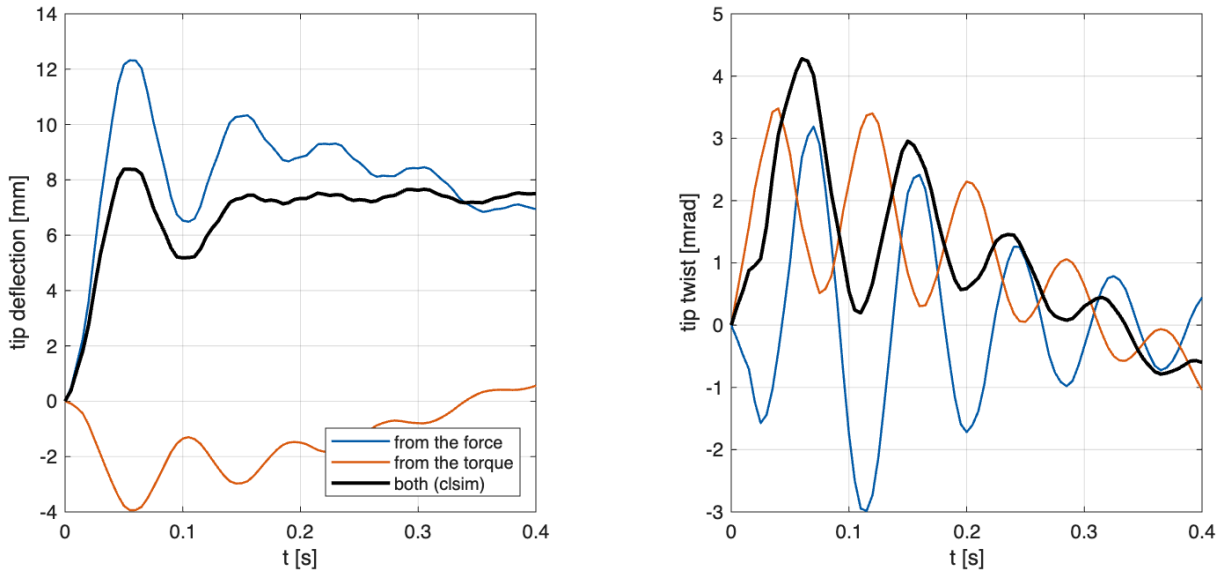


Figure 13.2: Tip deflection and twist at 120 m/s: contributions of the force and of the torque, and the response to both (black).

Each output is a sum of channel contributions, and so is its error. `clsim` adds the estimated channel errors of each output and compares the total with $\text{AbsTol}_i + \text{RelTol}|y_i|$, with one absolute tolerance per output in its own units. Destructive cancellation between channels therefore shows up as an unresolved output instead of being hidden.

13.5 Verification

- **Transfer matrices.** Hybrid and implicit tip transfer matrices, all nine channels, agree to 1.6×10^{-13} . This holds for dry, uniform and tapered wings, with 1, 2, 4 and 8 strips, at five complex frequencies.
- **Structured model.** `cdyn` agrees with the scalar tip transfer of Chapter 12 to 2×10^{-14} .
- **Time responses.** The hybrid and implicit kernel banks agree to 5×10^{-14} . The matrix bank agrees with the sum of independent scalar `clsim` calls within the requested output tolerance: 3×10^{-4} mm against 10^{-2} .
- **A rational oracle.** A 2×2 rational model with a different transport delay in each channel agrees with MATLAB's `lsim` with zero-order hold to 1.4×10^{-9} .

These checks show that the representations agree; the physical references remain those of Chapter 12.

13.6 Scope

The new functions provide matrix models and MIMO zero-state responses. Chapter 14 adds characteristic searches directly on the analytic implicit matrix K . General MIMO transfer poles and transmission zeros remain unimplemented. The hybrid connector is an example for a chain of beam strips, not a general network assembler; its meromorphic port matrix is not an analytic characteristic matrix.

Chapter 14

Matrix modes without determinants

14.1 Characteristic modes are not transfer poles

Version 0.8.0 adds `cmodes`, which searches a square analytic matrix $K(s)$ directly. A characteristic value λ satisfies $K(\lambda)v = 0$ for some nonzero v . For a structured model $G = CK^{-1}B + D$, such a value need not be visible as a transfer pole: the input may not excite it or the output may not observe it. The new function deliberately does not perform that visibility analysis.

```
K = @(s) [s+1 .2*exp(-s); 0 s+2];
Kp = @(s) [1 -.2*exp(-s); 0 1];
box = [-3 0 -1 1];
[r,info] = cmodes(K,box,'AssumeAnalytic',true, ...
    'Derivative',Kp);
M = cdyn(K,[1;0],[1 0],0,'Dimensions',[2 1 1]);
[m,im] = cmodes(M,box,'AssumeAnalytic',true);
% Both searches return -1 and -2, but G(s) = 1/(s+1).
```

`AssumeAnalytic=true` asserts that K is regular and analytic in a neighborhood of the closed rectangle. The boundary must be nonsingular. A `cdyn` argument supplies only its K factor and domain guard; the other factors are not evaluated. Direct handles must return a fixed-size square matrix. Sparse inputs are accepted, but the initial engine uses dense LU/SVD and is intended for small to moderate matrices.

14.2 Counting without a product of pivots

Jacobi's identity connects the matrix and scalar argument principles:

$$N = \frac{1}{2\pi i} \oint_{\Gamma} \text{tr}(K(s)^{-1} K'(s)) ds = \frac{1}{2\pi} \Delta_{\Gamma} \arg \det K(s). \quad (14.1)$$

Choose diagonal nonsingular scalings L_0, R_0 at a regular reference point and freeze them. Then $A = L_0 K R_0$ has the same characteristic values. At each contour node compute $PA = LU$. The logarithmic determinant magnitude is $\sum_j \log |U_{jj}|$, and its phase is the sum of the pivot phases plus the permutation phase. There is no multiplication of pivots. Neither the changing pivot choices nor singular values are treated as analytic functions of s .

Bounded contour search for matrix modes

1. Select fixed scaling at an admissible regular reference node.
2. Sample the counterclockwise rectangle. Refine until phase/log-magnitude increments are resolved and integer winding agrees twice successively.
3. If an analytic derivative is supplied, cross-check the outer count by integrating $\text{tr}(A \setminus A')$.
4. Refine candidate locations by a damped bordered Newton solve; multiple-count cells can also use the logarithmic derivative.
5. Confirm a candidate by a small local contour. Compare its algebraic count with numerical nullity and the projected derivative.
6. Otherwise split the cell, shifting internal split lines if needed. Accept children only when their counts sum to the parent count.
7. Report unresolved cells/clusters when any work or resolution limit is reached. Never replace failure with a successful empty spectrum.

The bordered simple-mode step solves

$$\begin{bmatrix} A(\lambda) & A'(\lambda)v \\ v^* & 0 \end{bmatrix} \begin{bmatrix} \delta v \\ \delta \lambda \end{bmatrix} = - \begin{bmatrix} A(\lambda)v \\ 0 \end{bmatrix}, \quad \|v\|_2 = 1. \quad (14.2)$$

The next iterate uses $\lambda + \delta\lambda$ with damping inside the cell. Without a supplied derivative, four-direction finite differences are used only for refinement; the trace cross-check is explicitly marked unavailable. Singular vectors supply left/right residual checks, not the analytic target of a scalar zero search. Vectors are mapped back as $v = R_0 \tilde{v}$ and $w = L_0^* \tilde{w}$ and normalized in the original coordinates.

14.3 Multiplicity, clusters and numerical limits

For $K = (s+1)I_2$, the algebraic count and nullity are both two; the projected derivative is nonsingular, supporting a numerical semisimple double mode. For

$$K = \begin{bmatrix} s+1 & 1 \\ 0 & s+1 \end{bmatrix}, \quad (14.3)$$

the count is two but nullity is one. The initial engine returns a counted cluster with unknown local multiplicity, not an invented simple mode or a computed Jordan chain. Very close distinct roots may also remain clusters. `countComplete` can therefore be true while `complete` is false. The returned representative must be read together with `localCounts`, `multiplicity`, `nullity` and `clusters`.

Default local confirmation squares have half-width $10^{-8}(1+|\lambda|)$; rank decisions use 10^{-8} times the fixed scaled reference norm. Residuals use that same regular-reference norm rather than the vanishing matrix norm at a mode. Unscaled residuals are also returned. No partial multiplicities, contour moments or sparse large-scale extraction are claimed. Subdivision is not limited to the matrix dimension: the scalar matrix $\sinh s$ already has seven roots in $[-1, 1] \times [-10, 10]$.

14.4 Determinant and continuous-wing comparisons

For $K = c \begin{bmatrix} 0 & s+2 \\ s+1 & 0 \end{bmatrix}$, $c = 10^{-200}$ or 10^{200} , the raw determinant at $s = 0.5$ is zero or negative infinity in floating-point arithmetic. The scalar determinant search is unresolved; the matrix search finds $-1, -2$ with complete counts. This demonstrates a representable-range advantage, not immunity to ill-conditioning or to overflow inside the supplied function.

For the wing, search the implicit boundary-value matrix of Chapter 12, not the meromorphic hybrid port matrix of Chapter 13. Keep the number of exact numerical pieces fixed during each search so that the dimension does not change:

```
wing = wing_model('goland'); U = 150; pieces = 2;
K = @(s) wing_matrix(s,U,wing,pieces);
box = [-40 15 1 320]; % above the Theodorsen branch cut
[r,i] = cmodes(wing_cdyn(U,wing,pieces),box, ...
    'AssumeAnalytic',true);
[d,j] = croots(@(s) det(full(K(s))),box, ...
    'AssumeAnalytic',true);
```

Table 14.1: Matrix search versus determinant roots and independent dry frequencies. Error is $|\lambda - \lambda_{\text{ref}}|/(1 + |\lambda_{\text{ref}}|)$.

Model	Pieces	Dimension	Count	Determinant error	Dry exact error
Dry	1	12	4	$2.25 \cdot 10^{-13}$	$2.25 \cdot 10^{-13}$
Dry	2	18	4	$2.82 \cdot 10^{-15}$	$5.63 \cdot 10^{-16}$
Dry	4	30	4	$2.60 \cdot 10^{-15}$	$4.22 \cdot 10^{-16}$
150 m/s	1	12	3	$5.88 \cdot 10^{-16}$	—
150 m/s	2	18	3	$9.21 \cdot 10^{-16}$	—
150 m/s	4	30	3	$2.50 \cdot 10^{-16}$	—

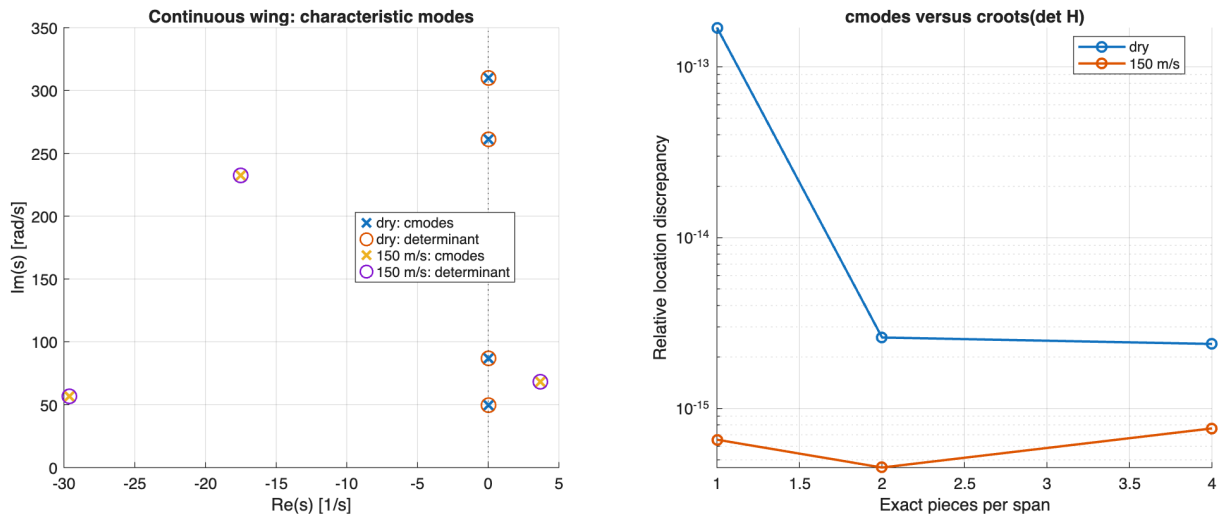


Figure 14.1: Continuous-wing modes from the matrix and determinant routes; agreement across fixed exact subdivisions.

All six runs have reconciled counts. The dry search window is $[-2, 2] \times [1, 320]$; the air-loaded window is the one in the listing (its left edge is kept well away from the mode near $-29.6 + 57i$). Subdivision changes the numerical representation, not the constant-strip equations. The improved dry frequency error with two/four pieces supports a conditioning benefit in this example, not a universal monotonic rule. Matrix/determinant agreement is not independent validation of the shared aerodynamic model; the physical checks remain those of Chapter 12. The reproducible script is `examples/continuous_wing/run_matrix_modes_comparison.m`.

14.5 The flutter mode shape

The right null vector at a mode holds the scaled states at every node of K . Propagating the root state z_0 with the exact strip propagators, $z(y) = e^{A(\lambda,U)y} z_0$ within each strip, gives the mode

shape at every span position without interpolation or modal basis (`wing_mode_shape`). At the flutter speed $U_f = 136.984$ m/s, `cmodes` returns the mode $\lambda = 70.0330i$, and the propagated shape satisfies the free-tip conditions to 5×10^{-14} relative to the root loads (Figure 14.2). At the tip the twist is as large as the bending motion, $b|\alpha|/|w| = 1.034$, and lags it by 62° . This phase difference lets the air do net work on the structure in each cycle, the mechanism of bending–torsion flutter. One and two exact pieces give the same ratio and phase.

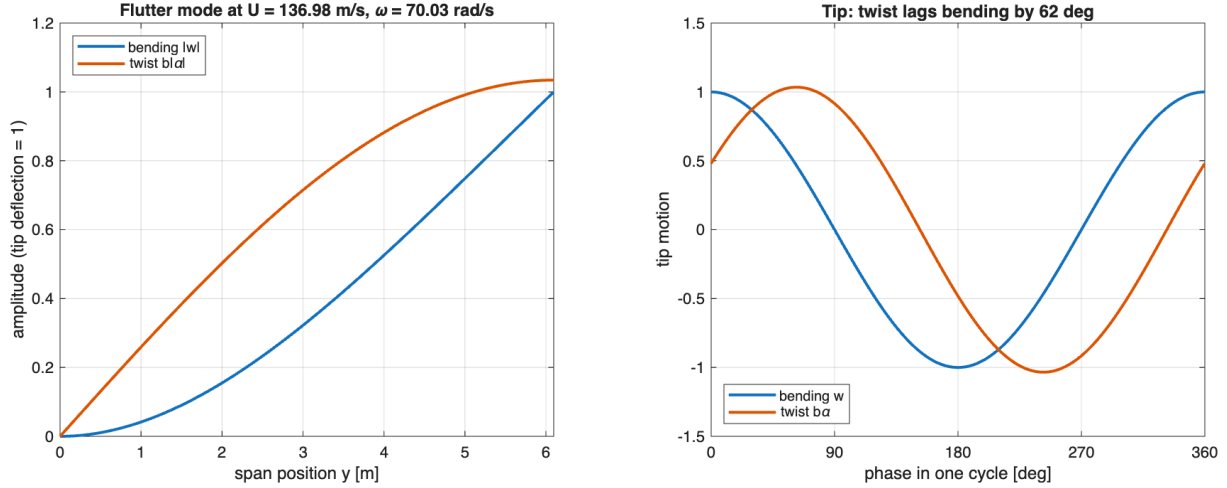


Figure 14.2: Flutter mode of the continuous Goland wing. Left: bending and twist amplitudes along the span (tip deflection normalized to one). Right: tip motion over one cycle.

This qualifies the contour-based M3 migration, not general MIMO spectra. Adequate sampling and analyticity remain assumptions; `certified` is always false. Work limits do not control allocations inside callbacks. Full contracts and deferred capabilities are documented in `docs/api/cmodes.md`.

Chapter 15

Validation and limitations

15.1 Test suite

The test suite (`buildtool test` and `buildtool docs`) has four layers.

Unit tests of the public interface: agreement of `croots`, `cpoles`, `czeros` and `cpzmap` with the core solver; polynomial input against `roots`; cancellations; warnings for exploratory and incomplete searches; input errors.

Regression tests of the numerical behavior: a triple root; three roots under scalings of $10^{\pm 100}$; a function without zeros; boundary roots and exhausted budgets that must not be reported as complete; heat, string and duct poles against closed forms; total and partial cancellation; an opaque handle with $Z - P = 0$; symbolic and LTI input; the delay tools (closed-form critical delay, delay-independent stability, Padé predictions, bound-based counts); the coupled beam (static compliance, limits, scaling, finite-element convergence); aeroelasticity (published flutter speeds, Theodorsen’s function, load assembly, stability counts, divergence, an independent harmonic flutter solver, Jones state space); and a function that raises an error outside the search region, which the solver must never evaluate there.

Baseline results of the research code from which the toolbox was extracted must be reproduced within a relative tolerance of 10^{-9} .

Documentation tests execute every MATLAB block of the README and of the Markdown documentation, the quick-start scripts, and check that code shown in tutorials is identical to the example files.

The time-response extension additionally checks analytical impulse/step responses, unstable and fractional models, direct transmission, external delays, ZOH/FOH input reconstruction, optional MATLAB `lsim` agreement, conjugacy, resource limits, and explicit unresolved inversion cases. Kernel reuse (`ckernel`) is compared with ordinary `clsim` calls for all inversion methods and both holds, with a delayed `tf` against the shifted delay-free `lsim` response (5×10^{-8}), and for superposition (10^{-12}); a counting evaluator confirms that reuse evaluates the model zero times.

15.2 Independent references in the studies

The studies of Chapters 7–10 compare the solver with closed forms (critical delay of P1; heat, string and duct spectra), modal formulas (Kelvin–Voigt beam), the direct critical-delay method and bound-based counts (stability maps), time-domain integration with `dde23`, a finite-element model (coupled beam), and published flutter benchmarks with an independent harmonic formulation (aeroelasticity). The references are never used as Newton seeds.

Chapter 11 adds method-of-steps, diffusion, rod modal and beam finite-element transient references, plus FFT/quadrature comparisons for the aeroelastic input/output channel. Chapter 12

compares the continuous wing with closed-form beam frequencies, an independent finite-element model with Hankel-form air loads (which converges to the continuous flutter speed at second order) and an analytic modal time series.

15.3 Limitations

- Scalar spectral searches, analytic matrix characteristic searches and MIMO zero-state responses are supported. General MIMO transfer poles and transmission zeros are not yet implemented. Defective matrix clusters remain explicitly unresolved.
- Rectangular regions only; roots outside the rectangle are not considered, and infinite spectra are not enumerated.
- Completeness is a numerical check under the analyticity assumption (Section 4.7), not an interval-arithmetic proof.
- Analyticity of function handles, and the location of branch and accumulation points, must be established by the user.
- Roots closer than the confirmation resolution are reported as clusters; near pole-zero pairs may be reported as cancelled.
- The time-delay tools assume one constant delay, real coefficients and (for the bound) a retarded system.
- Tested with MATLAB R2023b; other releases and Octave have not been tested.
- Time responses require a valid causal inversion domain, zero initial states and prehistory, and explicit input interpolation. Numerical refinement is not a certified tail or error bound.

Appendix A

Proofs

Proposition 7.2. Let $\operatorname{Re} s \geq 0$ and $F(s, T) = 0$. Then $|D(s)| = |N(s)|e^{-T \operatorname{Re} s} \leq |N(s)|$. With $r = |s|$, the triangle inequality gives $|D(s)| \geq r^m - \sum_{k < m} |d_k| r^k$ and $|N(s)| \leq \sum_{k \leq p} |n_k| r^k$, hence $h(r) = r^m - \sum_{k < m} |d_k| r^k - \sum_{k \leq p} |n_k| r^k \leq 0$. Since $p < m$, the coefficients of h have exactly one sign change, so by Descartes' rule of signs h has exactly one positive root R (if some coefficient other than the leading one is nonzero), and $h(r) > 0$ for $r > R$. Therefore $r \leq R$. Nothing depends on T . $\square$

Proposition 7.5. The signs of $\operatorname{Re}(ds/dT)$ and of $\operatorname{Re}[(ds/dT)^{-1}]$ coincide. From (7.6), using $e^{-sT}N(s) = -D(s)$ at a root,

$$\left(\frac{ds}{dT}\right)^{-1} = \frac{D'(s) + e^{-sT}N'(s)}{s e^{-sT}N(s)} - \frac{T}{s} = \frac{1}{s} \left[\frac{N'(s)}{N(s)} - \frac{D'(s)}{D(s)} \right] - \frac{T}{s}.$$

At $s = i\omega$, $-T/(i\omega)$ is purely imaginary and $\operatorname{Re}[\zeta/(i\omega)] = \operatorname{Im}(\zeta)/\omega$. For a real polynomial P , $\frac{d}{d\omega} \ln |P(i\omega)|^2 = 2 \operatorname{Re}[iP'(i\omega)/P(i\omega)] = -2 \operatorname{Im}[P'(i\omega)/P(i\omega)]$. Hence

$$\operatorname{Re}\left(\frac{ds}{dT}\right)^{-1} = \frac{1}{2\omega} \frac{d}{d\omega} [\ln |D(i\omega)|^2 - \ln |N(i\omega)|^2] = \frac{G'(\omega)}{2\omega |D(i\omega)|^2},$$

using $|D| = |N|$ at the crossing. Since $\omega > 0$ the claim follows. $\square$

Proposition 7.6. (a) On the imaginary axis $|P_n/Q_n| = 1$ by (7.9), so $D + NP_n/Q_n = 0$ requires $|D(i\omega)| = |N(i\omega)|$. (b) The phase condition is $\phi_n(\omega T) = -\theta_i - 2\pi k$; since ϕ_n decreases monotonically from 0 to $-n\pi$, it has a solution, which is unique, if and only if $\theta_i + 2\pi k < n\pi$. (c) With $R(z) = P_n(z)/Q_n(z)$ in place of e^{-z} , $F = D + R(sT)N$, $F_s = D' + RN' + TR'N$ and $F_T = sR'N$. Using $RN = -D$ and $\rho = R'/R$,

$$\left(\frac{ds}{dT}\right)^{-1} = -\frac{F_s}{F_T} = \frac{1}{s[-\rho(sT)]} \left[\frac{N'}{N} - \frac{D'}{D} \right] - \frac{T}{s}.$$

On the imaginary axis $R(iy) = e^{i\phi_n(y)}$ gives $\rho(iy) = \phi'_n(y)$, real and negative; for the exact delay $\rho \equiv -1$. The positive factor $-1/\phi'_n$ does not change the sign of the real part, which is therefore $\operatorname{sign} G'(\omega_i)$ as in Proposition 7.5. $\square$

Error of the derivative formula of Section 4.2. For analytic f , $\frac{f(z+h)-f(z-h)}{2h} = f' + \frac{h^2}{6} f''' + O(h^4)$ and $\frac{f(z+ih)-f(z-ih)}{2ih} = f' - \frac{h^2}{6} f''' + O(h^4)$. Their mean is the formula of Section 4.2, with error $O(h^4)$. $\square$

Appendix B

Reproducibility

All numbers, tables and figures of this manual are produced by the code distributed with ContourRoots. From the repository folder:

```
buildtool examples % time-delay, distributed and coupled-beam studies
buildtool manual   % documentation figures and this PDF
buildtool test     % unit and regression tests
buildtool docs     % documentation tests
```

File	Content
matlab/complex_spectrum.m	core solver (Chapter 4)
matlab/private/spectrum_*.m	counts, Newton, subdivision, input adapters
matlab/croots.m, cpoles.m, czeros.m, cpzmap.m, ndpair.m	public interface
matlab/delay/*.m	time-delay and Padé tools (Chapter 7)
examples/time_delay/run_delay_study.m	Chapter 7; calls <code>run_pade_critical_study</code> and <code>run_pade_pitfalls</code>
examples/distributed_systems/run_nonrational_examples.m	Chapter 8
examples/coupled_beam/run_coupled_beam_study.m	Chapter 9
examples/aeroelasticity/run_aeroelastic_study.m	Chapter 10
examples/time_response/run_time_response_study.m	Chapter 11
examples/models/*.m	distributed and coupled-beam models, finite elements
tools/make_doc_figures.m	figures of the Markdown documentation
manual/	this manual (L ^A T _E X sources and TikZ figures)

The results in this version were computed with MATLAB R2023b. Generated data are written to `output/`, which is not versioned; the studies take a few minutes.

Authorship and AI assistance

ContourRoots was created and is maintained by Flávio Luiz Cardoso-Ribeiro. It was developed extensively with AI coding assistance, using Claude Code (Anthropic) and Codex (OpenAI), including code generation, documentation and tests; Codex was also used for an independent review of version 0.1.0. Problem formulation, project direction and release decisions are led by the maintainer. Numerical validation and known limitations are documented in Chapter 15, in the Markdown documentation (`docs/validation.md`, `docs/diagnostics_and_limits.md`) and in `ROADMAP.md`.

Bibliography

- [1] Lars V. Ahlfors. *Complex Analysis*. McGraw-Hill, New York, 3 edition, 1979.
- [2] Uri M. Ascher, Robert M. M. Mattheij, and Robert D. Russell. *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations*, volume 13 of *Classics in Applied Mathematics*. SIAM, Philadelphia, 1995.
- [3] George A. Baker and Peter Graves-Morris. *Padé Approximants*, volume 59 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 2 edition, 1996.
- [4] J. R. Banerjee. Modal analysis of sailplane and transport aircraft wings using the dynamic stiffness method. *Journal of Physics: Conference Series*, 721:012005, 2016. doi: 10.1088/1742-6596/721/1/012005.
- [5] Richard Bellman and Kenneth L. Cooke. *Differential-Difference Equations*. Academic Press, New York, 1963.
- [6] Dimitri Breda, Stefano Maset, and Rossana Vermiglio. Pseudospectral differencing methods for characteristic roots of delay differential equations. *SIAM Journal on Scientific Computing*, 27(2):482–495, 2005. doi: 10.1137/030601600.
- [7] Dimitri Breda, Stefano Maset, and Rossana Vermiglio. *Stability of Linear Delay Differential Equations: A Numerical Approach with MATLAB*. Springer, 2015. doi: 10.1007/978-1-4939-2107-2.
- [8] A. A. Cal. *A Unified Approach to Flutter, Dynamic Stability and Response Analysis of High Aspect Ratio Aircraft*. PhD thesis, City University London, 1992. URL <https://openaccess.city.ac.uk/id/eprint/29042/>.
- [9] Kenneth L. Cooke and P. van den Driessche. On zeroes of some transcendental equations. *Funkcialaj Ekvacioj*, 29:77–90, 1986.
- [10] Ruth F. Curtain and Kirsten Morris. Transfer functions of distributed parameter systems: A tutorial. *Automatica*, 45(5):1101–1116, 2009. doi: 10.1016/j.automatica.2009.01.008.
- [11] F. R. de Hoog, J. H. Knight, and A. N. Stokes. An improved method for numerical inversion of laplace transforms. *SIAM Journal on Scientific and Statistical Computing*, 3(3):357–366, 1982. doi: 10.1137/0903022.
- [12] L. M. Delves and J. N. Lyness. A numerical method for locating the zeros of an analytic function. *Mathematics of Computation*, 21(100):543–560, 1967.
- [13] John W. Edwards. Unsteady aerodynamic modeling and active aeroelastic control. Technical Report NASA-CR-148019, Stanford University / NASA, 1977. URL <https://ntrs.nasa.gov/citations/19780002074>.
- [14] Martin Goland. The flutter of a uniform cantilever wing. *Journal of Applied Mechanics*, 12(4):A197–A208, 1945.

- [15] Keqin Gu, Vladimir L. Kharitonov, and Jie Chen. *Stability of Time-Delay Systems*. Birkhäuser, 2003. doi: 10.1007/978-1-4612-0039-0.
- [16] Jack K. Hale and Sjoerd M. Verduyn Lunel. *Introduction to Functional Differential Equations*, volume 99 of *Applied Mathematical Sciences*. Springer, 1993. doi: 10.1007/978-1-4612-4342-7.
- [17] Christoph Kaiser and David Quero. Effect of aerodynamic damping approximations on aeroelastic eigensensitivities. *Aerospace*, 9(3):127, 2022. doi: 10.3390/aerospace9030127.
- [18] Piotr Kowalczyk. GRPF: Global complex roots and poles finding algorithm. <https://github.com/PioKow/GRPF>. Accessed 2026-09-26.
- [19] Peter Kravanja and Marc Van Barel. *Computing the Zeros of Analytic Functions*, volume 1727 of *Lecture Notes in Mathematics*. Springer, Berlin, 2000.
- [20] Wim Michiels and Silviu-Iulian Niculescu. Characterization of delay-independent stability and delay interference phenomena. *SIAM Journal on Control and Optimization*, 45(6): 2138–2155, 2007. doi: 10.1137/050641223.
- [21] Wim Michiels and Silviu-Iulian Niculescu. *Stability, Control, and Computation for Time-Delay Systems: An Eigenvalue-Based Approach*. SIAM, 2 edition, 2014. doi: 10.1137/1.9781611973631.
- [22] Nejat Olgac and Rifat Sipahi. An exact method for the stability analysis of time-delayed linear time-invariant (LTI) systems. *IEEE Transactions on Automatic Control*, 47(5): 793–797, 2002. doi: 10.1109/TAC.2002.1000275.
- [23] Robert Parini. cxroots: A Python module to find all the roots of a complex analytic function within a given contour. <https://github.com/rparini/cxroots>. Accessed 2026-09-26.
- [24] III Perry, Boyd. Re-computation of numerical results contained in NACA report no. 496. Technical Report NASA/TP-2015-218765, NASA Langley Research Center, 2015. URL <https://ntrs.nasa.gov/citations/20150014000>.
- [25] L. F. Shampine and S. Thompson. Solving DDEs in MATLAB. *Applied Numerical Mathematics*, 37(4):441–458, 2001. doi: 10.1016/S0168-9274(00)00055-6.
- [26] Theodore Theodorsen. General theory of aerodynamic instability and the mechanism of flutter. Technical Report Report 496, NACA, 1935.
- [27] Tomáš Vyhlídal and Pavel Zitek. Mapping based algorithm for large-scale computation of quasi-polynomial zeros. *IEEE Transactions on Automatic Control*, 54(1):171–177, 2009.
- [28] K. Walton and J. E. Marshall. Direct method for TDS stability analysis. *IEE Proceedings D – Control Theory and Applications*, 134(2):101–107, 1987.